

Compositional Correctness via Directed Containers: A Complete Technical Dossier

MacBeth
(with N. Ghani, R. Langer)
Kodamai

2026-09-24

Draft — all four pillars assembled (Theory, Applications, Formalisation, Impact); a handful of Applications chapters remain honest stubs, and the tax chapter is illustrative-only. See the Honest Ledger.

Abstract

This dossier collects the container / directed-container research programme into a single document, organised around one thesis: the correctness of a composite computational system is governed by a single, checkable equation,

$$\text{a composite is correct} \iff (\mathbf{L}) \wedge [\omega] = 0 \in H^2.$$

The document has four pillars — Theory, Applications, Formalisation, Impact — and an honest ledger of what is proved, what is machine-verified, and what remains open. This is a *skeleton*: the front matter is written in full, and each downstream section names the source file(s) from which its body will be assembled.

Contents

How to read this document	5
1 Overview: One Thesis	6
1.1 The organizing equation	6
1.2 Why this is a foundation and not a slogan	6
1.3 The four movements	7
2 The Honest Ledger	8
I Theory	9
3 Preliminaries: the machinery	10
3.1 Representable functors and the Yoneda lemma	10
3.2 Density, the extension functor, and Fam(Setop)	11
3.3 Directed containers and the composition operator $\triangleleft$	12
3.4 Day convolution and the four monoidal structures	13
3.5 Cohomological conventions	14
4 The equivalence chain	15
4.1 One object, four dialects	15
4.2 Containers and the extension	15
4.3 Directed containers	16
4.4 Directed containers are comonads (M2)	16
4.5 The object dictionary: directed containers are small categories (M4)	17
4.6 Polynomial comonoids (M5, bridge)	17
4.7 The chain, assembled; and the variance caveat	18
5 Zappa–Szép products and distributive laws as the composition tool	20
5.1 Composition is a weld	20
5.2 The distributive law: the algebra of the weld	21
5.3 The recognition problem and the free-presheaf lemma	22
5.4 The pairwise criterion	23
5.5 A genuinely pairwise obstruction	24
5.6 The bridge: orchestration composites are Zappa–Szép products	25

6	The correctness criterion $(L) \wedge [\omega] = 0 \in H^2$	27
6.1	The thesis equation	27
6.2	The global condition is a degree-two class	27
6.3	The minimal faithful instance: $[\omega] = \varepsilon$, machine-checked	28
6.4	Two ω -sites: degree-1 descent versus degree-2 Schreier	29
7	The H^2-cluster of obstruction theories	30
7.1	Three homes on one object	30
7.2	The refuted unifier, and the honest umbrella	31
7.3	The crown: the cluster is filtered, not a product	32
8	Change-of-base, $\text{Fam}(\mathcal{C}^{\text{op}})$ generality, and the $\{\text{Id}\}$ classification	33
8.1	The base as a degree of freedom	33
8.2	Positive suite I: m -containers, codensity, polynomiality	34
8.3	Positive suite II: the change-of-base census	35
8.4	Positive suite III: the four monoidal structures	36
8.5	Positive suite IV: when does substitution exist over a base?	37
8.6	The linear wall: Vec is neither full nor faithful	38
8.7	The $\{\text{Id}\}$ collapse	39
9	The set-theoretic frontier: Vec-admissibility and the <i>Gap-S</i> arc	41
9.1	From admissibility to a module	41
9.2	The first count: false under CH	43
9.3	The sharp invariant: $\mu_S(M) = \mathfrak{d}$	43
9.4	The sharpened refutation, and why it matters	44
9.5	The residual, honestly	45
II	Applications	47
10	GA / island-model composition	48
10.1	Migration topology is a distributive law	48
10.2	Level 1: the reachability comonad	49
10.3	Level 2: composition via Zappa–Szép — the weld condition	49
10.4	Diversity dynamics and the laxator as obstruction	50
11	Blockchain: re-entrancy as a failed distributive law	52
11.1	The idea: re-entrancy is a failed distributive law	52
11.2	Smart contracts as polynomial coalgebras	53
11.3	Composition, interleaving, and the Zappa–Szép weld	53
11.4	The obstruction is a degree-two class, and it is machine-checked	54
11.5	The EVM instantiation: the DAO and the token bit	56
12	Tax computation (Catala)	58
12.1	The categorical picture (speculative)	58
12.2	Where the correctness equation would enter (statement of intent)	59

13 Agent orchestration: the harness is a worker	60
13.1 The harness is a worker	60
13.2 Tool-calling protocols are directed containers	62
13.3 The workers type hierarchy	63
13.4 Effects and coeffects: the reverse-entwining arrow	64
14 Security audits as failed distributive laws	66
14.1 From the re-entrancy weld to the audit desk	66
14.2 State divergence as a missing distributive law	66
14.3 A taxonomy indexed by which law fails	67
14.4 Status and honest scope	68
15 Supply chains	69
15.1 The base chain as a directed container	69
15.2 Two flows sharing a warehouse	70
15.3 Why $\mathbb{Z}/n$, and not just a parity bit	72
15.4 Olog merges: the same machine at $n = 2$	73
15.5 Scope and honesty	73
III Formalisation	75
16 Milestones M1–M6: status	76
16.1 The milestone ledger	76
16.2 The verified core: M1–M4	76
16.3 The open frontier: M5 and M6 (Honest Ledger item 3)	78
16.4 Adjacent verified structure	78
17 The sorry-free Lean corpus	79
17.1 What “lean-verified” means, and what it does not	79
18 H^2-engines and the free/cofree comonad in Lean	83
18.1 The free monad as a $\triangleleft$ -monoid	83
18.2 The universal property of the free monad	84
18.3 The unified cyclic H^2 -engine: $H^2(C_n; \mathbb{Z}/m) \cong \mathbb{Z}/\gcd(n, m)$	84
18.4 The two superseded engines: norm and fibre	85
19 Change-of-base adjunction and collapse theorems in Lean	87
19.1 The left adjoint $F_q \dashv (-) \triangleleft q$	87
19.2 Parameter naturality: F is contravariant in q	88
19.3 The Vec biproduct collapse, and the non-faithfulness hedge	88
19.4 The QPF abs/repr collapse: $\text{Bag}_2 = \text{Sym}^2$	89
19.5 The enriched full-and-faithful collapse $\mathcal{CL} \cap \mathcal{UN} = \{\text{Id}\}$ (narrowed scope)	89
IV Impact	91
20 Where compositional correctness has economic consequences	92
20.1 The through-line: a wrong weld costs money	92

20.2	Three failures, one obstruction	92
20.3	Why the specialisation is principled, not post-hoc	93
20.4	The decision procedure and its economic reading	94
21	The AI + Lean pipeline: honesty and trust-grade methodology	95
21.1	The problem this solves	95
21.2	The trust-grade ladder	95
21.3	The registry	96
21.4	The Lean statement self-audit	96
21.5	Forcing functions: honesty made structural	97
21.6	“Correction is the theorem”	97
22	External-publication status	99

How to read this document

This is a document assembled in stages, and all four pillars are now written. Part I (Theory) is complete: the preliminaries (Chapter 3); the equivalence chain (Chapter 4); Zappa–Szép products and distributive laws (Chapter 5); the correctness criterion (Chapter 6); the H^2 -cluster (Chapter 7); change-of-base and the $\{\text{Id}\}$ classification (Chapter 8); and the set-theoretic frontier (Chapter 9). Part II (Applications) runs from GA / island models (Chapter 10), through the flagship blockchain re-entrancy chapter (Chapter 11), security audits (Chapter 14) and agent orchestration (Chapter 13), to supply chains (Chapter 15) and tax (Chapter 12). Part III (Formalisation) records the milestone ledger (Chapter 16), the sorry-free corpus (Chapter 17), and the two machine-checked pillars — the H^2 -engines and free/cofree comonad (Chapter 18) and the change-of-base adjunction and collapse theorems (Chapter 19). Part IV (Impact) gives the economic reading (Chapter 20), the AI + Lean pipeline (Chapter 21), and the external-publication status.

Not every chapter carries the same weight, and the document says so at a glance. Each downstream chapter is graded, and a few are deliberately *stubs* or *gaps*: the tax chapter (Chapter 12) has *no* formal artifact and is framed as illustrative-only (Honest Ledger item 2); supply chains (Chapter 15) and part of the security-audit chapter rest on registry computations whose full written bodies are still owed; and milestones M5–M6 are open / conjectural rather than proved (Honest Ledger item 3). Every headline claim carries its trust grade inline, so a reader can tell load-bearing results from promissory ones without leaving the page.

The trust grades. Throughout, claims carry a trust grade drawn from the programme’s standard ladder, in increasing order of external verifiability:

speculative < computed < proved < peer-reviewed < published < lean-verified.

Here *computed* means checked on explicit small cases or by symbolic computation; *proved* means a paper proof we believe complete; *peer-reviewed* means refereed internally (currently: Rick, a fellow agent); *published* means externally published (currently: nothing — see the Honest Ledger); and *lean-verified* means machine-checked, sorry-free, in Lean 4/Mathlib. The ordering places *lean-verified* at the top because a sorry-free proof term is the highest available warrant — with the essential caveat, stated plainly in Part IV, that “verified” certifies the proof, not that the *statement* is the one you meant.

Chapter 1

Overview: One Thesis

1.1 The organizing equation

The claim that unifies this programme is that the composition of heterogeneous computational systems is governed by a single equation, and that the equation is *checkable*:

$$\boxed{\text{a composite is correct} \iff (\mathbf{L}) \wedge [\omega] = 0 \in H^2}$$

The two conjuncts are independent and have different characters.

(L) — **the local condition.** Each part of the system is a *directed container* — equivalently, a small category, equivalently a polynomial comonoid. Condition (L) says that each part composes coherently *with itself*: its identities and its associative composition are the counit and comultiplication of a comonoid structure. This is the content of the equivalence chain

$$\mathbf{Cont} \simeq \mathbf{DCont} \simeq (\text{polynomial comonoids}) \simeq \mathbf{Cat},$$

which four independent lines of work reach from four directions. (L) is a condition on the pieces, in isolation.

$[\omega] = 0$ — **the global condition.** Given local pieces, the question is whether they *weld* into a single coherent whole. The weld is a Zappa–Szép product, equivalently a distributive law between the pieces’ comonads/monads. Such a weld exists precisely when a degree-two cohomology class $[\omega] \in H^2$ vanishes. Where (L) is about each part alone, $[\omega] = 0$ is about their interaction: it is the obstruction to gluing.

Everything in the programme is one of three things: a *proof* that this equation is the right one and holds where claimed; an *application* in which a real system’s failure mode is exactly $[\omega] \neq 0$; or a *formalisation* that makes the check machine-executable.

1.2 Why this is a foundation and not a slogan

The equation is not imposed; it is discovered at the meeting point of three independent research programmes. Container theory (Nottingham: Abbott–Altenkirch–Ghani), directed containers (Tallinn: Ahman–Uustalu), and the **Poly** programme (Topos Institute: Spivak) developed substantially the same mathematics in three dialects, and all three converge on the equivalence chain that furnishes

(L). Convergence is the evidence: when three groups arrive at the same object from three directions, the object is real, not an artifact of notation.

What the present programme adds is the *composition* question none of the three answered head-on: when do two of these objects compose, and what obstructs it? The answer is the second conjunct. The pairwise Zappa–Szép criterion decomposes composability into a local factor (L) and a global factor (G), and $(G) \Leftrightarrow [\omega] = 0 \in H^2$. That identification — composability as the vanishing of a concrete cohomological obstruction — is the thesis of the dossier.

1.3 The four movements

The document proceeds in four movements, ordered so that each pillar earns the next.

I. Theory — the equation and its domain of validity. The equivalence chain (the (L) side); Zappa–Szép products and distributive laws as *the* composition tool, giving the pairwise criterion $(L) \wedge (G)$ with $(G) \Leftrightarrow [\omega] = 0$; the H^2 -cluster of three genuinely distinct obstruction theories on $\mathbf{DCont} \simeq \mathbf{Cat}$; the change-of-base / $\mathbf{Fam}(\mathcal{C}^{\text{op}})$ generality question, with the sharp negative that the self-enriched full-and-faithful semantics collapses to $\{\text{Id}\}$; and, at the frontier, the set-theoretic boundary of the equation’s domain — the Vec-admissibility / Gap-S arc, where the composability boundary of a categorical semantics turns out to be a cardinal characteristic of the continuum. Theory ends on the live edge, not a solved one.

II. Applications — where $[\omega] \neq 0$ costs money. The same mathematics across five domains: genetic-algorithm island-model composition, blockchain re-entrancy as a failed distributive law, security audits as failed distributive laws, agent orchestration (the harness as a worker), and tax computation. Real systems fail exactly when the weld obstruction is nonzero.

III. Formalisation — the check is executable. The Lean milestones and the sorry-free corpus: what is machine-verified versus proved on paper, stated honestly. This is what distinguishes the grant from hand-waving.

IV. Impact — and a note on honesty. Where compositional correctness has economic consequences, and the AI + Lean-pipeline honesty methodology: the discipline of separating “verified” from “correctly stated” and “reusable,” which is the programme’s real differentiator.

Chapter 2

The Honest Ledger

This chapter is deliberately near the front, not buried in an appendix. A dossier that claims a unifying equation must be equally clear about where the equation is unproved, where the applications lack artifacts, and what has and has not been externally validated. Four items:

1. **The Gap-S / Vec-admissibility wall is OPEN.** Whether **Vec** is $\triangleleft$ -admissible reduces to a projectivity question over the full linear ring $S = \text{End}_k(E)$, and the module M in question has generation number $\mu_S(M) = \mathfrak{d}$, the dominating number. We have proved the conjecture (Conj. 6.2) is *false* under CH, under MA, and in the Cohen model, via cardinal counting (M projective $\Rightarrow 2^{\mathfrak{d}} = 2^{\aleph_0}$, so any model with $2^{\aleph_0} < 2^{\mathfrak{d}}$ refutes it). But the *full ZFC status is unproved*: the verdict currently leans *independent*, and the residual case is the random-real model ($\mathfrak{d} = \aleph_1 < \mathfrak{c}$). We do not yet know whether M is projective there. This is the programme’s deepest open problem, and it is open.
2. **Tax computation has NO formal artifact.** The Catala / tax-computation application (§II.3) is currently a design sketch — ontology DAG, scope composition, comonadic event-sourced projections — with no proof, no Lean, and no running code. It is framed throughout as illustrative-only.
3. **Equivalence-chain milestones M5–M6 are not proved — not even on paper.** Milestones M1–M4 (with converses M2b, M3b) are lean-verified. M5 and M6 are *neither* formalised *nor* proved on paper: the registry grades M5 (enriched **DCont** = **Cof**) *unclassified*, recording that Clarke–Di Meglio state the enriched statement *open*, and grades M6 (∞ -**DCont** = **Cat**, Kan Chen’s Conjecture 7.2) *speculative*. So the formalised chain is exactly the non-enriched, 1-categorical core M1–M4; the enriched and ∞ -categorical extensions are frontier, not ledger. Any claim that “the equivalence programme is fully formalised” must be read as scoped to M1–M4 (see Chapter 16).
4. **Nothing is externally published.** Under the ACM authorship rule an AI agent cannot be sole author, and the external submission queue is held by Robin. The strongest current grade anywhere in this dossier is *peer-reviewed*, meaning refereed internally by Rick (a fellow agent) — e.g. the holonomy-composition / Zappa–Szép bridge and the m -containers result. No claim here is *published* in the external sense.

A fifth, smaller honesty note carried from the Theory pillar: the conjectured *protomodular unifier* of the three H^2 obstruction theories was *refuted*. The three theories are a genuinely distinct structured family, not one theorem wearing three hats.

Part I

Theory

Chapter 3

Preliminaries: the machinery

This chapter fixes notation and recalls the standard tools the rest of the dossier leans on, in one place, so that the later chapters can move without pausing to reintroduce them. *Almost everything here is standard* and is collected for self-containedness: the Yoneda lemma is Yoneda’s and Mac Lane’s; density and the coend/Kan calculus follow Mac Lane [3], Riehl [4], and Loregian [2]; Day convolution is Day’s [1]. The only programme-internal content is the *container reading* of each tool — the translation into shapes and positions — which is *proved* and is what makes the later theorems computable. Two facts of Ghani’s are foregrounded because the composition machinery uses them as black boxes: that the container-extension functor *is* a left Kan extension along the representable embedding (§3.2), and that the closed structure of a Day category is *forced on representables and extended by density* (§3.4).

The reader who wants a map: §3.1–3.2 (representables, Yoneda, density, the extension functor) are used everywhere the (L) side appears — the equivalence chain of Chapter 4, and, in an essential and less obvious way, the module reduction of the set-theoretic frontier (Chapter 9), where $\text{Nat}(F, -) \cong \text{Hom}_S(F(E), -)$ is Yoneda in disguise. Section 3.3 fixes the directed-container and composition ($\triangleleft$) notation carried by the Zappa–Szép chapters. Section 3.4 (Day convolution) supports the change-of-base chapter (Chapter 8). Section 3.5 fixes the cohomological conventions — H^2 , the class $[\omega]$, the Lyndon–Hochschild–Serre transgression — on which the correctness criterion and the H^2 -cluster of Chapter 7 stand.

Conventions. **Set** is the category of sets and functions. For a locally small category $\mathcal{C}$ we write $\mathcal{C}(a, b)$ for its hom-set; $\text{ob } \mathcal{C}$ for its objects. Functors are covariant unless a superscript op says otherwise. A *presheaf* on $\mathcal{C}$ is a functor $\text{Cop} \rightarrow \mathbf{Set}$; a *copresheaf* is a functor $\mathcal{C} \rightarrow \mathbf{Set}$. We write $\cong$ for canonical (natural) isomorphism and are careful to say *which* isomorphism when it matters.

3.1 Representable functors and the Yoneda lemma

Every construction in the dossier eventually reduces to a hom-set. The representable functors are the functors that *are* a hom-set with one argument held fixed, and they generate everything else.

Definition 3.1 (Representable copresheaf). Fix a set A . The *covariant representable* at A is

$$y^A := \mathbf{Set}(A, -) : \mathbf{Set} \longrightarrow \mathbf{Set}, \quad y^A(X) = \mathbf{Set}(A, X) = X^A,$$

sending $f : X \rightarrow Y$ to post-composition $y^A(f) = f \circ (-)$. A copresheaf $F : \mathbf{Set} \rightarrow \mathbf{Set}$ is *representable* if $F \cong y^A$ for some A , the *representing object*.

Remark 3.2 (The container reading). A *container* is a pair (S, P) of a set S of *shapes* and a family $P: S \rightarrow \mathbf{Set}$ of *position sets*, with *extension*

$$\llbracket S, P \rrbracket : \mathbf{Set} \rightarrow \mathbf{Set}, \quad \llbracket S, P \rrbracket(X) = \sum_{s \in S} \mathbf{Set}(P(s), X).$$

The *representable container* has a single shape and A positions, $(S, P) = (1, A)$, and $\llbracket 1, A \rrbracket(X) = \mathbf{Set}(A, X) = y^A(X)$. So representable copresheaves are exactly the extensions of single-shape containers, and the coproduct in the general formula is a coproduct of *representables* indexed by shapes. This is the whole story, phrased below first as a Kan extension (§3.2) and then as a Day convolution (§3.4). The same extension formula, over a base $\mathcal{C}$ other than $\mathbf{Set}$, is the object of study of Chapter 9.

Definition 3.1 is uniform in A : a function $a: A' \rightarrow A$ induces, by *pre-composition*, a natural transformation $y^A \Rightarrow y^{A'}$. Thus $A \mapsto y^A$ is a functor $y^{(-)}: \mathbf{Setop} \rightarrow [\mathbf{Set}, \mathbf{Set}]$, the *Yoneda embedding*. It is genuinely an embedding, as the Yoneda lemma shows.

Theorem 3.3 (Yoneda lemma; standard). *For any functor $F: \mathbf{Set} \rightarrow \mathbf{Set}$ and any set A there is a bijection*

$$\Theta_{F,A} : \mathbf{Nat}(y^A, F) \xrightarrow{\cong} F(A), \quad \Theta(\alpha) = \alpha_A(\mathrm{id}_A),$$

natural in F and A .

Proof. The inverse sends $x \in F(A)$ to $\Psi(x)$ with $\Psi(x)_X(g) = F(g)(x)$ for $g: A \rightarrow X$; naturality of $\Psi(x)$ is functoriality, $F(h)(F(g)(x)) = F(h \circ g)(x)$. Then $\Theta(\Psi(x)) = F(\mathrm{id}_A)(x) = x$. Conversely, for $\alpha: y^A \Rightarrow F$ and $x = \alpha_A(\mathrm{id}_A)$, naturality of α at $g: A \rightarrow X$ chased on id_A gives $\alpha_X(g) = F(g)(x) = \Psi(x)_X(g)$, so $\Psi(x) = \alpha$. Naturality in F and A is the same chase. $\square$

Corollary 3.4 (Full faithfulness). *$\mathbf{Nat}(y^A, y^B) \cong \mathbf{Set}(B, A)$, naturally; the Yoneda embedding $y^{(-)}: \mathbf{Setop} \rightarrow [\mathbf{Set}, \mathbf{Set}]$ is fully faithful. Natural transformations between representables are exactly functions, in the opposite direction.*

Corollary 3.4 is the reason the container-morphism convention (Notation 3.6) has position maps running *backwards*: a map of representables is a backwards map of representing objects.

3.2 Density, the extension functor, and $\mathbf{Fam}(\mathbf{Setop})$

The Yoneda lemma says representables see everything. Density says they *are* everything, once colimits are allowed.

Theorem 3.5 (Density / co-Yoneda; standard). *Every functor $F: \mathbf{Set} \rightarrow \mathbf{Set}$ is canonically a colimit of representables:*

$$F \cong \int^{A \in \mathbf{Set}} F(A) \bullet y^A,$$

where $F(A) \bullet y^A$ is the $F(A)$ -fold copower of y^A . Equivalently F is the colimit of the representables $y^A \xrightarrow{\Psi(x)} F$ over the category of elements (A, x) , $x \in F(A)$.

Reference. The density formula, dual to Yoneda; Riehl [4, §6.2], Loregian [2, Ch. 2]. Its universal property is Theorem 3.3 read backwards: a cocone from $F(A) \bullet y^A$ to G is a family $F(A) \rightarrow \mathbf{Nat}(y^A, G) \cong G(A)$ natural in A , i.e. a natural transformation $F \Rightarrow G$. $\square$

The representables are thus a *dense generator* of $[\mathbf{Set}, \mathbf{Set}]$: two constructions that agree on representables and preserve the relevant colimits agree everywhere. This uniformity is the workhorse of the whole (L) side.

Notation 3.6 (**Cont** and $\mathbf{Fam}(\mathbf{Setop})$). A morphism of containers $(S, P) \rightarrow (T, Q)$ is a pair (u, v) with $u: S \rightarrow T$ a map of shapes and v a family $v_s: Q(us) \rightarrow P(s)$ of position maps *going backwards*. This is exactly an S -indexed family of objects of $\mathbf{Setop}$ together with a map of index sets and, over it, morphisms in $\mathbf{Setop}$; hence

$$\mathbf{Cont} \cong \mathbf{Fam}(\mathbf{Setop}),$$

the *free coproduct completion* of $\mathbf{Setop}$ — the universal category with small coproducts containing $\mathbf{Setop}$. Under $\llbracket - \rrbracket$, a family $(P(s))_{s \in S}$ maps to $\sum_{s \in S} y^{P(s)}$.

Theorem 3.7 (Extension is a left Kan extension; Ghani’s observation, proved). *Let $\mathcal{A}$ be a category and $Y: \mathcal{A} \rightarrow \mathbf{Fam}(\mathcal{A})$ the singleton embedding $a \mapsto (a)$. For any $F: \mathcal{A} \rightarrow \mathcal{E}$ into a category with small coproducts, the coproduct-preserving extension $\widehat{F}((a_s)_{s \in S}) = \coprod_{s \in S} F(a_s)$ is the left Kan extension $\mathbf{Lan}_Y F$, and it preserves representables ($\widehat{F} \circ Y \cong F$). Specialising $\mathcal{A} = \mathbf{Setop}$, $F = y^{(-)}$, and $\mathcal{E} = [\mathbf{Set}, \mathbf{Set}]$, the container-extension functor is that Kan extension:*

$$\llbracket - \rrbracket = \mathbf{Lan}_Y y^{(-)}, \quad \llbracket S, P \rrbracket = \coprod_{s \in S} y^{P(s)}.$$

Proof. By the pointwise coend formula [3, Ch. X] and the description of morphisms out of a singleton in a free coproduct completion, $\mathbf{Fam}(\mathcal{A})(Ya, (a_s)_s) \cong \coprod_s \mathcal{A}(a, a_s)$, so

$$(\mathbf{Lan}_Y F)((a_s)_s) \cong \int^a \left(\coprod_s \mathcal{A}(a, a_s) \right) \bullet F(a) \cong \coprod_s \int^a \mathcal{A}(a, a_s) \bullet F(a) \cong \coprod_s F(a_s),$$

the inner coend being density (Theorem 3.5) at a_s . Preservation of representables is the one-term case $S = 1$. For $\mathcal{A} = \mathbf{Setop}$ and $F = y^{(-)}$ this reads $\llbracket S, P \rrbracket = \coprod_s y^{P(s)}$, whose value at X is $\sum_s \mathbf{Set}(P(s), X) = \llbracket S, P \rrbracket(X)$ (Remark 3.2). A size remark: although $\mathbf{Setop}$ is not small, part of the statement is really the universal property of the free coproduct completion, which holds verbatim, and the coproducts landed on are small since shape sets are. $\square$

Two consequences are used without reproof throughout Part I: (i) any structure on $\mathbf{Setop}$ transported to **Cont** need only be defined on singletons and extended by coproducts, and the result is automatically the Kan extension; (ii) to check a functor out of **Cont** agrees with $\llbracket - \rrbracket$ it suffices to check agreement on representables plus preservation of coproducts. “A container is a coproduct of representables” is not a slogan but a computational method.

3.3 Directed containers and the composition operator $\triangleleft$

The extension functors above are the raw material; the (L) side of the correctness criterion asks which of them compose *with themselves* coherently. That is the content of directed containers, whose full theory — including the equivalence chain — is Chapter 4. Here we fix only the notation the composition chapters need.

The category **Cont** carries a monoidal structure $\triangleleft$, *composition*, that on extensions is composition of endofunctors: $\llbracket p \triangleleft q \rrbracket \cong \llbracket p \rrbracket \circ \llbracket q \rrbracket$. Concretely, with $p = (S, P)$ and $q = (T, Q)$, a shape of $p \triangleleft q$ is a shape $s \in S$ together with a $P(s)$ -indexed family of shapes of q , and a position is a dependent pair — the “a shape, then a shape at each position” law that makes $\triangleleft$ the substitution tensor. The unit is the identity container $y^1 = (1, 1)$, whose extension is the identity functor.

Definition 3.8 (Directed container, informally). A *directed container* is a container p equipped with the structure of a $\triangleleft$ -comonoid: a counit $p \rightarrow y^1$ and a comultiplication $p \rightarrow p \triangleleft p$ satisfying the comonoid laws. Equivalently — and this is the equivalence chain — it is a small category, a polynomial comonoid in **Poly**, or a cofunctor domain, one object presented four ways. We write **DCont** for the category of directed containers.

Theorem 3.9 (The equivalence chain; assembled in Chapter 4). *There are equivalences of categories*

$$\mathbf{Cont} \supseteq \mathbf{DCont} \simeq (\text{polynomial comonoids in } \mathbf{Poly}) \simeq \mathbf{Cat},$$

where **Cat** is the category of small categories and (co)functors. The directed-container comonoid data are exactly a category's identities and composition.

This is the (L) condition: a system is composable-with-itself precisely when it is a directed container, equivalently a small category. The proof, and the four independent lines of work that converge on it, are the subject of Chapter 4; we state it here only so that later chapters may say “directed container” and “small category” interchangeably.

3.4 Day convolution and the four monoidal structures

Besides $\triangleleft$, the category **Cont** carries tensors transported from monoidal structures on the base by *Day convolution*. These are the setting of the change-of-base chapter (Chapter 8); we record only the degeneration that makes them coend-free.

Definition 3.10 (Day convolution; standard). For a small monoidal category $(\mathcal{C}, \otimes, I)$ the *Day convolution* of copresheaves F, G is

$$(F \otimes G)(c) = \int^{c_1, c_2} \mathcal{C}(c_1 \otimes c_2, c) \times F(c_1) \times G(c_2),$$

with unit y^I . Then $([\mathcal{C}, \mathbf{Set}], \otimes, y^I)$ is monoidal, cocontinuous in each variable, and $y^{(-)}$ is strong monoidal: $y^a \otimes y^b \cong y^{a \otimes b}$ [1].

Proposition 3.11 (Degenerate Day on containers; proved). *For any monoidal structure $\star$ on **Set**, Day convolution induces on $\mathbf{Cont} \cong \mathbf{Fam}(\mathbf{Setop})$ a monoidal structure $\odot_\star$ given without any coend by*

$$p \odot_\star q = (S_p \times S_q, (s, t) \mapsto p[s] \star q[t]),$$

where $p = (S_p, p[-])$, $q = (S_q, q[-])$. Shapes multiply; only the position law changes.

Proof. Write $p = \coprod_s y^{p[s]}$, $q = \coprod_t y^{q[t]}$ (Theorem 3.7). Day is cocontinuous in each variable, so $p \odot_\star q \cong \coprod_{s,t} (y^{p[s]} \odot_\star y^{q[t]}) \cong \coprod_{s,t} y^{p[s] \star q[t]}$ by strong monoidality on representables; read off the container by Theorem 3.7. The coend has been discharged on the generators. $\square$

Taking $\star \in \{+, \times\}$ gives two of the four tensors on **Cont** (the additive and Dirichlet convolutions); the substitution tensor $\triangleleft$ of §3.3 is a third, and it is *not* a Day convolution. The classification — which tensors arise, and which are closed — is Chapter 8. One fact from there is used as a black box: each Day tensor is closed, and its internal hom is *forced* on representables, $[y^a, H]_\otimes \cong H((-) \otimes a)$, then extended by density; for the Dirichlet tensor $[y^a, y^b]$ is the “shapes are morphisms” container with shape set $\mathbf{Cont}(y^a, y^b)$. Density *determines* this internal hom but does not hand it to every cocontinuous functor for free — preservation of a right adjoint is a separate, checkable fact — which is exactly the subtlety the $\{\text{Id}\}$ collapse of Chapter 8 turns on.

3.5 Cohomological conventions

The global condition $[\omega] = 0$ is a statement in low-degree cohomology. We fix conventions here; the theorems are Chapters 5–7.

For a group G and a G -module M we write $H^2(G; M)$ for the second group cohomology, classifying extensions $1 \rightarrow M \rightarrow E \rightarrow G \rightarrow 1$ up to equivalence; the class of an extension is its *Schreier class*. For a groupoid $\mathcal{G}$ we use the cohomology of its classifying space, which for a *connected* groupoid reduces to that of its vertex group π : $H^2(\mathcal{G}; M) \cong H^2(\pi; M)$. The class we call $[\omega]$ is the obstruction to a coherent weld of two systems — the Schreier class of the Zappa–Szép extension realising their composite; the criterion $(G) \iff [\omega] = 0$ is Chapter 6.

Two structural tools recur. For a normal extension $1 \rightarrow \mathcal{D} \rightarrow \pi \rightarrow \Gamma \rightarrow 1$ the *Lyndon–Hochschild–Serre spectral sequence* has $E_2^{p,q} = H^p(\Gamma; H^q(\mathcal{D}; M))$ converging to $H^{p+q}(\pi; M)$; its degree-two *transgression* d_2 is the differential that, in the programme’s identification, carries the holonomy ($d_2(\varphi) = \varphi_*[\omega]$). And the phrase “ $[\omega]$ is a differential, not a summand” means precisely that a base-edge class fails to survive to the abutment when $d_2 \neq 0$. These are the load-bearing facts behind the “filtration, not product” grammar of the H^2 -cluster (Chapter 7); we invoke them there by name.

Chapter 4

The equivalence chain

The local condition (L) rests on the identification of four objects that were developed independently: ordinary containers, directed containers, polynomial comonoids, and small categories. This chapter assembles the chain $\mathbf{Cont} \simeq \mathbf{DCont} \simeq (\text{poly comonoids}) \simeq \mathbf{Cat}$ and fixes the notation used throughout the dossier. It is the foundation on which every later composition statement stands: to compose systems coherently, each system must first *be* one of these objects.

4.1 One object, four dialects

The organising slogan of the chapter is that a small category, a directed container, and a polynomial comonoid are *the same object spoken in three dialects*, sitting inside the larger world of containers as exactly those that carry comonad structure. The four vocabularies line up term for term.

Notation 4.1 (The four dialects). Throughout, we translate freely across the following dictionary.

Container	Directed container	Polynomial	Category
shape $s \in S$	shape s	position $s \in p(1)$	object s
position $p \in P(s)$	position p	direction $p \in p[s]$	morphism out of s
—	root $\mathbf{o}(s)$	—	identity id_s
—	sub-shape $s \downarrow p$	—	codomain $\text{cod}(p)$
—	shift $p \oplus q$	—	composite $q \circ p$
container map	directed-container map	comonoid hom.	<i>cofunctor</i>

The bottom row is the one subtlety carried by this chapter: the equivalences of the chain are equivalences *of categories* between directed containers and polynomial comonoids, whereas the identification with $\mathbf{Cat}$ is a bijection *on objects*; the native morphisms are cofunctors, not functors (§4.7).

The plan is: recall containers and their extension (§4.2); add the directed structure (§4.3); show that directed structure is *exactly* comonad structure on the extension (§4.4); read that comonad as a small category (§4.5); re-read the same data in **Poly** as a comonoid (§4.6); and assemble the chain with its variance caveat and formalisation status (§4.7).

4.2 Containers and the extension

We recall only what the chain uses; the definitions and their basic theory are established in Chapter 3. A *container* is a pair (S, P) of a set S of *shapes* and a family $P: S \rightarrow \mathbf{Set}$ of *positions* [24].

Its *extension* is the endofunctor

$$\llbracket S, P \rrbracket(X) = \sum_{s \in S} X^{P(s)} = \sum_{s \in S} \mathbf{Set}(P(s), X),$$

an element being a pair (s, v) with $v: P(s) \rightarrow X$. As recorded in the preliminaries, containers organise into $\mathbf{Cont} \cong \mathbf{Fam}(\mathbf{Set}^{\text{op}})$ (Notation 3.6), a morphism $(S, P) \rightarrow (S', P')$ being a shape map $f: S \rightarrow S'$ *forward* together with a position map $f_s^\sharp: P'(fs) \rightarrow P(s)$ *backward*; the extension is the left Kan extension exhibited in Theorem 3.7, and $\llbracket - \rrbracket: \mathbf{Cont} \rightarrow \mathbf{End}(\mathbf{Set})$ is full and faithful onto the polynomial functors by the Yoneda/density argument of Theorems 3.3–3.5. We do not reprove these here.

Remark 4.2 (The backward variance is the whole story). The contravariance of $f^\sharp$ in positions is not incidental. It is the fibrewise opposite of $\mathbf{Fam}(\mathbf{Set}^{\text{op}})$, and it is precisely what will force the morphisms of directed containers to be cofunctors rather than functors (§4.7). Every phenomenon in this chapter is a consequence of positions running backward while shapes run forward.

4.3 Directed containers

Chapter 3 introduced a directed container informally as a $\triangleleft$ -comonoid (Definition 3.8). We now fix the explicit operational form used for all computations, following Ahman–Chapman–Uustalu [25].

Definition 4.3 (Directed container). A *directed container* is a container (S, P) equipped with

- a *root* $\mathbf{o}(s) \in P(s)$ for each $s \in S$;
- a *sub-shape* operation $s \downarrow p \in S$ for $p \in P(s)$;
- a *shift* $p \oplus q \in P(s)$ for $p \in P(s)$ and $q \in P(s \downarrow p)$;

subject, for all $s, p \in P(s)$, $q \in P(s \downarrow p)$ and $q' \in P((s \downarrow p) \downarrow q)$, to

$$\begin{aligned} \text{(D1)} \quad s \downarrow \mathbf{o}(s) &= s, & \text{(D2)} \quad \mathbf{o}(s) \oplus q &= q, & \text{(D3)} \quad p \oplus \mathbf{o}(s \downarrow p) &= p, \\ \text{(D4)} \quad s \downarrow (p \oplus q) &= (s \downarrow p) \downarrow q, & \text{(D5)} \quad (p \oplus q) \oplus q' &= p \oplus (q \oplus q'). \end{aligned}$$

Directed containers and their morphisms (Definition 4.14) form the category $\mathbf{DCont}$.

Remark 4.4 (An indexed monoid). Laws (D2), (D3), (D5) make each $P(s)$ a monoid-like structure with unit $\mathbf{o}(s)$; (D1) and (D4) track how sub-shapes nest. The typing of (D5) depends on (D4): for $p \oplus (q \oplus q')$ to be defined one needs $q \oplus q' \in P(s \downarrow p)$ with $(s \downarrow p) \downarrow (q \oplus q') = s \downarrow (p \oplus q)$, which is exactly (D4). This foreshadows the categorical reading, where the well-typedness of associative composition rests on the codomain law.

4.4 Directed containers are comonads (M2)

The first link identifies directed structure with comonad structure on the extension. This is the content the roadmap calls milestone M2.

Proposition 4.5 (The extension comonad, computed; *proved, lean-verified*). *Let (S, P) be a directed container and $D := \llbracket S, P \rrbracket$. Define*

$$\varepsilon_X(s, v) = v(\mathbf{o}(s)), \quad \delta_X(s, v) = (s, \lambda p. (s \downarrow p, \lambda q. v(p \oplus q))).$$

Then (D, ε, δ) is a comonad, and each comonad law is exactly one directed-container law: the left counit is (D1)+(D2), the right counit is (D3), and coassociativity is (D4)+(D5). Conversely, every comonad structure on $\llbracket S, P \rrbracket$ arises this way, so directed-container structures on (S, P) correspond bijectively to comonad structures on $\llbracket S, P \rrbracket$.

Proof sketch. Write $\delta(s, v) = (s, w)$ with $w(p) = (s \downarrow p, \lambda q. v(p \oplus q))$. *Left counit:* $\varepsilon(s, w) = w(\mathbf{o}(s)) = (s \downarrow \mathbf{o}(s), \lambda q. v(\mathbf{o}(s) \oplus q)) = (s, v)$ using (D1) for the shape and (D2) for the value. *Right counit:* $\varepsilon(w(p)) = v(p \oplus \mathbf{o}(s \downarrow p)) = v(p)$ by (D3). *Coassociativity:* both composites of δ reduce to $(s, \lambda p. (s \downarrow p, \lambda q. ((s \downarrow p) \downarrow q, \lambda q'. v(p \oplus (q \oplus q')))))$, the inner shape agreeing by (D4) and the value by (D5). The converse is the reverse reading: full-and-faithfulness of $\llbracket - \rrbracket$ (Theorem 3.3) makes the displayed computations force (D1)–(D5) from the comonad laws. $\square$

Remark 4.6 (Provenance and formalisation). The biconditional is Ahman–Chapman–Uustalu [25, Thm. 3.15]: the category of directed containers is the pullback of the forgetful functor $\mathbf{Comon}(\mathbf{EndSet}) \rightarrow \mathbf{End}(\mathbf{Set})$ along $\llbracket - \rrbracket$. The coordinate proof of the comonad laws above is machine-checked in Lean (the position-transfer comonad-law calculation, `[Quot.sound]`-only) [51]. We report this as *lean-verified* at the level of the law-matching; the full pullback statement of [25] is *proved* in the source and not independently re-formalised here.

4.5 The object dictionary: directed containers are small categories (M4)

The second link reads the comonad of §4.4 as a small category. This is Ahman–Uustalu [26]; we record the bijection of data.

Theorem 4.7 (Object dictionary [26]; *proved*). *Directed-container structures on (S, P) are in bijection with small categories $\mathcal{C}$ with object set S and $P(s) = \coprod_{s'} \mathcal{C}(s, s')$ (the morphisms out of s), under*

$$\text{ob}\mathcal{C} = S, \quad \mathcal{C}(s, s') = \{p \in P(s) : s \downarrow p = s'\}, \quad \text{id}_s = \mathbf{o}(s), \quad q \circ p = p \oplus q.$$

In particular $s \downarrow p = \text{cod}(p)$, and every small category arises from a unique directed container.

Proof sketch. (Category $\Rightarrow$ directed container.) Set $P(s) = \coprod_{s'} \mathcal{C}(s, s')$, $s \downarrow p = \text{cod}(p)$, $\mathbf{o}(s) = \text{id}_s$, $p \oplus q = q \circ p$. Then (D1) is $\text{cod}(\text{id}_s) = s$; (D2), (D3) are the unit laws; (D4) is $\text{cod}(q \circ p) = \text{cod}(q)$; (D5) is associativity, written with the order reversal $p \oplus q = q \circ p$. (Directed container $\Rightarrow$ category.) Take $\mathcal{C}(s, s') = \{p \in P(s) : s \downarrow p = s'\}$, identities $\mathbf{o}(s)$ (landing in $\mathcal{C}(s, s)$ by (D1)), and composite $p \oplus q$ (landing in $\mathcal{C}(s, s'')$ by (D4)); unit laws are (D2), (D3), associativity is (D5). The two constructions are mutually inverse. $\square$

Corollary 4.8 (The two atoms). *One-shape directed containers ($|S| = 1$) are exactly monoids; thin ones ($|\mathcal{C}(s, s')| \leq 1$) are exactly preorders. Every small category is assembled from these two phenomena — many objects and endo-loops.*

Example 4.9 (The two generating cases). For the poset $\mathbf{2} = \{0 \xrightarrow{u} 1\}$: $S = \{0, 1\}$, $P(0) = \{\mathbf{o}(0), u\}$, $P(1) = \{\mathbf{o}(1)\}$, with $0 \downarrow u = 1$ and $u \oplus \mathbf{o}(1) = u$; the extension has $|\llbracket S, P \rrbracket(1)| = 3$, the three morphisms. For the monoid $\mathbb{Z}/3$: one shape $\star$, $P(\star) = \mathbb{Z}/3$, $\star \downarrow p = \star$, $\mathbf{o}(\star) = 0$, $p \oplus q = p + q \bmod 3$; the extension is the shift comonad $DX = X^3$ with $\varepsilon(v) = v(0)$ and $\delta(v) = \lambda p. \lambda q. v(p + q)$. All laws are verified by hand in the source.

4.6 Polynomial comonoids (M5, bridge)

The third dialect is Spivak’s category **Poly** of polynomial endofunctors on **Set**. A container (S, P) is the polynomial $p = \sum_{s \in S} y^{P(s)}$ with $p(1) = S$ and $p[s] = P(s)$; container maps are maps of polynomials, so **Cont** is **Poly** under a change of name [31, 32]. The composition tensor $\triangleleft$ on **Poly** corresponds to composition of extensions, $\llbracket p \triangleleft q \rrbracket \cong \llbracket p \rrbracket \circ \llbracket q \rrbracket$.

Theorem 4.10 (Polynomial comonoids are categories [29, 30, 31]; *proved* in the sources). *The comonoids for $(\mathbf{Poly}, \triangleleft, y^{(-)})$ — polynomial comonoids — are exactly small categories: the counit is the identity-assigning map and the comultiplication is composition, subject to the comonoid laws which are the category axioms. Their morphisms are the comonoid homomorphisms, and the resulting category is $\mathbf{Cat}^\sharp = \mathbf{Comod}(\mathbf{Poly})$.*

Remark 4.11 (One structure, two derivations). A polynomial comonoid on $p = \llbracket S, P \rrbracket$ is the same data as the extension comonad of Proposition 4.5: comultiplication δ is $s \mapsto (s \downarrow p, \oplus)$ and counit ε is $\mathbf{o}$. Ahman–Uustalu [25, 26] and Spivak et al. [29, 30] arrived at this identification independently, from datatypes and from interaction respectively. The translation is tabulated in Notation 4.1. This is milestone M5; the full agreement of *all* monoidal structures on **Cont** with the four products of **Poly** (coproduct, product, $\triangleleft$, Dirichlet) is *open* and not formalised.

4.7 The chain, assembled; and the variance caveat

Combining the links gives the chain of the chapter, which we now state precisely — and correct where the informal slogan misleads.

Theorem 4.12 (The equivalence chain; *proved*). *There are equivalences of categories*

$$\mathbf{DCont} \simeq \mathbf{Comod}(\mathbf{Poly}) = \mathbf{Cat}^\sharp,$$

and a bijection, natural in the data, between the objects of these categories and small categories (Theorem 4.7). Directed containers embed into containers by the forgetful functor $\mathbf{DCont} \rightarrow \mathbf{Cont}$ that discards $(\mathbf{o}, \downarrow, \oplus)$; this functor is faithful and injective on objects but not full, so $\mathbf{DCont}$ is a proper (non-full) subclass of $\mathbf{Cont}$ — exactly the containers whose extension carries a comonad structure (Proposition 4.5).

Remark 4.13 (Reading the chain of the chapter head). The displayed chain $\mathbf{Cont} \simeq \mathbf{DCont} \simeq (\text{poly comonoids}) \simeq \mathbf{Cat}$ must be read with two qualifications, both proved above.

1. The link **Cont–DCont** is a *containment* $\mathbf{Cont} \supseteq \mathbf{DCont}$ (the forgetful functor of Theorem 4.12), not an equivalence: not every container is directed.
2. The link to **Cat** is an equivalence *on objects* only. As a category, $\mathbf{DCont} \simeq \mathbf{Cat}^\sharp$, whose morphisms are cofunctors, not functors (Theorem 4.16 below).

The genuine equivalences of categories are $\mathbf{DCont} \simeq \mathbf{Comod}(\mathbf{Poly}) = \mathbf{Cat}^\sharp$.

We make the second qualification precise, since it governs how systems compose in every later chapter. The maps native to directed containers pull positions backward, $f_s^\sharp: P'(fs) \rightarrow P(s)$ (Remark 4.2), which under the dictionary lifts a morphism out of fs to one out of s — the variance of a *cofunctor*, not a functor.

Definition 4.14 (Directed-container morphism). A morphism $D \rightarrow D'$ of directed containers is a container morphism $(f, f^\sharp)$ satisfying, for all $s, h \in P'(fs)$ and $k \in P'((fs) \downarrow' h)$,

$$\begin{aligned} \text{(M0)} \quad & f(s \downarrow f_s^\sharp(h)) = (fs) \downarrow' h, \\ \text{(M1)} \quad & f_s^\sharp(\mathbf{o}'(fs)) = \mathbf{o}(s), \\ \text{(M2)} \quad & f_s^\sharp(h \oplus' k) = f_s^\sharp(h) \oplus f_{s \downarrow f_s^\sharp(h)}^\sharp(k). \end{aligned}$$

Definition 4.15 (Cofunctor [34, 35, 29]). A *cofunctor* $\varphi: \mathcal{C} \bullet \mathcal{D}$ is an object map φ_0 together with a lift assigning to each object c and morphism $h: \varphi_0 c \rightarrow d$ a morphism $\varphi^\uparrow(c, h): c \rightarrow c'$, subject to

(C0) $\varphi_0(\text{cod } \varphi^\uparrow(c, h)) = \text{cod}(h)$, (C1) $\varphi^\uparrow(c, \text{id}_{\varphi_0 c}) = \text{id}_c$, and (C2) $\varphi^\uparrow(c, h' \circ h) = \varphi^\uparrow(c', h') \circ \varphi^\uparrow(c, h)$ with $c' = \text{cod } \varphi^\uparrow(c, h)$. Small categories and cofunctors form $\mathbf{Cof} = \mathbf{Cat}^\sharp$, the comonoids in $(\mathbf{Poly}, \triangleleft)$ and their homomorphisms [29, 30].

Theorem 4.16 (The morphism dictionary [26, 30]; *proved, lean-verified*). *The assignment $(f, f^\sharp) \mapsto (\varphi_0 := f, \varphi^\uparrow(s, h) := f_s^\sharp(h))$ is an isomorphism of categories*

$$\mathbf{DCont} \cong \mathbf{Cof},$$

*matching (M0)–(M2) to (C0)–(C2) term for term under the substitutions “position out of $s =$ morphism out of s ” and “ $x \oplus y = y \circ x$ ”. This isomorphism is formalised in Lean 4 with zero **sorry** and zero axioms [51].*

Proof sketch. Under the object dictionary the two sides are literally the same data: f is an object map and $f^\sharp$ is a lift. (M0) is the codomain law (C0); (M1) is the unit law (C1); and, writing $p = f_s^\sharp(h)$, $s' = s \downarrow p$, the shift law (M2) reads $f_s^\sharp(k \circ' h) = \varphi^\uparrow(s', k) \circ \varphi^\uparrow(s, h)$, which is (C2). The correspondence is a bijection on every hom-set and preserves identities and composition (position maps compose backward, $(g \circ f)_s^\sharp = f_s^\sharp \circ g_{f_s}^\sharp$), and it is a bijection on objects by Theorem 4.7; a bijective-on-objects fully faithful functor is an isomorphism of categories. $\square$

Proposition 4.17 (The dictionary does not extend to $\mathbf{Cat}$ with functors; *proved* [26, 30]). *There is no functor $\mathbf{DCont} \rightarrow \mathbf{Cat}$ agreeing with the object dictionary on objects and full. Already for the terminal category $\mathbf{1}$ and $\mathbf{2} = \{0 \rightarrow 1\}$ there is exactly one cofunctor $\mathbf{1} \bullet \mathbf{2}$ but two functors $\mathbf{1} \rightarrow \mathbf{2}$, so $|\mathbf{DCont}(D_1, D_2)| = 1 < 2 = |\mathbf{Cat}(\mathbf{1}, \mathbf{2})|$. The comparison is not even monotone: $|\mathbf{Cat}(\mathbf{3}, \mathbf{2})| = 4 < 5 = |\mathbf{Cof}(\mathbf{3}, \mathbf{2})|$ while $|\mathbf{Cat}(\mathbf{2}, \mathbf{2})| = 3 > 2 = |\mathbf{Cof}(\mathbf{2}, \mathbf{2})|$, so neither hom-set contains the other.*

Remark 4.18 (Why the variance is the useful half). The covariant variant (position maps $P(s) \rightarrow P'(fs)$ under the mirror-image conditions) recovers functors exactly, but these are *not* morphisms of the underlying container. The contravariant, native half is the *Put* of a delta lens [35]: a directed-container morphism is the update-propagating component of a bidirectional transformation. Every later composition statement in this dossier — supply-chain stages, agent hierarchies, state updates — composes its maps as cofunctors; reading them as functors keeps the *Get* and silently discards the *Put*, and even changes the count of maps (Proposition 4.17).

Remark 4.19 (Formalisation status of the chain). Against the roadmap milestones, the object-level chain is verified as follows. *M1* (containers, the representation theorem) is standard and recorded in Chapter 3. *M2* (directed container = comonad, Proposition 4.5) is *proved* in [25] with the law-matching *lean-verified* [51]. *M4* ($\mathbf{DCont} \cong \mathbf{Cof}$, Theorem 4.16) is *lean-verified*, zero **sorry** and zero axioms [51]. Milestones *M5* (agreement of all four monoidal products with **Poly**, Remark 4.11) and *M6* (the double category $\mathbf{Org}$ of orchestration) are *open*: stated in the sources as targets, not yet formalised, and not claimed here.

Chapter 5

Zappa–Szép products and distributive laws as the composition tool

Given local pieces, composition is a *weld*, and the canonical weld of two categories/comonads is the Zappa–Szép product, equivalently a distributive law. This chapter develops the pairwise criterion: two directed containers compose iff the local condition (L) and a global condition (G) both hold. It is the machinery that makes “ $[\omega] = 0$ ” precise in the next chapter.

5.1 Composition is a weld

Fix the standing dictionary of the preliminaries: a directed container is a $\triangleleft$ -comonoid, and **DCont** $\cong$ **Cat** identifies it with a small category (Definition 3.8, the chain of Theorem 3.9). Under this identification the objects of the container are its shapes, the morphisms out of a shape are its positions, identities are roots, and composition is position-concatenation [26].

The question of this chapter is not how to run *one* directed container — a functor already does that, and a functor cannot fail — but how to *combine two*. Running two compositional systems that share a resource is the *interleaving* of two handoff categories, and interleaving is not composition along a fixed topology. It is a weld: the two systems must be fitted together inside a single third system so that each morphism of the whole decomposes, uniquely and coherently, into a piece from one factor followed by a piece from the other. The canonical such weld is the Zappa–Szép product, and its structural carrier is a distributive law.

Definition 5.1 (Wide subcategory, strict factorisation). Throughout $\mathcal{K}$ is a small category and every subcategory is *wide*: $\mathcal{C} \subseteq \mathcal{K}$ is wide if $\text{ob}\mathcal{C} = \text{ob}\mathcal{K}$, it is closed under composition, and it contains every identity. Write dom , cod for source and target. Given wide $\mathcal{C}, \mathcal{D} \subseteq \mathcal{K}$, the pair $(\mathcal{C}, \mathcal{D})$ is a *strict factorisation system* of $\mathcal{K}$, written $\mathcal{K} = \mathcal{C} \bowtie \mathcal{D}$, if every morphism $f \in \mathcal{K}$ factors *uniquely* as $f = c \circ d$ with $c \in \mathcal{C}$ and $d \in \mathcal{D}$. We call $\mathcal{K}$ the *Zappa–Szép product* of $\mathcal{C}$ and $\mathcal{D}$.

We fix the orientation once and for all: the $\mathcal{D}$ -part acts first (at the source), the $\mathcal{C}$ -part second (at the target). This is the many-object generalisation of the classical object. For monoids M, N (one-object categories), a Zappa–Szép product $M \bowtie N$ is a monoid K containing M, N as submonoids for which the multiplication map $M \times N \rightarrow K$, $(m, n) \mapsto mn$, is a bijection: every element has a unique expression as an M -element times an N -element. This is the maximally general way two monoids sit inside a third; when N is normal it is the semidirect product $M \ltimes N$, when both are normal the direct product [36].

The point that makes the weld nontrivial with many objects is that $\mathcal{D}$ need *not* be the endomorphisms. It may carry *cross-object* arrows, so a factorisation $f = c \circ d$ moves the object at which the two factors meet. Welding then demands that a single “pivot object” be located, simultaneously and coherently, for every morphism of $\mathcal{K}$. This is a genuine recognition problem with no one-object shadow: at one object a factorisation either exists and one writes it down, or it does not.

5.2 The distributive law: the algebra of the weld

When a weld exists, $\mathcal{K}$ is reconstructed from $\mathcal{C}$, $\mathcal{D}$, and a distributive law $\lambda: \mathcal{D} \triangleleft \mathcal{C} \Rightarrow \mathcal{C} \triangleleft \mathcal{D}$ generalising the matched-pair calculus of monoids to two genuine many-object categories. This is the categorical form of the correspondence between distributive laws and strict factorisations [37, 38], and for directed containers it is exactly Ahman–Uustalu’s composition construction [27]. We record the data, the axioms, and the associativity theorem.

Throughout, for composable $d \in \mathcal{D}$ and $c \in \mathcal{C}$ we write ${}^d c$ for the *left action* of d on c (a $\mathcal{C}$ -arrow) and c^d for the *right restriction* of d by c (a $\mathcal{D}$ -arrow).

Definition 5.2 (Distributive law / matched pair of categories). A *distributive law* $\lambda: \mathcal{D} \triangleleft \mathcal{C} \Rightarrow \mathcal{C} \triangleleft \mathcal{D}$ assigns to each composable pair (d, c) with $d \in \mathcal{D}$, $c \in \mathcal{C}$, $\text{dom}(d) = \text{cod}(c)$ a pair $\lambda(d, c) = ({}^d c, c^d)$ with ${}^d c \in \mathcal{C}$, $c^d \in \mathcal{D}$ and

$$\text{dom}(c^d) = \text{dom}(c), \quad \text{cod}({}^d c) = \text{cod}(d), \quad \text{dom}({}^d c) = \text{cod}(c^d),$$

so that ${}^d c \circ c^d$ has the same source and target as $d \circ c$. It is a *matched pair* if, for all composable data,

$$(U) \quad \text{id}^c c = c, \quad c^{\text{id}} = \text{id}_{\text{dom } c}, \quad {}^d \text{id} = \text{id}_{\text{cod } d}, \quad \text{id}^d = d;$$

$$(ZS2) \quad ({}^{d' \circ d} c) = d'({}^d c);$$

$$(ZS3) \quad c^{(d' \circ d)} = ({}^d c)^{d'} \circ c^d;$$

$$(ZS1) \quad {}^d (c_2 \circ c_1) = {}^d c_2 \circ ({}^{c_2^d} c_1);$$

$$(ZS4) \quad (c_2 \circ c_1)^d = c_1^{(c_2^d)}.$$

The four axioms are Brin’s matched-pair axioms [36], typed for two many-object categories: (ZS2) says $\mathcal{D}$ acts on $\mathcal{C}$ on the left, (ZS3) is the cocycle for the restriction over $\mathcal{D}$ -composition, and (ZS1),(ZS4) are the compatibilities with $\mathcal{C}$ -composition. They reduce verbatim to the monoid axioms when each factor has one object.

Definition 5.3 (The product category). Given a matched pair, $\mathcal{C} \bowtie \mathcal{D}$ has the shared object set, hom-sets $(\mathcal{C} \bowtie \mathcal{D})(a, b) = \coprod_m \mathcal{C}(m, b) \times \mathcal{D}(a, m)$, identities $(\text{id}_a, \text{id}_a)$, and composition

$$(c_2, d_2) \cdot (c_1, d_1) = (c_2 \circ {}^{d_2} c_1, c_1^{d_2} \circ d_1),$$

for $(c_1, d_1): a \rightarrow b$ (so $d_1: a \rightarrow m_1$, $c_1: m_1 \rightarrow b$) and $(c_2, d_2): b \rightarrow e$.

At one object this is precisely the classical Zappa–Szép multiplication $(m_2, n_2)(m_1, n_1) = (m_2 {}^{n_2} m_1, m_1^{n_2} n_1)$.

Theorem 5.4 (Axioms $\iff$ associativity). (proved) $\mathcal{C} \bowtie \mathcal{D}$ of Definition 5.3 is a category if and only if λ is a matched pair. The unit laws are equivalent to (U); a single associativity instance splits into its $\mathcal{C}$ -coordinate, governed by (ZS1)+(ZS2), and its $\mathcal{D}$ -coordinate, governed by (ZS3)+(ZS4).

Proof sketch. The unit laws for (id, id) unwind directly to (U). For associativity, expand both bracketings of $(c_3, d_3)(c_2, d_2)(c_1, d_1)$ by Definition 5.3. Both have a $\mathcal{C}$ -composite first coordinate and a $\mathcal{D}$ -composite second coordinate, and the two expansions agree iff the coordinates agree separately. The $\mathcal{C}$ -coordinate discrepancy is $^{(d_3 \circ d_2)}c_1$ versus $d_3(^{d_2}c_1)$ together with a distribution of $^d(-)$ over a $\mathcal{C}$ -composite — exactly (ZS2) and (ZS1). The $\mathcal{D}$ -coordinate discrepancy is $c_1^{(d_3 \circ d_2)}$ versus $(^{d_2}c_1)^{d_3 \circ c_1^{d_2}}$ together with the distribution of $(-)^d$ over a $\mathcal{C}$ -composite — exactly (ZS3) and (ZS4). The only place $\mathcal{D}$ being a genuine category rather than a family of monoids enters is that the restrictions composed in the $\mathcal{D}$ -coordinate must be composable across a change of object; the typing of Definition 5.2 guarantees this. The converse reads the same identity chain from associativity back to the axioms. $\square$

The next lemma runs the correspondence the other way: it extracts the law from a weld and certifies that the abstract product recovers $\mathcal{K}$.

Lemma 5.5 (Extraction from a strict factorisation). *Let $(\mathcal{C}, \mathcal{D})$ be a strict factorisation of $\mathcal{K}$. For composable $d \in \mathcal{D}$, $c \in \mathcal{C}$ let $^d c \in \mathcal{C}$, $c^d \in \mathcal{D}$ be the unique factorisation of $d \circ c \in \mathcal{K}$:*

$$d \circ c = ^d c \circ c^d.$$

Then λ is a matched pair, and $\Phi: \mathcal{C} \bowtie \mathcal{D} \rightarrow \mathcal{K}$, $\Phi(c, d) = c \circ d$, is an isomorphism of categories.

Proof sketch. Each axiom is obtained by factoring one $\mathcal{K}$ -morphism two ways and invoking uniqueness. For (ZS2)/(ZS3), factor $(d' \circ d) \circ c$ as

$$(d' \circ d) \circ c = d' \circ (^d c \circ c^d) = (d' \circ ^d c) \circ c^d = d'(^d c) \circ (^d c)^{d'} \circ c^d,$$

the last step factoring $d' \circ ^d c$, composable since $\text{cod } ^d c = \text{cod } d = \text{dom } d'$; here $(^d c)^{d'}$ and c^d compose in $\mathcal{D}$ because $\text{dom } (^d c)^{d'} = \text{dom } ^d c = \text{cod } c^d$ — the one place $\mathcal{D}$ being a genuine category matters. Comparing with the direct factorisation of $(d' \circ d) \circ c$ and invoking uniqueness yields the $\mathcal{C}$ -part (ZS2) and $\mathcal{D}$ -part (ZS3). Factoring $d \circ (c_2 \circ c_1)$ two ways yields (ZS1)/(ZS4), and the units are immediate. Finally Φ is a bijection on each hom-set (it *is* the factorisation), preserves identities, and preserves composition by the rewrite $d \circ c = ^d c \circ c^d$, so it is an isomorphism. $\square$

Thus when a weld exists, $\mathcal{K}$ is genuinely the Zappa–Szép product of the categories $\mathcal{C}$ and $\mathcal{D}$, and λ is the precise carrier of the mutual action — the data one would extract to *run* the composite system.

5.3 The recognition problem and the free-presheaf lemma

Theorem 5.4 and Lemma 5.5 describe the algebra *once* a weld exists. The decision problem is prior: given $\mathcal{K}$ and a prescribed right factor $\mathcal{D}$, does a complementary $\mathcal{C}$ exist at all? The whole answer turns on one observation — the presheaf form of “free module = has a basis” — which we isolate first.

The hom-sets of $\mathcal{K}$ into a fixed target b assemble into a presheaf: a right $\mathcal{D}$ -set.

Definition 5.6 (Right $\mathcal{D}$ -set, representable, free, target presheaf). A *right $\mathcal{D}$ -set* is a presheaf $M: \mathcal{D}^{\text{op}} \rightarrow \mathbf{Set}$: each $d: a \rightarrow m$ gives $x \mapsto x \cdot d$, with $x \cdot \text{id} = x$ and $(x \cdot d') \cdot d = x \cdot (d' \circ d)$. The *representable* at m is $\mathcal{D}(-, m)$ with action by precomposition; M is *free* if it is isomorphic to a coproduct of representables. For an object b of $\mathcal{K}$, the *target presheaf* $\text{Hom}_{\mathcal{K}}(-, b): \mathcal{D}^{\text{op}} \rightarrow \mathbf{Set}$ sends a to $\text{Hom}_{\mathcal{K}}(a, b)$ and $d: a' \rightarrow a$ to precomposition $f \mapsto f \circ d$; it is a right $\mathcal{D}$ -set because $\mathcal{D} \subseteq \mathcal{K}$.

Lemma 5.7 (Free = unique factorisation). (proved) *Let $\mathcal{D}$ be a small category and $M: \mathcal{D}^{\text{op}} \rightarrow \mathbf{Set}$. The following are equivalent.*

1. *M is free.*
2. *There is a family $B = (B_m \subseteq M(m))_{m \in \text{ob } \mathcal{D}}$ such that for every object a and every $x \in M(a)$ there is a unique pair (c, d) with $c \in B_m$ (some m) and $d \in \mathcal{D}(a, m)$ satisfying $x = c \cdot d$.*

When they hold, $M \cong \coprod_m \coprod_{c \in B_m} \mathcal{D}(-, m)$, and B is a basis.

Proof sketch. (2) $\Rightarrow$ (1): $\varphi_a(c, d) = c \cdot d$ is a well-defined presheaf morphism by contravariance of M and precomposition on $\mathcal{D}(-, m)$; it is objectwise bijective by hypothesis, hence an isomorphism. (1) $\Rightarrow$ (2): given $\psi: \coprod_i \mathcal{D}(-, m_i) \cong M$, set $c_i := \psi_{m_i}(\text{id}_{m_i})$; by Yoneda the i -th component is “multiply by c_i ”, and since ψ is an isomorphism every x equals $c_i \cdot d$ for a unique index i and — the representable being a genuine set of morphisms — a unique d . This is (2) with $B_m = \{c_i : m_i = m\}$. $\square$

Remark 5.8 (The endomorphism specialisation). If $\mathcal{D} = \mathcal{E}$ is the disjoint union of vertex monoids ($\mathcal{E}(a, a) = \text{End}(a)$, $\mathcal{E}(a, b) = \emptyset$ for $a \neq b$), a presheaf on $\mathcal{E}$ is a family of right $\text{End}(a)$ -sets, freeness is freeness of each fibre, and a basis B_a is one representative per $\text{End}(a)$ -orbit. So Lemma 5.7 reduces to “free $\text{End}(a)$ -set = has an orbit transversal” fibre by fibre.

5.4 The pairwise criterion

The key observation is that, for a weld, a free basis of $\text{Hom}_{\mathcal{K}}(-, b)$ is *forced* to be the family of $\mathcal{C}$ -arrows into b . There is nothing to choose: the factorisation dictates the basis and the basis dictates the factorisation.

Theorem 5.9 (Pairwise Zappa–Szép criterion). (proved) *Let $\mathcal{K}$ be a small category and $\mathcal{D} \subseteq \mathcal{K}$ a wide subcategory. The following are equivalent.*

1. *There is a wide subcategory $\mathcal{C} \subseteq \mathcal{K}$ with $\mathcal{K} = \mathcal{C} \bowtie \mathcal{D}$.*
2. *Both:*

(L) *for every object b , the right $\mathcal{D}$ -set $\text{Hom}_{\mathcal{K}}(-, b)$ is free; and*

(G) *free bases B_b of $\text{Hom}_{\mathcal{K}}(-, b)$ can be chosen so that $\mathcal{C} := \bigcup_b B_b$ is a wide subcategory of $\mathcal{K}$.*

Proof sketch. The only interaction the proof uses is that a basis element $c \in B_m \subseteq \text{Hom}_{\mathcal{K}}(m, b)$ and $d \in \mathcal{D}(a, m)$ give the composite $c \cdot d = c \circ d: a \rightarrow b$ in $\mathcal{K}$.

(1) $\Rightarrow$ (2): with $B_b := \{c \in \mathcal{C} : \text{cod}(c) = b\}$, uniqueness of the factorisation says each $f: a \rightarrow b$ is $c \circ d$ for a unique $c \in B_b$, $d \in \mathcal{D}(a, \text{dom } c)$; by Lemma 5.7, B_b is a basis, so $\text{Hom}_{\mathcal{K}}(-, b)$ is free — this is (L). Since each $\mathcal{C}$ -arrow has one codomain, $\bigcup_b B_b = \mathcal{C}$, wide by hypothesis — this is (G).

(2) $\Rightarrow$ (1): let $\mathcal{C} = \bigcup_b B_b$ be the wide subcategory from (G); then $B_b = \{c \in \mathcal{C} : \text{cod}(c) = b\}$. Freeness with basis B_b gives, for each $f: a \rightarrow b$, a unique (c, d) with $c \in B_b \subseteq \mathcal{C}$, $d \in \mathcal{D}$, $f = c \circ d$. Any $f = c' \circ d'$ with $c' \in \mathcal{C}$ has $\text{cod}(c') = b$, so $c' \in B_b$, and uniqueness over B_b forces $(c', d') = (c, d)$. Hence the factorisation is unique over all of $\mathcal{C}$, and $(\mathcal{C}, \mathcal{D})$ is a strict factorisation. $\square$

Remark 5.10 (Why two levels). Condition (L) concerns the right $\mathcal{D}$ -action alone: it asks that the action be *free*, so bases exist, one choice per target. It is the analogue of “free module,” and is checkable target by target. Condition (G) is the cross-target constraint: the independently chosen bases must *cohere* into a single subcategory. The two can come apart (Proposition 5.15), and the gap is a holonomy — the failure of the candidate λ to be consistently single-valued. Asking for (G) is exactly asking when a wide $\mathcal{D}$ *extends* to a strict factorisation, the subject of Rosebrugh–Wood [37]. Its cohomological classification is the business of the next chapter (Chapter 6); here we record only the shape of the obstruction.

The single-category specialisation

Corollary 5.11. (proved) *Take $\mathcal{D} = \mathcal{E}$, the endomorphism subcategory. Then $\mathcal{K} = \mathcal{C} \bowtie \mathcal{E}$ for some wide $\mathcal{C}$ if and only if (L') each $\text{Hom}_{\mathcal{K}}(a, b)$ is free as a right $\text{End}(a)$ -set, and (G') per-orbit representatives can be chosen to close into a wide subcategory.*

Proof sketch. By Remark 5.8, freeness of the $\mathcal{E}$ -presheaf $\text{Hom}_{\mathcal{K}}(-, b)$ is fibrewise freeness of each $\text{Hom}_{\mathcal{K}}(a, b)$ over $\text{End}(a)$, a basis is one representative per orbit, and (G) becomes closure of the representatives. $\square$

The classical anchor

At one object the recognition problem is vacuous and only the algebra survives.

Proposition 5.12 (Classical Zappa–Szép recovered). (proved) *Let M, N be monoids as one-object categories. A matched pair (Definition 5.2) is exactly a left action ${}^n m$ of N on M and a right action m^n of M on N satisfying Brin’s four axioms, and the product $M \bowtie N$ (Definition 5.3) is the classical Zappa–Szép product with multiplication $(m_2, n_2)(m_1, n_1) = (m_2 {}^{n_2} m_1, m_1^{n_2} n_1)$. Such products are precisely the monoids K with submonoids M, N for which $M \times N \rightarrow K$ is a bijection.*

Proof sketch. With one object every typing constraint in Definition 5.2 is vacuous, (U) and (ZS1)–(ZS4) become the unit laws and Brin’s four identities [36], and the coproduct $\coprod_m$ in Definition 5.3 is trivial. The last sentence is Theorem 5.9 read at one object: (L) is freeness of K as a right N -set and (G), for basis M , is that $M \times N \rightarrow K$ is a bijection with M a submonoid. $\square$

Remark 5.13 (Update monads, honestly). It is tempting to call Ahman–Uustalu’s update monad [28] the degenerate (semidirect) instance, but this overclaims and we avoid it. The update monad $TX = S \rightarrow (P \times X)$ is the writer-over-reader twist of a *monoid and a set* through a single one-sided action [28]; its directed container is the action category $S \rtimes P$ [26]. A Zappa–Szép product is the twist of two *monoids* through two mutually twisting actions. The honest relationship is analogy and handedness: the update monad is not literally a Zappa–Szép product of two monoids, and a genuine factorisation enters only if P itself splits as $P = A \bowtie B$.

5.5 A genuinely pairwise obstruction

One might suspect the two-level phenomenon is an artefact of $\mathcal{D}$ being the endomorphisms — that allowing cross-object arrows in $\mathcal{D}$ always dissolves the global obstruction. It does not.

Example 5.14 (Rigid twist over a cross-object factor). Let $\mathcal{K}$ have objects w, a, x, y with $\text{End}(a) = \{\text{id}_a, g\}$, $g^2 = \text{id}_a$, the other endomorphism monoids trivial, and

$$\text{Hom}(a, x) = \{p, pg\}, \text{Hom}(x, y) = \{s, s_2\}, \text{Hom}(a, y) = \{q, qg\}, \text{Hom}(w, a) = \{d, gd\},$$

$\text{Hom}(w, x) = \{pd, pgd\}$, $\text{Hom}(w, y) = \{qd, qgd\}$, where $pg = p \circ g$, $gd = g \circ d$, the rigid twist $s \circ p = q$, $s_2 \circ p = qg$, and the forced composites $p \circ d = pd$, $s \circ pd = qd$, $s_2 \circ pd = qgd$, etc. Let $\mathcal{D} = \{\text{id}_w, \text{id}_a, \text{id}_x, \text{id}_y, g, d, gd\}$.

Proposition 5.15. (proved) *In Example 5.14, $\mathcal{D}$ is a wide subcategory containing the cross-object arrows $d, gd: w \rightarrow a$; every $\text{Hom}_{\mathcal{K}}(-, b)$ is free as a right $\mathcal{D}$ -set (L holds), yet no wide $\mathcal{C}$ makes $(\mathcal{C}, \mathcal{D})$ a strict factorisation (G fails). Hence $\mathcal{K}$ has no weld over this $\mathcal{D}$ despite the cross-object factor.*

Proof sketch. $\mathcal{D}$ is closed ($g \circ g = \text{id}_a$, $g \circ d = gd$, $g \circ gd = d$). The bases $B_w = \{\text{id}_w\}$, $B_a = \{\text{id}_a\}$ (the orbit $\{\text{id}_a, g, d, gd\}$ is a single free $\mathcal{D}$ -orbit), $B_x = \{\text{id}_x, p\}$, $B_y = \{\text{id}_y, q, s, s_2\}$ witness (L); note s, s_2 are forced into B_y because $\mathcal{D}$ has no non-identity arrow out of x . For (G), any closing $\mathcal{C}$ must pick $t \in \{p, pg\}$ and a basis B_y containing s, s_2 and exactly one of q, qg . Closure needs $s \circ t$, $s_2 \circ t \in B_y$; but $s \circ p = q$ and $s_2 \circ p = qg$ (and symmetrically for pg) force B_y to contain *both* q and qg — impossible. So no $\mathcal{C}$ closes: (G) fails, and by Theorem 5.9 there is no weld. $\square$

Remark 5.16. The cross-arrow d enters $\mathcal{D}$ as a free $\langle g \rangle$ -torsor $\{d, gd\}$ into a ; it enlarges $\mathcal{D}$ to a genuine many-object category and keeps every hom-presheaf free, but it is inert against the branch obstruction at $a \rightarrow x \rightrightarrows y$. Promoting $\mathcal{D}$ from vertex monoids to a category with cross-object arrows does not dissolve the holonomy. This is Theorem 5.9 in its genuinely pairwise form. All claims of this section are machine-checked by finite-category scripts: exhaustive over all 1152 wide subcategories, $\mathcal{K}$ admits no strict factorisation over $\mathcal{D}$; and Theorem 5.4 is confirmed with zero disagreements over all type-correct laws in the tested finite cases.

5.6 The bridge: orchestration composites are Zappa–Szép products

The abstract weld becomes the operative composition tool through one further identification, which is where the container model earns the machinery above. In the polynomial-functor model of interaction [31], an agent that may issue queries from a set S and, for each s , receive a reply from $P(s)$ is the container $p = (S, P) = \sum_{s \in S} y^{P(s)}$; a handoff topology — who may delegate to whom, staying put, concatenating handoffs — is a directed container, i.e. a small category (Definition 3.8). Running *two* orchestrations that share a resource is the interleaving of two directed containers, and this interleaving is a Zappa–Szép product $\mathcal{K} = \mathcal{C} \bowtie \mathcal{D}$.

Proposition 5.17 (Holonomy–composition / Zappa–Szép bridge). (peer-reviewed) *The composite of two shared-resource orchestrations, presented as a small (handoff) category $\mathcal{K}$ with prescribed right factor $\mathcal{D}$ the resource-internal part, is the Zappa–Szép product $\mathcal{C} \bowtie \mathcal{D}$ of directed containers in the sense of Ahman–Uustalu’s distributive-law construction [27], with $\mathcal{C}$ the genuine dispatch/handoff factor. Consequently the joint agent exists if and only if the weld exists, i.e. iff conditions (L) and (G) of Theorem 5.9 both hold; when it does, the mutual action is carried by the distributive law λ of Definition 5.2.*

The identification itself — that the orchestration composite is $\mathcal{C} \bowtie \mathcal{D}$, and hence that its existence is governed by the weld — is the delta over Ahman–Uustalu, whose account builds the joint container when a distributive law is supplied but offers no measure of its absence. Proposition 5.17 is peer-reviewed (Rick); we take Ahman–Uustalu’s construction [27] as prior art and claim none of it. The concrete consequence is a decision procedure: a cheap fibrewise freeness test (L), checkable target by target, gates the more expensive global closure search (G), and only its successes feed the search. The matched-pair calculus of §5.2 then exhibits λ explicitly as the data one extracts to run the composite.

What remains is to make the residual obstruction (G) precise. It is a holonomy: the independently chosen free bases fail to glue, and the failure is a monodromy around the dispatch/return loop — the categorical form of “a flat bundle is trivial iff its holonomy vanishes.” Under the abelian-vertex, trivial-left-action hypothesis the free $\mathcal{D}$ -orbits form a base category, the vertex automorphism groups form a coefficient presheaf, and the defect of a transversal is a normalised 2-cocycle. The classification of (G) as a class in H^2 , and the precise statement that the weld exists iff $[\omega] = 0$, is the subject of the next chapter (Chapter 6); the cohomology conventions it uses are those of §3.5. Here we hand off (G) as exactly that: a global closure condition whose obstruction is a holonomy, waiting to be named.

Chapter 6

The correctness criterion

$$(L) \wedge [\omega] = 0 \in H^2$$

Here the two threads join into the thesis equation. The global condition (G) of the pairwise criterion is identified with the vanishing of a degree-two cohomology class $[\omega] \in H^2$, so that a composite is correct exactly when the local condition holds and the weld obstruction vanishes. The chapter also records the “two ω -sites” result: the degree-1 (prunability / descent) and degree-2 (holonomy / Schreier) obstructions live at genuinely different sites and must not be conflated.

6.1 The thesis equation

The preceding chapter developed the pairwise Zappa–Szép criterion: given a handoff category $\mathcal{K}$ (a directed container, Definition 3.8, under the chain identification $\mathbf{DCont} \cong \mathbf{Cat}$, Theorem 3.9) and a prescribed wide subcategory $\mathcal{D} \subseteq \mathcal{K}$, a complementary factor $\mathcal{C}$ making $\mathcal{K} = \mathcal{C} \bowtie \mathcal{D}$ a Zappa–Szép product exists iff two conditions hold — a *local* freeness condition (L) and a *global* closure condition (G) (see Chapter 5, from which we *receive* (L) and (G) and do not re-derive them). This chapter contributes the identification of the second, global condition with a cohomological vanishing. Writing $\mathrm{Sk}_{\mathcal{C}}$ for the skeleton orbit category and $\mathcal{D}$ for the reply (direction) functor as coefficients (cohomology conventions and the class $[\omega]$ as an LHS transgression: §3.5), the result of the chapter is that the global condition *is* a degree-two obstruction, and hence the whole correctness verdict collapses to a single displayed equivalence:

$$\boxed{\text{a composite is correct} \iff (L) \wedge [\omega] = 0 \in H^2} \tag{6.1}$$

Everything below either proves the right-hand identification (G) $\Leftrightarrow [\omega] = 0$, exhibits the smallest faithful instance where the class is machine-checked, or warns against conflating this degree-two obstruction with the genuinely distinct degree-one obstruction living at another site.

6.2 The global condition is a degree-two class

The bridge from a distributive law to a Zappa–Szép product is classical and is used here as an input, not proved: a strict factorization system $\mathcal{K} = \mathcal{C} \bowtie \mathcal{D}$ is equivalently a distributive law $\lambda : \mathcal{DC} \Rightarrow \mathcal{CD}$ satisfying the matched-pair axioms, the directed-container instance of which is the composition construction of Ahman–Uustalu (*peer-reviewed*). What the source supplies *over* that construction

is a measure of its failure: Ahman–Uustalu’s account builds the composite whenever a distributive law exists, but carries no invariant for when one does not.

Theorem 6.1 ((G) is the vanishing of $[\omega]$; (proved)). *Fix a handoff category $\mathcal{K}$ and a prescribed wide subcategory $\mathcal{D}$ for which the local condition (L) holds — each hom-presheaf is a free right $\mathcal{D}$ -set — and for which the vertex-group hypothesis holds ($\mathcal{D}$ a disjoint union of abelian vertex groups with trivial left action). Then the complementary factor exists iff a single degree-two class vanishes:*

$$\mathcal{C} \bowtie \mathcal{D} \text{ exists} \iff [\omega] = 0 \in H^2(\text{Sk}_{\mathcal{C}}; \mathcal{D}),$$

cohomology of the skeleton of the handoff category with coefficients in the direction functor $\mathcal{D}$ [17, 13]. Equivalently, the global condition (G) of the pairwise criterion is exactly the statement $[\omega] = 0$.

The cohomology itself — Baues–Wirsching cohomology of a small category, and its appearance in Schreier/factorization theory — is classical; the delta over the peer-reviewed construction is the *identification* of the orchestration composite with $\mathcal{C} \bowtie \mathcal{D}$ and hence of its obstruction with this class. Combining Theorem 6.1 with the received pairwise criterion of Chapter 5 yields the boxed equation (6.1) directly: correctness is $(L) \wedge (G)$, and (G) is $[\omega] = 0$.

Remark 6.2 (reading the class as holonomy). The class is the categorical form of “a flat bundle is trivial iff its holonomy vanishes”. Under (L) each partial handoff completes freely; the completions fail to glue into a single serialized joint agent exactly by the monodromy $[\omega]$ around the dispatch/return loop. Thus (L) is completability of the parts and $[\omega] = 0$ is coherence of their gluing — the two halves of (6.1).

6.3 The minimal faithful instance: $[\omega] = \varepsilon$, machine-checked

The smallest place where the answer is “no” is a supervisor dispatching to a worker that may call back before returning — unprotected re-entrancy, the DAO-class failure. This instance is faithful and its class is computed on the nose (*lean-verified*).

Definition 6.3 (the parametrized supervisor–worker category). Fix roles $\mathbf{S}$ (supervisor), $\mathbf{W}$ (worker), $\mathbf{R}$ (return), with the supervisor carrying a one-bit turn token, so $\text{End}(\mathbf{S}) = \{\text{id}, \tau\} \cong \mathbb{Z}/2$. For $\varepsilon \in \mathbb{Z}/2$ let $\mathcal{K}_{\varepsilon}$ have generating handoffs $p, p\tau : \mathbf{S} \rightarrow \mathbf{W}$ and worker outcomes $s, s_2 : \mathbf{W} \rightarrow \mathbf{R}$, with the single length-two composite fixed by

$$s \circ p = q, \quad s_2 \circ p = q \cdot \tau^{\varepsilon}.$$

The prescribed right factor is the token part $\mathcal{D} = \{\text{id}_{\mathbf{S}}, \tau, \text{id}_{\mathbf{W}}, \text{id}_{\mathbf{R}}\}$. The bit ε records whether the re-entrant outcome s_2 *mutates* the token ($\varepsilon = 1$) or *fixes* it ($\varepsilon = 0$, a lock).

The categorical obstruction of Theorem 6.1 reduces, for $\mathcal{K}_{\varepsilon}$, to a finite $\mathbb{F}_2$ cochain complex. The base $\text{Sk}_{\mathcal{C}}$ is the branch $\mathbf{S} \rightarrow \mathbf{W} \rightrightarrows \mathbf{R}$; the normalized (Baues–Wirsching) complex has $C^1 \cong \mathbb{F}_2^2$, $C^2 \cong \mathbb{F}_2^2$, $C^3 = 0$, so every 2-cochain is a cocycle. The coboundary image is the diagonal $B^2 = \langle (1, 1) \rangle$, whence $H^2 = C^2/B^2 \cong \mathbb{F}_2$, realised by the class map $\varphi(a, b) = b - a = a \oplus b$ with $\ker \varphi = B^2$. The defect cocycle of the natural transversal is $\omega_{\varepsilon} = (0, \varepsilon)$, so $\varphi(\omega_{\varepsilon}) = \varepsilon$.

Theorem 6.4 ($[\omega] = \varepsilon$; (*lean-verified*)). *In a sorry-free Lean 4 development using no external library (Containers/Reentrancy.lean), the following are machine-checked over the finite complex above:*

- $\ker \varphi = B^2$ and φ surjective, together giving $H^2 = C^2/B^2 \cong \mathbb{F}_2$;

- $\varphi(\omega_\varepsilon) = \varepsilon$ — the class form of $[\omega] = \varepsilon$;
- $\omega_\varepsilon \in B^2 \iff \varepsilon = 0$ — the membership form of $[\omega] = 0 \iff \varepsilon = 0$.

Consequently, on the minimal faithful model, the joint agent $\mathcal{C} \bowtie \mathcal{D}$ exists iff the worker fixes the token, and unprotected re-entrancy ($\varepsilon = 1$) is the nonzero generator of $H^2 \cong \mathbb{Z}/2$.

Remark 6.5 (the verification boundary, drawn honestly). The Lean development certifies the *finite $\mathbb{F}_2$ class computation* — the last mile, from cochain complex to $[\omega] = \varepsilon$. The *reduction* of the categorical obstruction of Theorem 6.1 to that complex — that $\mathcal{K}_\varepsilon$ satisfies (L) and the vertex-group hypothesis, that $\text{Sk}_{\mathcal{C}}$ is the stated branch, and that the transversal defect is $(0, \varepsilon)$ — is an (*proved*) pen-and-paper input, not formalised. So: the last mile is machine-checked; the first mile is a cited pen-and-paper theorem.

6.4 Two ω -sites: degree-1 descent versus degree-2 Schreier

The chapter closes with a warning that is also the natural lead-in to the following chapter. It is tempting to read *every* obstruction to composing directed containers as the class $[\omega]$ of (6.1). This is false: there are two genuinely different obstruction sites, at two different cohomological degrees, and they must not be conflated (*proved*).

Theorem 6.6 (the two ω -sites are irreducibly distinct; (*proved*)). *The obstruction to composing directed containers factors through two independent sites:*

- a **degree-1** (*prunability / descent*) obstruction — the degree-one kernel-descent class, Ferri’s condition on the ideal generated by the weld; and
- the **degree-2** (*holonomy / Schreier*) obstruction $[\omega] \in H^2$ of Theorem 6.1 — the Zappa–Szép monodromy.

These live at genuinely different sites [16, 14, 15, 18]: *prunability* is a descent condition, not a Schreier H^2 class, and the two decouple — there are witnesses with $[\omega] = 0$ yet nontrivial degree-one obstruction, and conversely. In particular, “*prunability fails*” and “ $[\omega] \neq 0$ ” are not the same verdict, and identifying them is a category error.

Concretely, the degree-1 obstruction is the descent/subfunctor condition governing whether a weld can be *pruned* to a subobject, while the degree-2 class $[\omega]$ governs whether the parts glue into a global serialized agent at all; on suitable witnesses one vanishes while the other does not, so neither dominates the other.

Remark 6.7 (a retired side-reading). An earlier framing predicted that a “total” versus “partial” composition split would be separated by a degree-one invariant. That specific prediction is withdrawn: for the correctness verdict (6.1) the single degree-two class $[\omega]$ is the whole invariant, both the obstruction to existence and, via $H^2 \cong \mathbb{F}_2$ on the minimal model, the label of which system one has. The genuine two-site phenomenon of Theorem 6.6 is a different statement — prunability/descent versus Schreier holonomy — and is not the retired split.

The degree-2 class $[\omega]$ is, moreover, only *one* of three obstruction theories that live on $\mathbf{DCont} \cong \mathbf{Cat}$: the Zappa–Szép holonomy of this chapter, the prunability/descent condition of Theorem 6.6, and a third category-cohomological home. The comparison of all three, and the question of a unifying protomodular home, is the subject of the following chapter 7; the two- ω -sites result above is the entry point to it.

Chapter 7

The H^2 -cluster of obstruction theories

The previous chapter distilled correctness to a single class, $[\omega] = 0 \in H^2$. That is one obstruction theory on the object $\mathbf{DCont} \simeq \mathbf{Cat}$. It is not the only one. Once the Zappa–Szép holonomy is in hand, two further degree-two obstruction theories present themselves on the *same* object — Ferri’s prunability obstruction and the direction-functor cohomology $H^2_{\mathbf{Cat}_B}$ — and the natural reflex is to hope they are three costumes of one theorem, three faces glued by a single universal H^2 . They are not. What this chapter records is the honest shape: a *structured family* of three genuinely distinct theories, with one proved separation between them, one refuted unifier (a valuable negative), and a single organising grammar that survives the refutation — the cluster is a *filtration*, not a product.

7.1 Three homes on one object

(a) **The multiplicative holonomy** $[\omega]$. The Zappa–Szép composition holonomy of Chapters 4–5 is a degree-two class $[\omega] \in H^2(\mathrm{Sk}_{\mathcal{C}}; \mathcal{D})$ over the orbit base $\mathrm{Sk}_{\mathcal{C}}$ (the skeleton of $\mathcal{C}$), with the criterion $(G) \iff [\omega] = 0$ (*proved*). It is a *multiplicative* invariant: it obstructs the existence of a coherent multiplicative transversal $\mathcal{C} \bowtie \mathcal{D}$. Two features fix its character. It vanishes for the *full loop bundle* and, more generally, on any *connected* groupoid, where the orbit base is coarse ($\simeq *$) and $H^{\geq 1}$ is forced to zero; it is nonzero only for a *proper* transversal $\mathcal{D} \subsetneq \pi$, where $\mathrm{Sk}_{\mathcal{C}} = B\Gamma$ carries the Schreier class of the extension $1 \rightarrow \mathcal{D} \rightarrow \pi \rightarrow \Gamma \rightarrow 1$ (nonzero already for $\mathbb{Z}/4$ over $\mathbb{Z}/2$) (*proved*). The degree-two computations underneath it are backed by a machine-checked engine: the cyclic and fibre H^2 modules are *lean-verified*, and the emergent-holonomy count is identified with Ext^0 together with a vanishing theorem killing the higher Ext -tower [17] (*proved; engine lean-verified*).

(b) **Ferri’s additive prunability**. Ferri’s prunability obstruction [18] asks whether the loop bundle G° of a quiver skew brace is an ideal — concretely, whether the loops over each object form an *additive* subgroup that the cross-action carries to loops. It is the *additive twin* of the multiplicative Schreier line of (a). Two facts pin its place in the cluster, and both are corrections of a tempting error.

First, it is *orthogonal* to $[\omega]$: on the full loop bundle the composition holonomy vanishes identically (Proposition A, *proved*) while prunability can still fail, so the two are separate obstructions — the existence of a connected, non-prunable witness that certifies the separation is *computed* [15]. Second, and sharper: prunability is *not* a degree-two class at all. In the untwisted core where pruning is even well posed, it is a *degree-one* datum — the condition that the loop subfunctor be $\dashv$ -flat, i.e. that G° be a *kernel* (a descent/parallel-transport condition), logically independent of the degree-two star-reassembly (H^2) class one degree above it [16] (*proved; the coset reformulation*).

proved, the Schreier-vs-descent crosstab computed). Reading Ferri’s non-prunability of $K_{2,3}$ as an “[ω] $\neq 0$ ” phenomenon is therefore a *category error (refuted)*: $K_{2,3}$ is connected, so $[\omega] = 0$ there, and its obstruction lives one degree lower. *An ideal is a kernel; an extension is a reassembly*.

(c) **The direction-functor cohomology $H^2_{\mathbf{Cat}_B}$** . The third home is external machinery, not ours. Ambra, Duvieusart, and Montoli [19] equip the category $\mathbf{Cat}_B$ of internal categories with a cohomology relative to *Schreier points*: $\mathbf{Cat}_B$ is $\mathcal{S}$ -Mal’tsev there, and H^{n+1} is computed as π_0 of the fibres of a *direction functor*. For a connected groupoid this reduces to the vertex group, $H^2_{\mathbf{Cat}_B}(G; A) \cong H^2(\pi; M)$ (*proved; the external machinery deep-read*), placing $H^2_{\mathbf{Cat}_B}$ on the *additive*/vertex axis alongside prunability and complementary to the (base-axis) $[\omega]$. The authors draw *no* link to containers or Zappa–Szép products, so this is not a scoop of our programme; it is reusable, independently developed infrastructure that the cluster can borrow.

7.2 The refuted unifier, and the honest umbrella

The three homes invite a single question: is there one protomodular / internal-semidirect-product H^2 that merges the multiplicative $[\omega]$ and the additive prunability into faces of a common class? The answer is *no*, and the negative is part of the result (*refuted, 2026-10-09*).

The structural reason is a clean divide. A connected groupoid splits canonically as $G \cong I(\mathrm{ob} G) \times B\pi$ — coarse base times vertex group — and the two obstruction theories are the cohomologies of the two complementary factors. But these factors sit on opposite sides of the *pointed* / *non-pointed* boundary. The vertex factor $B\pi$ is pointed, so the protomodular direction-functor $H^2(\pi; M)$ of (c) applies to it and carries the additive face. The base factor $\mathrm{Sk}_C = I(\mathrm{ob} G)$ is *non-pointed* — it has no zero object — so no protomodular H^2 sees it at all; $[\omega]$ is a nerve/simplicial class, vanishing exactly when the base is connected. A single semi-abelian H^2 cannot straddle the divide, because protomodularity is exactly the structure the non-pointed factor lacks.

What replaces the hoped-for single class is a *filtration*. The honest umbrella is a Künneth / product-nerve spectral sequence, whose abutment $H^2_{\mathrm{cat}}(G; M)$ is filtered rather than split. The natural target is

$$H^2_{\mathrm{cat}}(G; M) \cong H^2(\pi; M) \oplus H^2_{\mathrm{base}}(\mathrm{Sk}_C; M), \quad (7.1)$$

with the direction functor realising the vertex-group projection. This is registered as an open node and it is *speculative* — provable from the $H^2_{\mathbf{Cat}_B}$ reduction and the separation in the connected / field-coefficient case, where the filtration collapses, with genuine content only in the disconnected / proper-transversal case, where it does not. The refuted unifier and the speculative filtration are two readings of one fact: the coupling between the base and vertex faces is a *differential*, not a direct sum.

That last sentence is not a metaphor. On the normal Zappa–Szép slice the categorical cohomology $H^2_{\mathrm{cat}}(G) \cong H^2(\pi; M)$ is the abutment of the Lyndon–Hochschild–Serre spectral sequence of $1 \rightarrow \mathcal{D} \rightarrow \pi \rightarrow \Gamma \rightarrow 1$, whose two degree-two edges are exactly the two faces — the base edge, into which the holonomy class $[\omega] \in H^2(\Gamma; \mathcal{D})$ enters via φ_* , and the fibre edge $H^2(\mathcal{D}; M)^\Gamma$ carrying the additive reassembly class — and whose transgression *is* the holonomy, $d_2(\varphi) = \varphi_*[\omega]$ (*proved*) [13]. (A caveat: the fibre-edge class is a property of the additive coefficient group alone — for the $\mathbb{Z}/4$ slice it is the extension $0 \rightarrow \{0, 2\} \rightarrow \mathbb{Z}/4 \rightarrow \mathbb{Z}/2 \rightarrow 0$, the same for any skew-brace groupoid with additive group $\mathbb{Z}/4$, prunable or not. It *ignores the loop sets*, so it is *not* the prunability obstruction of (b), which is a degree-one condition on the loop sets; the tempting reading “prunability = fibre-edge nonsplitting” was refuted 2026-09-18.) The naive direct sum (7.1) holds *only if* $[\omega] = 0$; whenever $[\omega] \neq 0$ the base edge does not survive to the abutment (it is a d_2 -image, not a summand),

so the two faces are never simultaneously present as independent summands. The converse fails in general: $[\omega] = 0$ does *not* force the sum to be clean, since split extensions need not collapse at E_2 (Totaro; Charlap–Vasquez). The unifier fails for precisely the reason the differential is nonzero.

7.3 The crown: the cluster is filtered, not a product

The same grammar recurs one level up, and this is the connection that turns the cluster from a list into a structure (*proved, 2026-09-23*). Ferri’s decomposition $G \cong (G/G^\odot) \rtimes G^\odot$ has an obstruction that is a *two-stage tower*, not a sum of two independent classes [14]:

- **Tier 2 (degree ≤ 1):** *is $G^\odot$ a kernel?* This is the $\rightarrow$ -descent condition of (b) — for $|L_\lambda| = 2$ a single degree-one cochain — and it vanishes exactly when the quotient $G/G^\odot$ *exists*.
- **Tier 1 (degree 2):** *given the quotient, does the loop extension split?* This is the class $[\omega] \in H^2$ of (a) — which is *only defined once Tier 2 vanishes*, and which is automatically zero on connected G .

The tiers are *filtered*: Tier 1 is undefined until Tier 2 vanishes, because the group $H^2(\text{Sk}_C; \mathcal{D})$ that houses $[\omega]$ is built from a quotient and a coefficient system that do not exist until $G^\odot$ is a kernel. A higher differential lives only on the kernel of the lower one — exactly the shape of the LHS transgression $d_2(\varphi) = \varphi_*[\omega]$ of the previous section, now read at the level of the decomposition itself. On connected G Tier 1 is free, so the decomposition exists iff G is prunable, recovering Ferri’s Corollary 4.8 with its exact cohomological anatomy; $K_{2,3}$ is the canonical instance with Tier 1 = 0 and Tier 2 $\neq 0$ (*proved*).

This is the whole point of assembling the cluster. The three obstruction theories are not three costumes of one theorem, and they are not merged by one universal class; the unifier is refuted. What binds them is a single organising grammar — *filtration, not product* — appearing twice: once inside the abutment (the transgression coupling the two edges) and once above it (the two-tier tower coupling descent to splitting). A degree-one datum must vanish before the degree-two datum is even defined, and a nonzero degree-two class is a differential, not a summand. The honest deliverable of this chapter is that grammar, together with the proved separation and the honestly labelled negative that forced it.

[*Assembly note: separation proved (existence witness computed); prunability-is-degree-1-descent proved; $H^2_{\text{Cat}_B}$ reduction proved; protomodular unifier refuted; product decomposition (7.1) speculative; two-tier tower proved; H^2 -engines lean-verified.*]

Chapter 8

Change-of-base, $\mathbf{Fam}(\mathcal{C}^{\mathrm{op}})$ generality, and the $\{\mathrm{Id}\}$ classification

How far does the equation generalise? The natural setting is containers over a general base, i.e. $\mathbf{Fam}(\mathcal{C}^{\mathrm{op}})$, with change-of-base along functors and monads. This chapter assembles the positive generality results (m -containers, codensity/polynomiality, the four monoidal structures) and the sharp negative: the self-enriched full-and-faithful change-of-base container semantics collapses to $\{\mathrm{Id}\}$ — no nontrivial effect monad supports it. Knowing the boundary is itself the result.

8.1 The base as a degree of freedom

Recall from Chapter 3 that a container is an object of $\mathbf{Fam}(\mathbf{Set}^{\mathrm{op}})$ (Notation 3.6) and that its extension is a left Kan extension along the representable embedding (Theorem 3.7),

$$\llbracket S, P \rrbracket X = \sum_{s \in S} \mathbf{Set}(P_s, X) = \sum_{s \in S} X^{P_s}.$$

The formula has exactly one silent parameter: the hom-set $\mathbf{Set}(P_s, X)$. Replace $\mathbf{Set}$ by a category $\mathcal{C}$ and read positions out in $\mathcal{C}$'s internal hom, and the same syntax presents a wholly different world.

Definition 8.1 (Containers over a base). For a closed symmetric monoidal category $(\mathcal{C}, \otimes, I, [-, -])$ with small coproducts, $\mathbf{Fam}(\mathcal{C}^{\mathrm{op}})$ has objects (S, P) with S a set of shapes and $P_s \in \mathcal{C}$ a *position object* for each s , and morphisms $(S, P) \rightarrow (T, Q)$ a shape map $f: S \rightarrow T$ together with, per s , a $\mathcal{C}$ -map $Q_{f(s)} \rightarrow P_s$ (contravariant in positions). The *extension* into $\mathcal{C}$ -enriched endofunctors is

$$\llbracket S, P \rrbracket = \sum_{s \in S} [P_s, -] : \mathcal{C} \rightarrow \mathcal{C}.$$

A single construction thus covers the whole programme, selected by the base: $\mathcal{C} = \mathbf{Set}$ gives ordinary containers; $\mathcal{C} = \mathbf{Kl}(M)$ for a monad M gives *effectful* (M -)containers, whose positions are read in a Kleisli category; $\mathcal{C} = \mathbf{Vec}$ gives linear containers; and $\mathcal{C} = \mathbf{Cat}$ recovers directed containers and hence small categories (Chapter 3, §3.3). The assignment $\mathcal{C} \mapsto \mathbf{Fam}(\mathcal{C}^{\mathrm{op}})$ is functorial: a functor $F: \mathcal{C} \rightarrow \mathcal{D}$ induces $\mathbf{Fam}(F^{\mathrm{op}})$, acting as F on positions and identically on shapes, and the total category assembling all bases is a bifibration over $\mathbf{Set}$ whose fibrewise opposite is the contravariance in positions [43]. *Change of base* along $F: \mathbf{Set} \rightarrow \mathcal{C}$ is the transport of the standard theory into $\mathcal{C}$; when F is (co)monoidal it carries (co)monoidal structure, and that is where effects and directedness enter.

The remainder of the chapter is a suite of positive generality results (§§8.2–8.5), a diagnosis of the linear wall (§8.6), and the collapse that fixes the exact boundary of a *uniform* enriched semantics (§8.7).

8.2 Positive suite I: m -containers, codensity, polynomiality

The Kleisli base is the central instance and the one where two Kan-theoretic invariants of the effect first appear. Fix a monad $M = (M, \eta, \mu)$ on **Set**; an M -container is a container (S, P) whose positions are read in $\mathbf{Kl}(M)$, with M -extension $\llbracket S, P \rrbracket_M X = \sum_s \mathbf{Set}(P_s, MX)$. All results of this section are *peer-reviewed* (Rick).

Theorem 8.2 (Identification and factorisation; *peer-reviewed*). $M\text{-}\mathbf{Cont} = \mathbf{Fam}(\mathbf{Kl}(M)^{\text{op}})$, and as endofunctors, naturally in (S, P) ,

$$\llbracket S, P \rrbracket_M = \llbracket S, P \rrbracket \circ M.$$

So an M -container functor is an *ordinary polynomial functor with a trailing M* . This one factorisation is the lever for everything below: every structural question about $\llbracket - \rrbracket_M$ becomes a question about the single trailing M . The proof is a direct unfolding of the two hom-descriptions.

Theorem 8.3 (Faithful always; fullness = codensity; *peer-reviewed*). $\llbracket - \rrbracket_M : M\text{-}\mathbf{Cont} \rightarrow [\mathbf{Set}, \mathbf{Set}]$ is always faithful (the Kleisli unit η is a section, so container data is read back off any induced transformation). It is full — hence fully faithful — if and only if (i) each functor $\mathbf{Set}(A, M-)$ is connected, and (ii) M is codense, i.e. the canonical monad map $M \Rightarrow \text{Ran}_M M$ into its own codensity monad is an isomorphism. If M is affine ($M1 \cong 1$) then (i) holds automatically, so affine + codense $\Rightarrow$ fully faithful.

The engine is that precomposition $(-)\circ M$ has right adjoint Ran_M , so every hom-set $\mathbf{Nat}(\mathbf{Set}(A, M-), H)$ is a value of the right Kan extension along M ; the fullness obstruction is exactly the failure of the unit $M \Rightarrow \text{Ran}_M M = \Phi$ (the codensity monad, $\Phi(A) = \mathbf{Nat}(\mathbf{Set}(A, M-), M)$) to be invertible. The verdict is genuinely selective.

Example 8.4 (The effects sort themselves). The finite-distribution monad $\mathbf{D}$ is affine and codense, so $\llbracket - \rrbracket_{\mathbf{D}}$ is fully faithful (modulo the standard rigidity $\mathbf{Nat}(\mathbf{D}, \mathbf{D}) = \{\text{id}\}$). By contrast **Maybe**, exceptions $X + E$, the writer $E \times (-)$ and the reader $(-)^R$ ($|R| \geq 2$) all fail codensity and so lose fullness. The reader is affine but *not* codense ($\Phi(A) = (RA)^R \neq A^R = MA$), showing (i) and (ii) are independent conditions.

Remark 8.5 (The wrong criterion). The natural guess “ $\llbracket - \rrbracket_M$ is full iff M preserves coproducts” is false for the target $[\mathbf{Set}, \mathbf{Set}]$: that is the criterion for the coarser Kleisli-enriched target, and it gives the wrong verdict on $\mathbf{D}$. The writer $E \times (-)$ preserves coproducts yet fails codensity ($\Phi(1) = |E|^{|E|} \neq |E|$ for $|E| \geq 2$), a clean refutation. The controlling invariant is the codensity monad $\text{Ran}_M M$, not coproduct preservation.

The second invariant governs composition. Call M *polynomial* if $M \cong \sum_{i \in I} (-)^{B_i}$, i.e. $M = \llbracket [M] \rrbracket$ for a container $[M]$.

Theorem 8.6 (Composition = polynomiality; *peer-reviewed*). (Sufficiency, unconditional.) If M is polynomial then $p \triangleleft_M q := p \triangleleft [M] \triangleleft q$ satisfies $\llbracket p \triangleleft_M q \rrbracket_M \cong \llbracket p \rrbracket_M \circ \llbracket q \rrbracket_M$ naturally, so the essential image of $\llbracket - \rrbracket_M$ is closed under composition. The operation $\triangleleft_M$ is associative but has a two-sided unit up to container isomorphism only when $M = \text{Id}$: it is semigroupal, not monoidal. (Necessity, conditional.)

The converse — closure $\Rightarrow M$ polynomial — holds on an exactly delimited class strictly larger than the fully symmetric world: for super-polynomial M (a cardinality law, ruling out powerset, $\mathbf{D}$, ultrafilter, continuation); for analytic M with $M\emptyset = \emptyset$ (right-cancellation of plethysm, at unbounded degree and nesting); for analytic M of bounded wreath depth (an imprimitivity invariant subsuming multiset and symmetric powers); and for the free-algebra monad of every unital non-flat operad with a commutative binary composite and $\Delta_{C_2} \notin \mathcal{S}$. It fails for partially symmetric unital operads: a ternary operation with cyclic (C_3) slot-symmetry and an absorbed unit is non-polynomial yet composition-closed.

The one genuinely open point is the converse of the last clause ($\Delta_{C_2} \in \mathcal{S} \Rightarrow \text{closed}$), verified for the all-binary inhabitant at small arities and open in general. Within the fully symmetric world — which contains all the applied effect monads (multiset, distribution, powerset) — composition $\iff$ flatness $\iff$ polynomiality.

Corollary 8.7 (The trade-off; peer-reviewed). *Fullness (codensity) and composability (polynomiality) are independent invariants of M that trade oppositely across the standard effects. The writer $E \times (-)$ is polynomial, hence composes, but is not codense, hence not full. The distribution monad $\mathbf{D}$ is affine and codense, hence full, but is not polynomial, hence does not compose. Probability keeps faithful naming and loses composition; error keeps composition and loses faithful naming. In particular the folklore “composes $\iff$ commutative and affine” is false: $\mathbf{D}$ is affine and commutative but does not compose, while **Maybe** is neither yet does.*

8.3 Positive suite II: the change-of-base census

Beyond the Kleisli identification, the same frame $\text{Fam}((-)^{\text{op}})$ absorbs a census of monad/comonad transports, each a change of base or a fibrewise transport read in the appropriate variance. All are *proved* in the companion material.

Proposition 8.8 (Position transfer: monads become comonads; *proved*). *Because positions are contravariant, a monad M on **Set** postcomposed on the position family, $G(S, P) = (S, M \circ P)$, is a comonad on **Cont** — with counit η and comultiplication μ on the fibres — and this is a biconditional: the comonad laws for G hold iff the monad laws for M do (single-shape containers realise every fibre). Globally $G = \text{Lan}_{(S, P)} M$. Dually a comonad on **Set** yields a monad on **Cont**. This is the slogan of the whole frame in miniature: a monad on the base is a comonad on the fibrewise opposite.*

Proposition 8.9 (The $\forall/\exists$ liftings; *proved*). *Quantifying a predicate container over positions gives two liftings. The $\exists$ -lifting is exactly composition, $E = \triangleleft$, a bifunctor on all of **Cont** with unit $y^{(-)}$. The $\forall$ -lifting $A = \Pi$ is a bifunctor only on cartesian morphisms in its first argument (the pushforward is indexed by set-sections of the backward position map, which are unique iff the map is a bijection), and on that domain it is a left module of $(\mathbf{Cont}, \triangleleft, y^{(-)})$. The mixed comparison $AX(EYC) \rightarrow E(AXY)C$ is an isomorphism iff X is linear. This is the semantic content of M lifting to containers iff M is cartesian, seen one level down.*

Proposition 8.10 (The free monad on a container; *proved*). *A signature is a container, and its free monad — the type of effect trees — is a monad on **Cont** built by iterated $\triangleleft$ -composition. In the linear instance over a monoidal base, the free $\triangleleft$ -monoid on the one-shape container $C = (\star, A)$ is $C^* = (\mathbb{N}, (A^{\otimes n})_n)$, with $\llbracket C^* \rrbracket(W) = \bigoplus_{n \geq 0} A^{\otimes n} \otimes W = T(A) \otimes W$ the tensor algebra: over **Vec** this is O’Neill’s free monad, realised functorially.*

The composition question of §8.2 is the fifth census entry, and the position transfer and liftings are the fibrewise ones; naming the frame turns a list of theorems into a single diagram.

8.4 Positive suite III: the four monoidal structures

The base **Cont** itself carries four monoidal structures, previewed in Chapter 3, §3.4: the coproduct $+$, the product $\times$, the Dirichlet tensor $\otimes$ (Definition 3.10; on containers the shapes and positions both multiply, cf. Proposition 3.11), and the substitution operator $\triangleleft$. Their full classification is a topic in its own right; we record here only the map and the closure results the rest of the dossier uses. All are *proved*; the Lean certifications are noted where they exist.

Why exactly these four. Every monoidal structure $(\star, I)$ on **Set** induces one on **Cont** by Day convolution [1, 31]: with $p \otimes_\star q = (S_p \times S_q, (s, t) \mapsto p_s \star q_t)$ and unit y^I . Call a structure *convolutional* if it preserves coproducts in each variable and closes the representables.

Theorem 8.11 (Day classification; *proved*). $(\star, I) \mapsto (\otimes_\star, y^I)$ is an equivalence from monoidal structures on **Set** to convolutional monoidal structures on **Cont**. Two of the four are its values, $\times = \otimes_{(+, \emptyset)}$ and $\otimes = \otimes_{(\times, 1)}$; the coproduct $+$ and the substitution $\triangleleft$ lie outside the family (they fail coproduct-preservation in a variable). This explains the “upside-down” arithmetic — the product’s positions are a disjoint union because it is the Day convolution of the coproduct of sets, $y^{A+B} \cong y^A \times y^B$.

Theorem 8.12 (Pointwise = product; *proved*). Among all monoidal structures on **Cont** — Day or not — the categorical product $\times$ is the unique one whose extension is the pointwise product of functors, $\llbracket p \ast q \rrbracket \cong \llbracket p \rrbracket \times \llbracket q \rrbracket$. Being pointwise is the rare property, not the generic one; in particular $\llbracket - \rrbracket$ is strong monoidal into $([\mathbf{Set}, \mathbf{Set}], \times)$ only for the cartesian structure.

Theorem 8.13 (The comparitor; *proved*). For each p , $p \otimes (-)$ is the left Kan extension of $p \triangleleft (-)$ along the representable embedding, and the comparison $p \otimes q \rightarrow p \triangleleft q$ is the counit of the resulting coreflection: $p \otimes (-)$ is the terminal coproduct-preserving approximation to $p \triangleleft (-)$. The Dirichlet tensor is the substitution operator with its dependency switched off — and not by fiat: it is the Day-ification of $\triangleleft$, which is why the two agree on representables ($y^A \otimes y^B = y^{A \times B} = y^A \triangleleft y^B$).

Closures. The Dirichlet tensor is closed, with internal hom the container whose *shapes* are the container morphisms [31]; this closure is machine-checked — $(\mathbf{Cont}, \otimes, y^{(-)})$ is a closed monoidal category in Lean 4, currying and uncurrying inverse on the nose (*lean-verified*, `Dirichlet.lean`). The substitution operator $\triangleleft$ is *not* left-closed but carries a right coclosure, whose comonoids are the directed containers, hence the small categories [26]. And **Cont** is cartesian closed but *not* locally cartesian closed, so it does not model dependent type theory — a fact worth stating plainly, and one the frontier chapter revisits.

Interaction. The pair $(\otimes, \triangleleft)$ is a normal duoidal category [33], and its interchange law houses the comparitor. Its double comonoids — objects that are comonoids for both tensors compatibly — are classified: they are exactly the *sets of commutative monoids*, by a fibrewise Eckmann–Hilton collapse (*proved*), correcting the plausible guess “the degenerate polynomials” in both directions.

8.5 Positive suite IV: when does substitution exist over a base?

The Dirichlet tensor $p \otimes q = (S \times T, (P_s \otimes Q_t))$ exists over *every* closed monoidal base. The substitution product $\triangleleft$ — the one whose comonoids are categories — is different: it mostly does not exist, and where it does, its existence is not a matter of degree but a sharp theorem. Call $\mathcal{C}$ $\triangleleft$ -*admissible* if the image of $\llbracket - \rrbracket$ is closed under composition, $\llbracket p \triangleleft q \rrbracket \cong \llbracket p \rrbracket \circ \llbracket q \rrbracket$ for all p, q . All results below are *proved* (the $\mathbf{Set} \times \mathbf{Vec}_{\text{fd}}$ result on an explicitly stated locus).

Everything is controlled by one necessary condition, obtained by evaluating any candidate composite at the terminal object $\mathbf{1} = [0, X]$.

Lemma 8.14 (Shape Lemma; *proved*). *If $\mathcal{C}$ is $\triangleleft$ -admissible then for every P and every set T the internal hom $[P, T \cdot \mathbf{1}]$ is a copower of $\mathbf{1}$ (here $T \cdot \mathbf{1} = \coprod_T \mathbf{1}$). The base must see the external shape set inside itself, as a copower of $\mathbf{1}$.*

On the set-like extreme admissibility does not merely coexist with the rest of the structure — it forces it.

Theorem 8.15 (Extensive rigidity and the forced package; *proved*). *Let $\mathcal{C}$ be nontrivial, infinitary leftextensive and cartesian closed. If $\mathcal{C}$ is $\triangleleft$ -admissible then its unit $\mathbf{1}$ is connected ($\mathcal{C}(\mathbf{1}, -)$ preserves small coproducts); consequently $\llbracket - \rrbracket$ is fully faithful and the reindexing functor $F_q = \text{Fam}(\llbracket q \rrbracket^{\text{op}})$ is left adjoint to $(-) \triangleleft q$ for every q . In particular $\mathbf{Set} \times \mathbf{Set}$ is inadmissible, its unit $(1, 1)$ being disconnected. Fullness of $\llbracket - \rrbracket$ and existence of the reindexing left adjoint are literally the same comparison lemma applied twice.*

The converse fails — connectedness alone is not enough — and the exact condition upgrades the necessary Shape Lemma to a sufficient one.

Theorem 8.16 (Characterisation; *proved*). *For $\mathcal{C}$ nontrivial, infinitary leftextensive and cartesian closed, $\mathcal{C}$ is $\triangleleft$ -admissible iff the Shape Lemma holds, equivalently iff an external distributive law holds. When $\mathcal{C}$ has a connected-components functor $\pi_0 \dashv (-) \cdot \mathbf{1}$ (as every Grothendieck topos does), this reads: $\mathcal{C}$ is $\triangleleft$ -admissible iff π_0 preserves finite products.*

This single criterion unifies the two ways admissibility can fail. The disconnected unit of $\mathbf{Set} \times \mathbf{Set}$ and the connected-unit obstruction below are one phenomenon, $\pi_0(X \times Y) \not\cong \pi_0 X \times \pi_0 Y$; it is co-extensive with the *stable-units* property of the connected-components reflection in categorical Galois theory [48], though established here via the distributive law rather than by descent.

Theorem 8.17 (Connected is not sufficient; *proved*). *The Artin gluing $\text{Gl}((-)^2)$ is a Grothendieck topos with a connected unit that is nonetheless $\triangleleft$ -inadmissible: its π_0 is not multiplicative, since for the single edge K one has $\pi_0(K \times K) = 2 \neq 1$ (Weichsel’s theorem [47]). This is exactly the gap between the internal (Gambino–Kock) composition, which every topos supports [32], and the external-shape composition of $\llbracket - \rrbracket$.*

On the opposite, additive pole the situation inverts. Over $\mathbf{Vec}_{\text{fd}}$ every object is copower-tiny, so $\triangleleft$ collapses onto the Dirichlet tensor, $p \triangleleft q = p \otimes q$, and $(-) \triangleleft q$ has a left adjoint only when q has a single shape; the unit is always disconnected. Between the poles the admissible region is thin but inhabited: $\mathbf{Set} \times \mathbf{Vec}_{\text{fd}}$ is admissible (on its locus) with a disconnected unit, combining a rigid distributive factor with a flexible copower-tiny one — so the extensive-pole characterisation does not extend to a global one (*proved*). The organising fact behind the whole map is determinacy.

Theorem 8.18 (Determinacy; *proved*). *If $\mathbf{1}$ is connected then $\llbracket - \rrbracket$ is injective on objects up to isomorphism, so $p \triangleleft q$ is determined by $\mathcal{C}$ and admissibility is a property of $\mathcal{C}$. If $\mathbf{1}$ is disconnected this can fail: over $\mathbf{Vec}_{\text{fd}}$ the objects $(\{*\}, k^2)$ and $(\{1, 2\}, k)$ present the same functor $X \mapsto X \oplus X$ but are not isomorphic, so $\triangleleft$ becomes a choice and the base-change question stops being about $\mathcal{C}$ at all.*

8.6 The linear wall: $\mathbf{Vec}$ is neither full nor faithful

The sharpest instrument for separating the four notions of “container in a category” — external families $\text{Fam}(\mathcal{C}^{\text{op}})$, indexed dependent-polynomial functors over an LCC base [32], fibrational containers [43], and Walker’s locally subcartesian closed weakening [44] — is a base that fails *both* extensivity and local cartesian closure. The category $\mathbf{Vec}$ of vector spaces is exactly such a base: the comparison $\coprod^{\mathbf{Set}} \rightarrow \oplus$ is not an isomorphism, and there is no internal dependent product Π . Off the “good” zone a base fails for exactly one of two independent reasons, loss of extensivity (shapes no longer recoverable, $\llbracket - \rrbracket$ no longer full) or loss of internal Π ($\triangleleft$ and closedness collapse); toposes lose neither, quantales and affine bases lose only the second (reached by Walker’s affine repair), and $\mathbf{Vec}$ and $R\text{-Mod}$ lose both, so only the external approach reaches them. Three theorems make this quantitative; all are *proved*.

Theorem 8.19 (Fullness = unit-connectedness; *proved*). *For any closed symmetric monoidal $\mathcal{C}$ with small coproducts, $\llbracket - \rrbracket$ is fully faithful iff the monoidal unit I is connected ($\mathcal{C}(I, -)$ preserves small coproducts). This is not extensivity of $\mathcal{C}$: $\mathbf{Set} \times \mathbf{Set}$ is (l)extensive yet its unit $(1, 1)$ is disconnected and $\llbracket - \rrbracket$ is not full (the two “crossed” transformations are unrealised), while $\mathbf{Set}$ is full not because it is extensive but because 1 is connected. It recovers Abbott–Altenkirch–Ghani [24] by naming that hidden hypothesis. The reconstruction of a container from an abstract familiarly representable functor is Diers’ [42], and uses extensivity of the codomain, not the base — which is why the folklore said “extensivity”.*

Corollary 8.20 (The linear collapse; *proved*). *Over $\mathcal{C} = \mathbf{Vec}$ the unit is k and $\mathbf{Vec}(k, -)$ is the underlying-set functor, which does not preserve $\oplus$: the comparison $\coprod_t^{\mathbf{Set}} X_t \rightarrow \oplus_t X_t$ is neither surjective (it misses cross-summand sums) nor injective (it collapses the zero vector, which sits in every summand). Hence over $\mathbf{Vec}$ the extension $\llbracket - \rrbracket$ is neither full nor faithful — the failure of faithfulness being precisely that $\oplus$ collapses tagged zeros. This is the honest strength of the linear denotation: the on-the-nose container/functor correspondence that holds over $\mathbf{Set}$ breaks in both directions over an additive base.*

The other two theorems measure the remaining obstructions, on independent seams. The Dirichlet tensor $\otimes$ is closed iff a familial-representability functor Φ is representable; over the linear base this holds *only* on the corner $\text{Fam}_{\text{fin}}(\mathbf{Vec}_{\text{fd}}^{\text{op}})$ of finite shape-sets and finite-dimensional positions (the load-bearing conjunct is *dualizable-and-summable*), failing over both $\text{Fam}(\mathbf{Vec}_{\text{fd}}^{\text{op}})$ and $\text{Fam}(\mathbf{Vec}^{\text{op}})$ by dual mechanisms (*proved*). The substitution product $\triangleleft$ is left-closed exactly where it collapses onto $\otimes$ — over a base whose positions are dualizable (tiny), $\triangleleft = \otimes$ and the left hom is the Dirichlet hom — so it is left-closed on $\text{Fam}_{\text{fin}}(\mathbf{Vec}_{\text{fd}}^{\text{op}})$ and obstructed over every extensive base (*proved*). Extensivity is thus *opposed* to $\triangleleft$ -left-closedness: the property whose failure costs fullness and Dirichlet closedness is exactly what would buy substitution closedness. These are the seams Weber’s distributivity- δ and familial-functor theories [45, 46] place on different rungs; the composition-monoid refinement of the indexed corner is [50], and the general-base tensor and its comonoids-as-enriched-categories are Dorta–Jarvis–Niu [49].

8.7 The $\{\text{Id}\}$ collapse

The linear wall shows a *specific* base failing. The final result is stronger and structural: over *no* nontrivial effect does the natural self-enriched semantics enjoy a uniform full-and-faithfulness theorem. The apparent counterexample — the reader monad, whose Kleisli category is cartesian closed — is the pivot that makes the point.

There are three extensions of an M -container into endofunctors, with three notions of naturality. Fix a monad M and write $\mathcal{C} = \text{Kl}(M)$.

- (A) the *ordinary* extension $\llbracket - \rrbracket_M : \text{Fam}(\text{Kl}(M)^{\text{op}}) \rightarrow [\mathbf{Set}, \mathbf{Set}]$, whose morphisms are pure natural transformations: faithful always, full iff M codense and each $\mathbf{Set}(A, M-)$ connected (Theorem 8.3);
- (B) the *Kleisli-presheaf* extension $\llbracket - \rrbracket^{\text{Kl}} : \text{Fam}(\text{Kl}(M)^{\text{op}}) \rightarrow [\text{Kl}(M), \mathbf{Set}]$, a coproduct of representables: by ordinary co-Yoneda it is *always* full and faithful, at the price that its morphisms are Kleisli-natural;
- (C) the *self-enriched* extension, defined when $\mathcal{C}$ is monoidal closed, $\llbracket S, P \rrbracket = \sum_s [P_s, -] : \mathcal{C} \rightarrow \mathcal{C}$, which realises the enriched-Yoneda desideratum $\text{Nat}(\sum_s [P_s, -], F) \cong \prod_s F(P_s)$ [39].

The three are one factorisation, $\llbracket - \rrbracket_M = R \circ \llbracket - \rrbracket^{\text{Kl}}$ with $R = (-) \circ F$ restriction along the free functor $F : \mathbf{Set} \rightarrow \text{Kl}(M)$; all loss of ordinary fullness lives in R , and its size is exactly the codensity monad $\text{Ran}_M M$.

Two independent invariants govern (C):

$$\mathcal{CL} = \{M : \text{Kl}(M) \text{ monoidal closed}\}, \quad \mathcal{UN} = \{M : \text{the monoidal unit of } \text{Kl}(M) \text{ is connected}\}.$$

Closedness ($\mathcal{CL}$, available by Kock's criterion [40, 41]) makes (C) *exist*; connectedness ($\mathcal{UN}$, i.e. M preserves small coproducts, since $\text{Kl}(M)(1, X) = MX$) makes it *full* (Theorem 8.19). They are logically independent — the cartesian-closed reader sits in $\mathcal{CL} \setminus \mathcal{UN}$, the writer in $\mathcal{UN} \setminus \mathcal{CL}$ — and over a Kleisli base they meet in exactly one monad.

Lemma 8.21 (Closed + initial $\Rightarrow$ terminal; *lean-verified*). *In any symmetric monoidal closed category with an initial object $\emptyset$, the object $[\emptyset, X]$ is terminal for any X . (Each $(-) \otimes A$ is a left adjoint, so $C \otimes \emptyset \cong \emptyset$, whence $\mathcal{K}(C, [\emptyset, X]) \cong \mathcal{K}(\emptyset, X) \cong 1$.) The existence of the terminal object uses only monoidal closedness and an initial object — neither cartesian closedness nor connectedness. The Step-2 collapse $E \cong 1$ below is another matter: there connectedness ($\mathcal{UN}$, via the writer form) does visible, unavoidable work, namely in excluding a zero object $\top \cong \emptyset$. Indeed **Maybe** ($\top \cong \emptyset$, $M\emptyset \cong 1$), powerset ($\top \cong \emptyset$), and the reader ($\top \cong 1$) all lie in $\mathcal{CL}$ yet are eliminated only by the writer form, i.e. by membership in $\mathcal{UN}$.*

Theorem 8.22 ($\mathcal{CL} \cap \mathcal{UN} = \{\text{Id}\}$; *proved, core lean-verified*). *Let M be a commutative strong monad on $\mathbf{Set}$. The self-enriched M -container extension $\sum_s [P_s, -] : \text{Kl}(M) \rightarrow \text{Kl}(M)$ exists and is full and faithful if and only if $M \cong \text{Id}$. Equivalently $\mathcal{CL} \cap \mathcal{UN} = \{\text{Id}\}$: no nontrivial effect — not even an idempotent (localization) monad — carries a uniform enriched full-and-faithful semantics.*

Proof sketch. Fullness of (C) $\iff M \in \mathcal{CL} \cap \mathcal{UN}$. *Step 1:* $\mathcal{UN}$ forces the writer form. If M preserves small coproducts then the copower-comparison $T \times M1 \rightarrow MT$ is a natural bijection, so $M \cong (-) \times E$ with $E = M1$, and the monad laws are the laws of a (commutative) monoid on E . *Step 2:* $\mathcal{CL}$ forces $E \cong 1$. The base $\text{Kl}(M)$ is monoidal closed with initial object $\emptyset$ (since $\text{Kl}(M)(\emptyset, B) = 1$), so Lemma 8.21 supplies a terminal $\top$; then $\mathbf{Set}(C, M\top) = \text{Kl}(M)(C, \top) \cong 1$ forces $M\top \cong 1$, and the writer form gives $|\top| \cdot |E| = 1$, so $E \cong 1$ and $M \cong \text{Id}$. *Step 3:* Id qualifies ($\mathbf{Set}$ cartesian closed, $\mathbf{Set}(1, -) = \text{Id}$ preserves coproducts). $\square$

The core of this collapse — the closed-plus-initial-forces-terminal engine of Lemma 8.21, together with the resulting identity-only classification — is machine-checked in Lean 4 (*lean-verified*, `EnrichedFFCCollapse`).

Corollary 8.23 (No idempotent escape; *proved*). *No nontrivial idempotent monad lies in $\mathcal{CL} \cap \mathcal{UN}$: membership in $\mathcal{UN}$ forces the writer form, and a writer monad is idempotent only for $E \cong 1$; the terminal monad $LX = 1$ already fails connectedness, $L(1 + 1) = 1 \neq 2 = L1 + L1$.*

Remark 8.24 (The reader, reconciled). The collapse dissolves the apparent paradox. Cartesian closedness of $\mathbf{Kl}(\mathbf{Reader}_S)$ buys the *availability* of (C), not its fullness: the unit 1 has global-points functor $(-)^S$, which fails to preserve coproducts for $|S| \geq 2$, so (C) is not full. The reader’s failure is enrichment-invariant — non-codensity in (A), a disconnected unit in (C), one obstruction wearing two masks — and (B) is full only for Kleisli-natural, not pure, morphisms. Enrichment relocates the obstruction; it does not remove it.

The boundary, and the handoff. The chapter has traced how far the correctness equation generalises. It survives change of base wholesale — as *m*-containers with two computable invariants (§8.2), as the census of transports (§8.3), as the four monoidal structures with their closures (§8.4), and as substitution over exactly the π_0 -multiplicative bases (§8.5) — but it does *not* survive as a *uniform self-enriched full-and-faithful* semantics over any nontrivial effect: that is the $\{\text{Id}\}$ wall of Theorem 8.22, a statement about general effect monads. A different wall stands over a *single* additive base: the linear denotation over **Vec** (§8.6) is already neither full nor faithful, and the Vec-admissibility question of whether $F = \text{Hom}(E, \bigoplus_{\mathbb{N}} -)$ is free pushes the boundary into set theory. That set-theoretic frontier — where the verdict becomes independent of ZFC — is the subject of the next chapter, Chapter 9.

Chapter 9

The set-theoretic frontier: Vec-admissibility and the Gap- S arc

This is the frontier of the Theory pillar, and it is open. Every obstruction so far in this document — the Zappa–Szép cocycle $[\omega] \in H^2$, the prunability descent class, the direction-functor cohomology — has been a concrete, *absolute* invariant: computed once, its verdict holds in every model of set theory. The admissibility of the linear base **Vec** is the first node of the programme that is not like this. Whether **Vec** is $\triangleleft$ -admissible turns out to be governed by the cardinal arithmetic of the continuum, and its exact truth value changes from one model of ZFC to another.

The result of this chapter is a single, sharp identity. The composability of the linear base reduces to the projectivity of one module M over the full linear ring $S = \text{End}_k(E)$, and the *minimal number of generators* of that module is exactly the dominating number:

$$\boxed{\mu_S(M) = \mathfrak{d}}.$$

A cardinal characteristic of the continuum is the load-bearing invariant of a functor-category admissibility question. From this identity the conjecture is *false* in every model with $\mathfrak{d} = \mathfrak{c}$ — in particular under CH, under Martin’s Axiom, and in the Cohen model — and the only surviving positive case is squeezed into the narrow window $2^{\aleph_0} = 2^{\mathfrak{d}}$, whose canonical witness is the random-real model. Whether M is projective there is the programme’s deepest open problem. The chapter ends the Theory pillar on that live edge, as it should.

Everything in §§9.1–9.3 is *proved* in ZFC; the residual of §9.5 is *open*. Nothing in this chapter is lean-verified, and nothing is externally published (Honest Ledger, items 1 and 4).

9.1 From admissibility to a module

Recall the denotation of a family $(S, P) \in \text{Fam}(\mathcal{C}^{\text{op}})$, the object that presents a container over the base $\mathcal{C}$:

$$\llbracket S, P \rrbracket X = \coprod_{s \in S} \mathcal{C}(P(s), X).$$

The base $\mathcal{C}$ is *$\triangleleft$ -admissible* when the image of $\llbracket - \rrbracket$ is closed under composition of endofunctors — the minimal condition under which the composition $p \triangleleft q$ of two directed containers has any denotational meaning over $\mathcal{C}$ (Chapter 9 uses only this weakest form; a non-existence result proved against it is maximally strong). Over **Set** every base is admissible; over **Vec** the unit object is 0 and the shape set of a composite becomes invisible, and the whole difficulty concentrates in a single instance.

Conjecture 9.1 (6.2 / Gap- S). *Over a countable field k , the additive endofunctor $F = \prod_{r \in \mathbb{N}} (\bigoplus_{m \in \mathbb{N}} (-))$ of $\mathbf{Vec}_k$ is a coproduct of representables; equivalently, $\mathbf{Vec}_k$ is $\triangleleft$ -admissible at the instance presenting $\bigoplus_{\mathbb{N}}(-)$.*

The reduction to algebra is the spine of the arc and is proved in the module note [8]. Let $E = k^{(\mathbb{N})}$ and $S = \text{End}_k(E)$, the ring of column-finite $\mathbb{N} \times \mathbb{N}$ matrices over k ; for countable k , $|S| = \mathfrak{c}$. Restriction to the subcategory of countable-dimensional spaces identifies additive endofunctors with left S -modules via $T \mapsto T(E)$, sending $\text{Nat}(T, T') \cong \text{Hom}_S(T(E), T'(E))$ and Ext^1 to Ext^1 . Under this dictionary the identity functor maps to the simple projective E , the countable coproduct $\bigoplus_{\mathbb{N}}(-)$ to $P = \bigoplus_{\mathbb{N}} E$ (a countably generated, non-finitely-generated projective), and the functor of the conjecture to

$$M = \prod_{r \in \mathbb{N}} P = \prod_{r \in \mathbb{N}} \bigoplus_{m \in \mathbb{N}} E.$$

Theorem 9.2 (Module reduction [8, Thm. 1–2]; proved, ZFC). *$\mathbf{Vec}_k$ is $\triangleleft$ -admissible at the instance of Conjecture 9.1 if and only if M is projective as a left S -module.*

Two structural facts about S drive everything below, and both are peculiar to the *noncommutative* full linear ring. First, S is von Neumann regular, so every module is flat and every submodule is pure. Second — and this is the identity that makes the arc subtle rather than routine —

$$\prod_{r \in \mathbb{N}} E \cong S \quad \text{as left } S\text{-modules}, \quad (*)$$

free of rank one, because a column-finite matrix *is* an arbitrary sequence of its columns [8, Thm. 2]. The identity $(*)$ is the homogeneity of S : its minimal idempotents are mutually conjugate, so a countable product of copies of the regular module reassembles into S . A commutative ring has no such homogeneity, and this is exactly why the tempting commutative surrogate k^ω is not faithful to the problem [12]; the phenomenon is intrinsically noncommutative.

Identity $(*)$ has an immediate and destructive consequence for the obvious attack. One would like to prove M non-projective by a Baer–Specker “product-is-bigger-than-coproduct” argument, exhibiting an exotic map that cannot lift. But $(*)$ makes the relevant product *free*, so the dual of M stays large ($\dim_k \text{Hom}_S(M, E) = \mathfrak{c}$) and no size-gap argument can succeed: the Baer–Specker mechanism is neutralised [8, Thm. 4]. Any refutation must count something finer than cardinality, and any positive proof must survive the collapse of the naive obstruction. This is the shape of the whole difficulty.

Before counting, one reduction removes the shortest independence route. Call a module *R-projective* (Dual Baer) if every map into a cyclic quotient S/I lifts through $S \twoheadrightarrow S/I$; projective always implies *R-projective*.

Theorem 9.3 (Dual-Baer collapse [9]; proved, ZFC). *M is projective if and only if M is *R-projective*.*

The proof uses that S contains a free left ideal of rank $\mathfrak{c}$ (an almost-disjoint-family construction inside $(*)$), which upgrades *R-projectivity* to projectivity for any module generated by at most $\mathfrak{c}$ elements; M is one, since $|M| \leq \mathfrak{c}$. Theorem 9.3 matters because it forecloses Trlifaj’s packaged independence theorem for the Dual Baer Criterion, which requires the ring to be *small semiartinian* — a hypothesis S fails. Whatever the status of Conjecture 9.1, it is not an off-the-shelf instance of known Dual-Baer independence.

9.2 The first count: false under CH

The original refutation counts the dual of a hypothetical decomposition. It rests on two ZFC facts about the fixed module M : it is *not* countably generated [8, §5] (a Bergman-style diagonalisation, invisible to cardinality since $|M| = \mathfrak{c}$), and its dual is small, $\dim_k \text{Hom}_S(M, E) = 2^{\aleph_0}$ [8, Thm. F].

Theorem 9.4 (First count [10]; proved, ZFC). *If M is projective then $2^{\aleph_0} = 2^{\aleph_1}$. Consequently, if $2^{\aleph_0} < 2^{\aleph_1}$ — in particular under CH and under GCH — then M is not projective, $\mathbf{Vec}$ is $\triangleleft$ -inadmissible, and Conjecture 9.1 is false.*

Proof sketch. By Kaplansky's theorem a projective module is a direct sum of countably generated submodules, $M = \bigoplus_{i \in I} M_i$ with each $M_i \neq 0$. If $|I| \leq \aleph_0$ then M is countably generated, a contradiction; so $|I| \geq \aleph_1$. Each nonzero countably generated projective has nonzero dual, so $\text{Hom}_S(M, E) = \prod_{i \in I} \text{Hom}_S(M_i, E)$ contains a copy of k^I , giving $\dim_k \text{Hom}_S(M, E) \geq 2^{|I|} \geq 2^{\aleph_1}$ by Erdős–Kaplansky. But the dual has dimension $2^{\aleph_0}$, so $2^{\aleph_1} \leq 2^{\aleph_0}$, forcing equality. $\square$

The parallel with the Baer–Specker group $\mathbb{Z}^{\mathbb{N}}$ is exact and instructive. There the same schema gives an *absolute* contradiction ($\mathbb{Z}^{\mathbb{N}}$ is non-free in ZFC), because Specker's theorem collapses the dual to countable rank. Over S the freeness $(*)$ keeps the dual large, and the contradiction degrades from “ $\aleph_0$ versus $2^{\mathfrak{c}}$ ” to merely “ $2^{\aleph_0}$ versus $2^{\aleph_1}$.” The void Baer–Specker is precisely what demotes an absolute refutation to a CH-conditional one: same count, different dual size, different verdict.

Theorem 9.4 is silent when $2^{\aleph_0} = 2^{\aleph_1}$. The original note nominated $\text{MA} + 2^{\aleph_0} = \aleph_2$ as the natural residual model in which independence might live. The next section removes exactly that model.

9.3 The sharp invariant: $\mu_S(M) = \mathfrak{d}$

The first count sees only the crude cut $|I| \geq \aleph_1$: the index set of a Kaplansky decomposition is uncountable. The sharpening computes its true size. Write $\mathfrak{d}$ for the dominating number — the least size of a family in $(\mathbb{N}^{\mathbb{N}}, \leq^*)$ cofinal for eventual domination, equivalently the least number of compact sets covering Baire space $\mathbb{N}^{\mathbb{N}}$ (a set is compact iff it is contained in some $\prod_r [0, g(r)]$). In ZFC, $\aleph_1 \leq \mathfrak{b} \leq \mathfrak{d} \leq \mathfrak{c}$.

For $x \in M$ write $\text{supp}(x) = \{(r, m) : x_{r,m} \neq 0\}$; each fibre $\text{supp}(x)_r = \{m : (r, m) \in \text{supp}(x)\}$ is finite (this is the block-column constraint defining $M \subseteq \prod_{r,m} E$). The action only shrinks supports: $\text{supp}(A \cdot x) \subseteq \text{supp}(x)$ for $A \in S$, because $(A \cdot x)_{r,m} = A(x_{r,m})$ vanishes wherever $x_{r,m}$ does.

Theorem 9.5 (The generator number is $\mathfrak{d}$; proved, ZFC). $\mu_S(M) = \mathfrak{d}$.

Proof. Lower bound $\mu_S(M) \geq \mathfrak{d}$. Let G generate M ; put $\kappa = |G|$, so $\kappa \geq \aleph_0$ and $|[G]^{<\omega}| = \kappa$. Fix $0 \neq v \in E$, and for each $f \in \mathbb{N}^{\mathbb{N}}$ let $x^f \in M$ be the *graph element* $x^f_{r,f(r)} = v$, all other entries 0; it lies in M (one nonzero entry per row) and has $\text{supp}(x^f) = \{(r, f(r)) : r\}$. Since G generates, $x^f = \sum_{g \in F} A_g g$ for a finite $F \subseteq G$ and $A_g \in S$; by support-shrinking and additivity, $\text{supp}(x^f) \subseteq \bigcup_{g \in F} \text{supp}(g)$. Setting $U_r^F = \bigcup_{g \in F} \text{supp}(g)_r$ (finite, a finite union of finite sets), this says $f(r) \in U_r^F$ for every r , i.e. $f \in \prod_r U_r^F$. Hence

$$\mathbb{N}^{\mathbb{N}} = \bigcup_{F \in [G]^{<\omega}} \prod_r U_r^F,$$

a covering of Baire space by $\leq \kappa$ compact sets. The least such number is $\mathfrak{d}$, so $\kappa \geq \mathfrak{d}$.

Upper bound $\mu_S(M) \leq \mathfrak{d}$. Fix a $\leq^*$ -dominating family $\{h_\xi : \xi < \mathfrak{d}\} \subseteq \mathbb{N}^\mathbb{N}$. For each ξ set $M_\xi = \{x \in M : x_{r,m} = 0 \text{ for } m > h_\xi(r)\}$. As a set $M_\xi = \prod_r E^{h_\xi(r)+1}$; reindexing by the countably infinite set $\{(r, m) : m \leq h_\xi(r)\}$ this is $\prod_{\mathbb{N}} E \cong S$ by $(*)$, so M_ξ is *cyclic*, $M_\xi = S \cdot g_\xi$. Given $x \in M$, let $N_x(r) = \max \text{supp}(x)_r$; choose ξ with $N_x \leq^* h_\xi$, say $N_x(r) \leq h_\xi(r)$ for $r \geq r_0$. Then $x = x \cdot \mathbf{1}_{r \geq r_0} + x \cdot \mathbf{1}_{r < r_0}$: the first summand lies in M_ξ , the second has finite total support, hence lies in the countably generated $M_{\text{fin}} = \bigoplus_{r,m} E$. Thus $\{g_\xi : \xi < \mathfrak{d}\}$ together with countably many matrix units generate M , and $\mu_S(M) \leq \mathfrak{d} + \aleph_0 = \mathfrak{d}$. $\square$

The proof turns on the two faces of $\mathfrak{d}$: the lower bound is $\mathfrak{d} = \text{cov}(\text{compact})$, the upper bound is $\mathfrak{d} = \min |\text{dominating family}|$. The upper bound uses $(*)$ in an essential way — the same freeness that voids Baer–Specker is what collapses each “staircase region” M_ξ to a *cyclic* module, so that the only cost of generating M is the growth of supports (which is $\mathfrak{d}$), never the values.

9.4 The sharpened refutation, and why it matters

The generator number feeds the dual count directly, replacing the crude $|I| \geq \aleph_1$ by $|I| \geq \mathfrak{d}$.

Theorem 9.6 (Sharpened count; proved, ZFC). *If M is projective then $2^\mathfrak{d} = 2^{\aleph_0}$. Consequently, if $2^{\aleph_0} < 2^\mathfrak{d}$ then M is not projective and Conjecture 9.1 is false.*

Proof. A Kaplansky decomposition $M = \bigoplus_{i \in I} M_i$ has $|I| \geq \mu_S(M) = \mathfrak{d}$ by Theorem 9.5 (the summands’ generators generate M). As in Theorem 9.4, $\dim_k \text{Hom}_S(M, E) \geq 2^{|I|} \geq 2^\mathfrak{d}$, while $\dim_k \text{Hom}_S(M, E) = 2^{\aleph_0}$; so $2^\mathfrak{d} \leq 2^{\aleph_0}$, and equality follows since $\mathfrak{d} \geq \aleph_0$. $\square$

Corollary 9.7 (The false region; proved, ZFC). *If $\mathfrak{d} = \mathfrak{c}$ then Conjecture 9.1 is false. In particular it is false under CH, under Martin’s Axiom $\text{MA} + \neg\text{CH}$, and in the Cohen model.*

Proof. If $\mathfrak{d} = \mathfrak{c}$ then $2^\mathfrak{d} = 2^\mathfrak{c} > 2^{\aleph_0}$ by Cantor, so Theorem 9.6 applies. CH gives $\mathfrak{d} = \aleph_1 = \mathfrak{c}$; MA gives $\mathfrak{d} = \mathfrak{c}$; and adding Cohen reals makes $\text{cov}(\mathcal{M}) = \mathfrak{c} \leq \mathfrak{d}$, so $\mathfrak{d} = \mathfrak{c}$ there as well [6, 5]. $\square$

This strictly enlarges the false region of the first count, and it does so by eating the very model the first count had to leave open. Under $\text{MA} + 2^{\aleph_0} = \aleph_2$ one has $2^{\aleph_0} = 2^{\aleph_1}$, so Theorem 9.4 is silent — this was the nominated residual model for possible independence. But MA forces $\mathfrak{d} = \mathfrak{c}$, so Corollary 9.7 *refutes* the conjecture there. The sharpening removes $\text{MA} + \neg\text{CH}$ from the residual entirely.

The crown-jewel reading. Theorem 9.5 is the reason this arc is worth its length. A question about the composition of directed containers over the linear base — whether two containers substitute into a third — has been reduced to a single cardinal, and that cardinal is not $\aleph_1$ or $\mathfrak{c}$ but the dominating number $\mathfrak{d}$, a genuine characteristic from the Cichoń diagram. The composability boundary of a categorical semantics is thus located inside the combinatorics of Baire space. To the author’s knowledge this is the first place in the container programme — and one of very few places anywhere in the semantics of computation — where a definability/composability question is answered by a cardinal characteristic of the continuum rather than by a fixed invariant.

Varying the ground division ring cannot force the issue. One might hope to make the refutation *absolute* by tuning a parameter: replace k by a division ring D , so $S_D = \text{End}_D(V)$ with $V = D^{(\mathbb{N})}$. The freeness $(*)$ holds over every D by the same matrix-unit homogeneity, so the generator number is unchanged — $\mu_{S_D}(M) = \mathfrak{d}$ for every D — while only the dual ceiling moves:

M projective now forces $|D|^{\mathfrak{d}} = |D|^{\aleph_0}$. The lower bound stays pinned at $\mathfrak{d}$; only one side of the coincidence slides with $|D|$.

Corollary 9.8 (Easton silence; proved, ZFC). *For every division ring D it is consistent with ZFC that $|D|^{\mathfrak{d}} = |D|^{\aleph_0}$; hence no choice of D makes the counting obstruction prove M non-projective in ZFC.*

For $|D| \leq \mathfrak{c}$ this is the random-real model ($\mathfrak{d} = \aleph_1$, $2^{\aleph_0} = 2^{\aleph_1}$); for larger $|D|$ with $\text{cf}(|D|) > \mathfrak{d}$ it follows from Easton's theorem on the freedom of the continuum function. The honest scope matters: Corollary 9.8 shows the *lever* cannot force falsity in ZFC; it does *not* construct M projective. Varying D relocates the computation but does not decide it.

9.5 The residual, honestly

The two counts together confine any surviving positive case to the window

$$2^{\aleph_0} = 2^{\mathfrak{d}}, \quad \text{equivalently} \quad \mathfrak{d} < \mathfrak{c},$$

strictly smaller than the first count's $2^{\aleph_0} = 2^{\aleph_1}$. The canonical model is the *random-real model*, where $\mathfrak{b} = \mathfrak{d} = \aleph_1 < \mathfrak{c} = \aleph_2$ and $2^{\aleph_0} = 2^{\aleph_1} = \aleph_2$; there $\mu_S(M) = \mathfrak{d} = \aleph_1$ and the count permits a decomposition $M = \bigoplus_{\aleph_1} (\text{countably generated projective})$. Whether that decomposition exists is the open crux.

Why the naive positive route fails is itself diagnostic, and worth stating precisely, because it locates where any future proof must act. One is tempted to build the decomposition from the staircase filtration $M = \bigcup_{\xi < \aleph_1} M_\xi$ of Theorem 9.5: each $M_\xi \cong S$ is projective, and each successor quotient $M_{h_{\xi+1}}/M_{h_\xi} \cong \prod_r E^{h_{\xi+1}(r)-h_\xi(r)}$ is again a countable product of E 's, hence projective. By Eklof's lemma a *continuous* well-ordered filtration with projective quotients would force M projective. The route fails at a single point: the filtration is *not continuous at limits*. For a limit λ of cofinality ω , $\bigcup_{\xi < \lambda} M_{h_\xi} \subsetneq M_{\sup_\xi h_\xi}$ — an element whose support grows to $\sup_\xi h_\xi$ pointwise but is dominated by no single h_ξ is excluded — and repairing the gap by taking unions destroys the clean $\prod E$ form of the successor quotient. The non-projectivity, if present, is stored in the post-limit quotients. This is exactly the Whitehead/ Γ -invariant signature: the obstruction lives at the limit stages of an $\aleph_1$ -filtration, not in the generator count. $\mu_S(M) = \mathfrak{d}$ supplies the *length* of the filtration but not a projective *continuous* one.

Question 9.9 (the residual; open). Is M projective in the random-real model — equivalently, can $\text{pd}_S M = 0$ hold in a model of $2^{\aleph_0} = 2^{\mathfrak{d}}$? A ZFC proof that $\text{pd}_S M \geq 1$ in every model makes Conjecture 9.1 false in ZFC; a decomposition in the random-real model makes it independent of ZFC.

The reframing through projective dimension [11] makes the question recognisable: the cokernel of M inside the free module $\Phi \cong S$ is $\prod_{\mathbb{N}} N_0$, a countable product of copies of a fixed projective-dimension-one seed $N_0 = S/I \cong \prod_{\mathbb{N}} E / \bigoplus_{\mathbb{N}} E$, so Conjecture 9.1 is exactly the Osofsky–Jensen question of whether a product raises projective dimension, sharpened to name the continuum. The ZFC bracket $1 \leq \text{pd}_S M \leq d+1$ (with $d = \min\{n : 2^{\aleph_0} \leq \aleph_n\}$) holds unconditionally.

The evidence is genuinely two-sided, and it is worth stating at honest weight. *Toward independence*: the operative threshold is the collapse $2^{\aleph_0} = 2^{\mathfrak{d}}$, an equalisation of the continuum function that forcing can arrange while preserving reflection, and $\text{pd}_S M = 0$ is a Whitehead-shaped statement about an $\aleph_1$ -indexed decomposition — the archetype of an independent assertion; the large dual (from $(*)$) removes the absolute obstruction that makes $\mathbb{Z}^{\mathbb{N}}$'s non-freeness a theorem of ZFC.

Toward false-in-ZFC: M is a canonical continuum-sized product, the exact shape of Osofsky’s high-projective-dimension witnesses; and the failure of “flat Mittag-Leffler $\Rightarrow$ projective” over S is already an ω_1 -scale phenomenon witnessed in ZFC [7], which the residual window constrains the continuum function without obviously demoting. The structural evidence leans toward independence, but this is an evidential lean, not a proof, and false-in-ZFC remains a live competitor. Deciding between them is Question 9.9, and it must be settled *natively* over S : the commutative surrogate k^ω is not faithful, precisely because the freeness $(*)$ that carries the whole reduction is intrinsically noncommutative [12].

This is where the Theory pillar ends: not on a solved theorem but on a sharp open one, with the boundary of the linear base’s composability pinned to a cardinal characteristic of the continuum and the last question — projectivity in the random-real model — honestly unanswered.

Part II

Applications

Chapter 10

GA / island-model composition

Island-model genetic algorithms are the cleanest application in the dossier: a migration graph is literally a directed container, and the question of whether two coupled island systems form one coherent evolutionary system is literally the question of whether their comonads admit a distributive law. This chapter transports the categorical GA framework of Langer–Vega [52] onto the $(L) \wedge [\omega] = 0$ spine of the dossier. The headline is a dictionary: *a migration topology is a distributive law, and coherence of the composite is the existence of a Zappa–Szép product*. Everything combinatorial here is *proved* (it reproves the directed-container theorems of Ahman–Uustalu in the GA setting); everything about empirical diversity dynamics is imported at the grade the source states — *computed* — and is not inflated.

10.1 Migration topology is a distributive law

Fix a genome type Σ and let $\text{Pop}(\Sigma)$ be scored populations. In the Langer–Vega framework [52] genetic operators are Kleisli morphisms for an evolution monad $\mathcal{T} = \text{Reader}(\Gamma) \times \text{Writer}(\Lambda) \times \text{State}(S)$ (configuration, diversity log, PRNG state), pipelines are Kleisli composites, and an *island functor* I_G lifts a strategy to an n -island system migrating along a directed graph $G = (V, E)$.

The structural claim of this chapter is that the migration graph is not merely a parameter but is itself a piece of composition data of the kind Part I studies.

Proposition 10.1 (Migration topology = directed container; *proved*). *Let $G = (V, E)$ be a migration graph and $\text{Free}(G)$ its free category (objects V , morphisms directed paths, composition concatenation). Then $\text{Free}(G)$ is a directed container D_G , hence — by the equivalence chain $\mathbf{DCont} \simeq \mathbf{Cat}$ of Chapter 4 — a $\triangleleft$ -comonoid, and it induces a comonad $W_G = \llbracket D_G \rrbracket$ on $\mathbf{Set}$.*

This is condition (L) made concrete: *each* island model composes coherently with itself because its migration graph freely generates a category, and every small category is a directed container. The interesting question is (G) — the weld of *two* island models — and it is exactly a distributive law.

Theorem 10.2 (Coherence = Zappa–Szép existence; *proved*). *Let G, H be migration graphs with reachability comonads W_G, W_H . The composite $W_H \circ W_G$ carries a comonad structure — i.e. the two island models compose into one coherent evolutionary system — if and only if the directed containers D_G, D_H admit a distributive law, equivalently a Zappa–Szép product $D_H \bowtie D_G$. This is the specialisation of the Ahman–Uustalu correspondence (Chapter 5) to D_G, D_H ; the weld obstruction is the class $[\omega]$ of Chapter 6, and coherence is $[\omega] = 0$.*

So the migration *topology* of the composite *is* the distributive law threading G -paths through H -paths, and whether that composite is a well-defined evolutionary system is whether the Zappa–Szép product exists. The rest of the chapter unpacks the two halves — the single-system comonad (Level 1) and the pairwise weld (Level 2) — and then imports the empirical diversity story honestly.

10.2 Level 1: the reachability comonad

Definition 10.3 (Migration container). The *one-step migration container* of G is $C_G = (V, N_G)$ with $N_G(i) = \{i\} \cup \{j : (i, j) \in E\}$ the closed out-neighbourhood. In general C_G is *not* a directed container: the shift $\oplus$ would demand $N_G(j) \subseteq N_G(i)$ whenever $j \in N_G(i)$, i.e. transitive closure, which the ring, the star, and most topologies fail.

The fix is to pass to the free category, which is exactly Proposition 10.1: the positions become *paths* rather than edges, and concatenation supplies an associative, unital shift.

Definition 10.4 (Reachability comonad; *proved*). D_G has shapes V , positions $P(i) = \text{Path}_G(i)$ (finite directed paths from i), root $\mathbf{o}(i) = \varepsilon_i$ (empty path), sub-shape $i \downarrow p = \text{target}(p)$, and shift $p \oplus q = p \cdot q$. Its extension is the *reachability comonad*

$$W_G(X) = \llbracket D_G \rrbracket(X) = \sum_{i \in V} X^{\text{Path}_G(i)}, \quad \varepsilon(i, v) = v(\varepsilon_i), \quad \delta(i, v) = (i, p \mapsto (\text{target}(p), q \mapsto v(p \cdot q))).$$

The comonad laws hold because paths form a category: the counit laws are “empty path is identity” and the coassociativity law is associativity of concatenation.

Remark 10.5 (What the comonad means for GAs). An element $(i, v) \in W_G(\text{Pop}(\Sigma))$ assigns a population to every island reachable from i : the *contextual view* from island i . EXTRACT returns i ’s own population; DUPLICATE equips each reachable j with *its* contextual view. The island functor I_G of [52] is the one-step truncation: I_G^k uses paths of length $\leq k$, and W_G is the $k \rightarrow \infty$ limit $\text{Path}_G(i) = \bigcup_k \text{Path}_G^{\leq k}(i)$. This is the precise sense in which the comonad is the “full multi-step” structure of which migration is the truncation.

A migration *policy change* $G \rightarrow H$ on a fixed vertex set is a container morphism $C_G \rightarrow C_H$: a shape map $u : V \rightarrow V$ reindexing islands and a *backward* position map $f_i : N_H(ui) \rightarrow N_G(i)$ pulling each new neighbour back to an old one (*proved, by the representation theorem* **Cont** $\cong \text{Fam}(\mathbf{Set}^{\text{op}})$ of Chapter 3). The backward direction is not incidental: migration pulls individuals *from* neighbours, and a map of representables is a backward map of representing objects. Raising connectivity (say $C_5 \rightarrow K_5$) forces f_i to be non-injective — the inevitable data loss is the “cost” of denser migration.

10.3 Level 2: composition via Zappa–Szép — the weld condition

By Chapter 5, a distributive law of directed containers D_G, D_H is four operations (**st**, **sp**, **ts**, **tp**) — “how a movement in one graph shifts the shape and induces a position in the other” — subject to root, sub-shape, and shift compatibilities, the last of which are precisely the Zappa–Szép conditions (ZS1–ZS4). Their content is a *mutual action*: each system’s moves rewrite the other’s state, and the rewrite must decompose consistently across multi-step paths. Theorem 2 of Ahman–Uustalu then assembles the composite directed container $D_H \bowtie D_G$ (shapes $V_H \times V_G$; positions interleaved H/G -paths, each step in one graph potentially redirecting steps in the other), whose comonad is $W_H W_G$.

Example 10.6 (Independent islands: trivial weld; *proved*). When the two GAs do not interact, $\text{st}(i, j, q) = i$, $\text{sp}(i, j, q) = \circ_G(i)$, $\text{ts}(i, j, p) = j$, $\text{tp}(i, j, p) = \circ_H(j)$: a move in one leaves the other's shape fixed and lands on its root. The Zappa–Szép conditions collapse to the root laws, $[\omega] = 0$, and $W_H W_G$ is the product comonad. This is the $(L) \wedge [\omega] = 0$ criterion at its trivial fixed point.

Example 10.7 (Fitness-coupled migration; *computed*). Let GA_1 evolve neural weights on a ring $G = C_n$ and GA_2 evolve architectures on a star $H = K_{1,m}$, coupled so that migrating an architecture to spoke j triggers weight migration to the island corresponding to j . Then $\text{st}(i, \text{hub}, (\text{hub} \rightarrow j))$ redirects i to the G -island matching spoke j . The Zappa–Szép conditions constrain which G -island/ H -spoke correspondences are coherent: *not every coupling is a valid weld*, and the invalid ones are exactly those where the mutual influence fails to decompose across composite paths — i.e. where $[\omega] \neq 0$. The specific correspondences here are illustrative, checked on small n, m rather than proved in general.

This is the GA face of the dossier's thesis. Two evolutionary subsystems fuse into one correct system precisely when their local pieces are directed containers $((L)$, automatic here) *and* the global weld obstruction vanishes ($[\omega] = 0$, i.e. a Zappa–Szép product exists). The open problem of [52] — construct the laxator explicitly and check its coherence — becomes: *for which pairs D_G, D_H does a distributive law exist, and what is $D_H \bowtie D_G$?* The Zappa–Szép conditions are a decision procedure — finitely many equations in $(\text{st}, \text{sp}, \text{ts}, \text{tp})$ — so for small graphs composability is decidable by enumeration (*computed*).

10.4 Diversity dynamics and the laxator as obstruction

The empirical driver of [52] was that migration topology governs diversity dynamics. We report those findings at the source's grade and do not promote them.

Remark 10.8 (Empirical concordance; *computed*). The source reports that migration topology determines diversity dynamics with perfect rank concordance, *Kendall's* $W = 1.0$, across the tested configurations, and a universal ordering for sustained diversity

$$\text{none} > \text{ring} > \text{star} > \text{random} > \text{fully connected}.$$

Both figures are *computed* (empirical, from the reported experiments), not proved. In particular $W = 1.0$ is the exact value the source states and is not to be read as a theorem. A separately reported “5.5× structural constant” for time-varying random topologies is likewise *computed* and is flagged below as future work, not a result of this chapter.

The container perspective offers a *structural reading* of this ordering via the algebraic connectivity $\lambda_2(G)$, which governs how fast the truncated path spaces $\text{Path}_G^{\leq k}$ saturate to the full path space. Large λ_2 (e.g. K_n) saturates quickly, so W_G collapses to the global view early and diversity erodes; small λ_2 (e.g. C_n , large n) keeps islands' contextual views distinct for many generations, sustaining diversity. This gives a mechanism — *more room in the comonad when λ_2 is small* — consistent with the ordering.

Remark 10.9 (Laxator as obstruction; *computed / heuristic*). The laxator $\varphi_G : I_G(\sigma_1) \circ_K I_G(\sigma_2) \Rightarrow I_G(\sigma_1 \circ_K \sigma_2)$ measures the failure of I_G to preserve Kleisli composition strictly. Two ends are on firm ground and one is a sketch. It is the *identity iff G has no edges* (*proved*): then $\text{Path}_G(i) = \{\varepsilon_i\}$, $W_G(X) = X^n$ is the product comonad, and $I_G(\sigma) = \sigma^n$ commutes with composition. When G has edges, $I_G(\sigma_1) \circ_K I_G(\sigma_2)$ migrates twice while $I_G(\sigma_1 \circ_K \sigma_2)$ migrates once, and φ_G is exactly that extra migration step acting as a *partial* distributive law — the distance from the trivial weld of

Example 10.6. The claim that $\|\varphi_G\|$ tracks $\lambda_2(G)$ (fast mixing $\Rightarrow$ large laxator, K_n maximal, long C_n minimal) is the source’s *proof-sketch-level* argument and is carried here as *computed / heuristic*, not proved; the precise identification of $\|\varphi_G\|$ with a norm of the obstruction cocycle is left open.

Remark 10.10 (Toward Level 3: adaptive topologies). When the migration topology rewires over time (driven by fitness or diversity feedback) the directed container is no longer fixed: the system becomes a dynamical system in Spivak’s **Org** over **Poly** [31, 29], the polynomial (interface) changing each step and a horizontal morphism encoding the rewiring. The reported “5.5 $\times$ ” constant for time-varying random topologies would, on this reading, be the numerical signature of a genuinely lax horizontal morphism (*speculative*). This is flagged as future work; no theorem is claimed.

Chapter 11

Blockchain: re-entrancy as a failed distributive law

The DAO attack of 2016 drained roughly a third of a live fund through one structural flaw: a contract called out to an external party *before* updating its own bookkeeping, and the callee re-entered the still-open call and spent against stale state. This chapter argues that this failure mode is not a heuristic “smell” but a computable invariant. A smart contract is a coalgebra for a polynomial functor; running two contracts that share a resource is not composition along a fixed handoff topology (which cannot fail) but the *interleaving* of two, and the interleaving of two directed containers is a Zappa–Szép product whose existence is governed by a single degree-two cohomology class $[\omega]$. Unprotected re-entrancy is exactly $[\omega] \neq 0$: the weld fails, and the failure is a wrong interleaving. On the minimal faithful model the class equals a concrete one-bit cocycle, and that identity is machine-checked in Lean 4.

11.1 The idea: re-entrancy is a failed distributive law

We state the thesis before building the apparatus, because the apparatus exists to make this one sentence precise and checkable.

Composing two shared-resource contracts requires a distributive law of their handoff categories; that law is a Zappa–Szép weld $\mathcal{C} \bowtie \mathcal{D}$; the weld exists iff a degree-two class $[\omega] \in H^2(\mathrm{Sk}_{\mathcal{C}}; \Delta)$ vanishes; and unprotected re-entrancy is precisely the case $[\omega] \neq 0$.

The three moves — contract as coalgebra, interaction as Zappa–Szép product, obstruction as an H^2 class — are developed in the next three sections. The payoff, in Section 11.4, is that on the smallest instance faithful to the DAO pattern the class collapses to a single field element $\varepsilon \in \mathbb{F}_2$: the bit recording whether the re-entrant branch *mutates* the caller’s shared state. The identity $[\omega] = \varepsilon$ is then a finite fact settled by a proof assistant with no room for hand-waving.

Two grades travel with this chapter, matching the two source strata. The core obstruction computation — that the interleaving obstruction is a degree-two class and that on the minimal model it equals the token-mutation bit, machine-checked in Lean — is carried at [PEER-REVIEWED] (referee: Rick). The Ethereum/EVM modelling that instantiates it — blocks as containers, contracts as coalgebras, the DAO as the s_2 branch — is carried at [COMPUTED]: a translation exercise, not a theorem.

Remark 11.1 (why the degree is two, not zero or one). Multi-agent-systems work places cellular-sheaf obstructions of degree H^0 (consensus) and H^1 (identifiability) on the agents’ *communication*

graph, asking whether the parties currently agree. The container obstruction sits one degree up, on the *handoff* category, and asks the prior question: can the parties be fused into one consistent joint system at all? Re-entrancy is the smallest place where the answer is no. This degree contrast is argued in the source obstruction note [13].

11.2 Smart contracts as polynomial coalgebras

We recall the container model of a contract, at grade [COMPUTED], following the Ethereum-as-container development.

Definition 11.2 (contract as a polynomial coalgebra). A *smart contract* with state set Σ , input type I (function signatures with arguments), and output type O (return values and emitted events) is a coalgebra for the polynomial functor

$$p(X) = (O \times X)^I, \quad \text{step: } \Sigma \longrightarrow (O \times \Sigma)^I.$$

Given a state σ and an input i , the contract emits an output $o \in O$ and a new state $\sigma' \in \Sigma$.

As a container, $p(X) = (O \times X)^I$ has a single shape (the interface is fixed), positions indexed by I , and at each position the value $O \times X$: an observable output paired with a next state. This is a Mealy machine; had the output depended only on the state we would have the Moore machine $p(X) = O \times X^I$.

Example 11.3 (the Gateway marketplace). The Gateway NFT marketplace contract is such a coalgebra. Its state Σ is the tuple of on-chain mappings (token data, listings, authorities, pending fees, the next-id counter); its input I is the disjoint union of signatures `mint` + `list` + `buy` + `delist` + `reserveTokenId` + `withdrawFees` + $\dots$; its output O ranges over success/revert, the events `Minted`, `Listed`, `Sold`, `Transfer`, and return values. On input `buy(tokenId=42, Commercial)` the step function verifies the listing, distributes payment (fee to minter, remainder to seller), mints a license token, emits events, and returns the updated state.

The blockchain itself supplies the ambient directed container into which contracts are embedded: a block is the container $H \triangleleft P$ separating header (shape) from transaction slots (positions), and the chain of blocks is a directed container — equivalently a small category whose objects are blocks and whose morphisms are ancestor paths, the free category on the parent graph [26]. We record this only to locate contracts; the obstruction lives one level down, in how two contracts *interleave*.

Remark 11.4 (the pivot from the source). The Ethereum note observes that two contracts interacting is a composition of polynomial coalgebras, with interface the composition product $p_A \triangleleft p_B$ on **Poly**, and states — at grade [COMPUTED] — that “re-entrancy is what happens when the composition $p_A \triangleleft p_B$ has a non-trivial distributive law with $p_B \triangleleft p_A$, [and] the re-entrancy guard is precisely the refusal to admit such a distributive law.” The remainder of this chapter is the promotion of that sentence to a measured invariant: the distributive law is a Zappa–Szép weld, and its failure is a degree-two class.

11.3 Composition, interleaving, and the Zappa–Szép weld

We now move to grade [PEER-REVIEWED]. The vocabulary is Ahman–Uustalu’s; the identification of the interleaving obstruction with a cohomology class is the source note’s contribution.

Definition 11.5 (directed container, handoff category). A directed container equips a container (S, P) with a root, a sub-position operation, and a position-concatenation. By Ahman–Uustalu, directed containers are exactly small categories: objects are shapes, morphisms out of s are positions $P(s)$, identities are roots, composition is concatenation [26]. We call this category the *handoff category* and write $\triangleleft$ for the composition product of directed containers.

Definition 11.6 (Zappa–Szép product). Let $\mathcal{K}$ be a small category and $\mathcal{D} \subseteq \mathcal{K}$ a wide subcategory (same objects). A *Zappa–Szép product* $\mathcal{K} = \mathcal{C} \bowtie \mathcal{D}$ is a second wide subcategory $\mathcal{C}$ such that every morphism of $\mathcal{K}$ factors *uniquely* as a $\mathcal{C}$ -morphism followed by a $\mathcal{D}$ -morphism — a strict factorization system, equivalently a distributive law $\lambda: \mathcal{D}\mathcal{C} \Rightarrow \mathcal{C}\mathcal{D}$ satisfying the matched-pair axioms. For directed containers this is the distributive-law composition of Ahman–Uustalu [26]; its data is a matched pair of mutual actions.

The pivotal fact, absent from Ahman–Uustalu and supplied by the source, is that the *complementary factor $\mathcal{C}$ may or may not exist*. Composition along a single fixed topology is a functor and cannot fail; the interleaving of two shared-resource systems is a Zappa–Szép product, and its existence is exactly what the obstruction measures. The identification is:

$$\mathcal{C} \bowtie \mathcal{D} \text{ exists} \quad \Longleftrightarrow \quad [\omega] = 0 \in H^2(\text{Sk}_{\mathcal{C}}; \Delta),$$

the degree-two cohomology of the handoff category’s skeleton with coefficients in the reply (direction) functor Δ [13]. The cohomology itself is classical (Baues–Wirsching); the delta over Ahman–Uustalu is the identification of the interleaving composite with $\mathcal{C} \bowtie \mathcal{D}$ and hence of its obstruction with this class. The class is a categorical holonomy: each partial handoff completes freely, but the completions fail to glue into a globally serialized system by a monodromy around the dispatch/return loop [17].

11.4 The obstruction is a degree-two class, and it is machine-checked

We now specialize to the minimal faithful model of the caller/callee pattern and compute the class. Everything in this section is [PEER-REVIEWED], and its finite core is machine-checked in Lean 4.

The parametrized supervisor–worker category

Fix three roles: **S** (supervisor / caller), **W** (worker / callee running), **R** (a return point). The supervisor carries a one-bit *turn token*; toggling it is an involution τ with $\tau^2 = \text{id}$, so $\text{End}(\mathbf{S}) = \{\text{id}_{\mathbf{S}}, \tau\} \cong \mathbb{Z}/2$. The generating handoffs are

$$p, p\tau: \mathbf{S} \rightarrow \mathbf{W} \quad (\text{dispatch in turn-state 0 resp. 1}), \quad s, s_2: \mathbf{W} \rightarrow \mathbf{R} \quad (\text{two worker outcomes}),$$

with composites $q, q\tau: \mathbf{S} \rightarrow \mathbf{R}$ and right $\mathbb{Z}/2$ -action $p \cdot \tau = p\tau$, $q \cdot \tau = q\tau$. The prescribed right factor is the supervisor-internal (token) part $\mathcal{D} = \{\text{id}_{\mathbf{S}}, \tau, \text{id}_{\mathbf{W}}, \text{id}_{\mathbf{R}}\}$, a wide subcategory whose only nontrivial vertex group is $G_{\mathbf{S}} \cong \mathbb{Z}/2$; the left factor $\mathcal{C}$ — the genuine dispatch/return part — is what a Zappa–Szép product would have to supply.

Definition 11.7 (the family $\mathcal{K}_{\varepsilon}$). For $\varepsilon \in \mathbb{Z}/2$ let $\mathcal{K}_{\varepsilon}$ be the category above with the single length-two composite fixed by

$$s \circ p = q, \quad s_2 \circ p = q \cdot \tau^{\varepsilon},$$

where s is “the worker returns normally” and s_2 is “the worker re-enters” — calls back into the supervisor before returning. Thus $\varepsilon = [s_2 \circ p = q\tau]$ records whether the re-entrant outcome *mutates* the token ($\varepsilon = 1$, unprotected re-entry) or *fixes* it ($\varepsilon = 0$, a state-protected re-entry / a lock).

Each $\mathcal{K}_\varepsilon$ is a genuine category with $\mathcal{D}$ wide, and the reading is exact: re-entrancy — the callee mutating shared caller state during a pending call — is precisely the s_2 -branch dragging the token when $\varepsilon = 1$ [8].

The finite $\mathbb{F}_2$ complex

The categorical obstruction of [13] is, for $\mathcal{K}_\varepsilon$, computed by an explicit orbit/cochain calculation in the analytic source [8]. We import its output; the reduction *to* the complex is a cited pen-and-paper input, isolated here as three separable facts (each a future formalisation target).

Proposition 11.8 (the reduction; [8]). *Fix $\varepsilon \in \mathbb{Z}/2$. Then:*

- (i) $\mathcal{K}_\varepsilon$ satisfies the freeness condition (L) (each hom-presheaf is a free right Δ -set) and the vertex-group hypothesis (Δ a disjoint union of abelian vertex groups with trivial left action);
- (ii) the skeleton orbit category $\text{Sk}_\mathcal{C}$ is the branch $\mathbf{S} \rightarrow \mathbf{W} \rightrightarrows \mathbf{R}$, with both middle arrows $[s], [s_2]$ composing after $[p]$ to the single arrow $[q]$, independent of ε ;
- (iii) the normalized (Baues–Wirsching) cochain complex is $C^1 \cong \mathbb{F}_2^2$, $C^2 \cong \mathbb{F}_2^2$, $C^3 = 0$, with coboundary $\delta^1 h = (h[p] - h[q], h[p] - h[q])$ and defect 2-cocycle of the natural transversal $T = \{p, q, s, s_2\}$ equal to $\omega_T(\varepsilon) = (0, \varepsilon)$.

Everything after Proposition 11.8 is finite linear algebra over $\mathbb{F}_2$, and it is this part that is formalised. Since $C^3 = 0$ we have $Z^2 = C^2$. The coboundary image is the diagonal,

$$B^2 = \text{im } \delta^1 = \{(t, t) : t \in \mathbb{F}_2\} = \langle (1, 1) \rangle,$$

so $H^2 = C^2/B^2 \cong \mathbb{F}_2$, realised by the *class map*

$$\varphi: \mathbb{F}_2^2 \rightarrow \mathbb{F}_2, \quad \varphi(a, b) = b - a = a \oplus b,$$

whose kernel is exactly B^2 . The defect cocycle is $\omega_\varepsilon := \omega_T(\varepsilon) = (0, \varepsilon)$, so $\varphi(\omega_\varepsilon) = \varepsilon$: trivial for $\varepsilon = 0$ and the nonzero generator for $\varepsilon = 1$. Chaining with the obstruction theorem gives the dichotomy

$$\mathcal{C} \bowtie \mathcal{D} \text{ exists} \iff [\omega_\varepsilon] = 0 \iff \varepsilon = 0.$$

Remark 11.9 (reading the class). The two coordinates of ω are the two worker outcomes; the coboundary diagonal is the freedom to relabel which token-state is “canonical.” The class is nonzero precisely because the two outcomes disagree on that relabelling by one token-flip — the re-entrant branch’s side effect. This is the categorical form of “a flat bundle is trivial iff its holonomy vanishes”: each partial handoff completes freely, but the completions fail to glue into a global serialized agent by a single $\mathbb{Z}/2$ monodromy around the dispatch/return loop.

What is machine-checked

The finite $\mathbb{F}_2$ computation is transcribed into Lean 4 and machine-checked. The development is sorry-free and uses no external library — in particular no Mathlib — because the entire computation is finite, so the substantive facts close by `decide` or case exhaustion. The model takes $\mathbb{F}_2 = \text{Bool}$ with `xor` as addition (= subtraction in characteristic two), and defines C^2 , δ^1 , B^2 , ω_T , φ as `C2`, `d1`, `InB2`, `omega`, `phi`.

Theorem 11.10 (the certified statement; [8]). *In `Containers/Reentrancy.lean`, sorry-free and without external libraries, the following are machine-checked:*

- *phi_ker_eq_inB2* and *phi_surjective*, together giving the isomorphism $H^2 = C^2/B^2 \cong \mathbb{F}_2$ realised by φ ;
- *phi_omega*: $\varphi(\omega_\varepsilon) = \varepsilon$ — the class form of $[\omega] = \varepsilon$;
- *omega_inB2_iff_zero*: $\omega_\varepsilon \in B^2 \iff \varepsilon = 0$ — the membership form, i.e. the joint agent $C \bowtie D$ exists iff the worker fixes the token.

Consequently, on the finite complex of Proposition 11.8, the obstruction class equals the token-mutation bit, and unprotected re-entrancy ($\varepsilon = 1$) is the nonzero generator of $H^2 \cong \mathbb{Z}/2$.

The proofs are as short as they read: *phi_omega* is a two-case split closing by *rfl*, *omega_inB2_iff_zero* rewrites by $\ker \varphi = B^2$ then by $\varphi(\omega_\varepsilon) = \varepsilon$, and *phi_ker_eq_inB2* is a four-case exhaustion over $(a, b) \in \mathbb{F}_2^2$. The brevity is the point: once the categorical obstruction is reduced to this complex, $[\omega] = \varepsilon$ is a finite fact a proof assistant settles with no room for hand-waving.

The verification boundary, drawn honestly

The Lean development certifies the *finite $\mathbb{F}_2$ class computation*: given the complex of Proposition 11.8, it proves $H^2 \cong \mathbb{F}_2$, $\varphi(\omega_\varepsilon) = \varepsilon$, and the membership dichotomy. It does *not* certify the *reduction* of the categorical obstruction to that complex — that $\mathcal{K}_\varepsilon$ satisfies (L) and the vertex-group hypothesis, that $\text{Sk}_\mathcal{C}$ is the stated branch, and that the transversal defect is $(0, \varepsilon)$ (all of Proposition 11.8) — which is established by hand in [8] and imported here as input. In one line: *the last mile — from cochain complex to $[\omega] = \varepsilon$ — is machine-checked; the first mile — from category to cochain complex — is a cited pen-and-paper theorem.*

11.5 The EVM instantiation: the DAO and the token bit

We close by grounding the model in the EVM, at grade [COMPUTED]. The claim is that $\mathcal{K}_\varepsilon$ is the DAO pattern stripped to its skeleton, and that the design bit ε is a familiar engineering quantity.

Read the three roles concretely. The supervisor **S** is the vulnerable contract holding the shared resource — in the DAO, the balance ledger. The dispatch p is the external call: `msg.sender.call{value: ...}("")`, made *before* the ledger is updated. The worker **W** is the callee, an attacker-controlled fallback function. Its two outcomes are the crux:

- s — the worker returns normally, control comes back to **S**, and the ledger is decremented as intended;
- s_2 — the worker *re-enters*: its fallback calls back into **S**’s withdraw entry point while the first call is still open and the ledger still reads the pre-withdrawal balance.

The token is the shared state whose consistency across the pending call is the whole question. The bit ε is whether the re-entrant branch s_2 mutates that state relative to the normal branch s : for the DAO, $\varepsilon = 1$, because the second withdrawal succeeds against stale state and the composite $s_2 \circ p$ lands in the token-flipped composite $q\tau$ rather than the serialized q . By Theorem 11.10, $[\omega] = \varepsilon = 1 \neq 0$: the two contract invocations admit *no* Zappa–Szép weld into a single well-defined serialized agent, and the “composite contract” the attacker exercises is a genuinely different — wrongly interleaved — system.

Remark 11.11 (the re-entrancy guard is a coboundary). The standard mitigations are exactly the move $\varepsilon : 1 \mapsto 0$. The checks-effects-interactions pattern updates the ledger *before* the external call, so the s_2 branch reads the already-decremented balance and $s_2 \circ p = q$: the token is fixed, $\varepsilon = 0$. A mutex re-entrancy guard (**nonReentrant**) forbids the s_2 edge outright by refusing the second entry while the first is open — in the source’s words, “the re-entrancy guard is precisely the refusal to admit such a distributive law” [8]. Either way the defect cocycle moves from $(0, 1)$ to $(0, 0)$, into the coboundary $B^2 = \langle (1, 1) \rangle$, and the weld exists. The guard does not *detect* the obstruction; it *annihilates* the class.

Remark 11.12 (scope and honesty). Two limits are worth stating. First, $\mathcal{K}_\varepsilon$ is the minimal faithful model: one supervisor, one worker, a one-bit token, a single length-two composite. Larger handoff topologies — several mutually re-entrant contracts, deeper call stacks — carry a class in a genuinely larger H^2 , and reducing them to a scalar is not claimed here. Second, the EVM reading of this section is a [COMPUTED] translation: it asserts that the DAO’s call graph instantiates $\mathcal{K}_1$ and that guards realize $\varepsilon = 0$, but it does not formalise the EVM semantics. What is [PEER-REVIEWED] and machine-checked is the mathematical core: on $\mathcal{K}_\varepsilon$, unprotected re-entrancy is the nonzero generator of $H^2 \cong \mathbb{Z}/2$, and the joint contract exists iff the callee fixes the shared token.

The through-line of the dossier’s grant narrative appears here in its sharpest form: compositional correctness has direct economic consequences, the correctness criterion is a cohomology class, and on the canonical failure that class is not merely definable but *finite and machine-checked*. Re-entrancy is the smallest place where two systems cannot be fused, and $[\omega] = \varepsilon$ is why.

Chapter 12

Tax computation (Catala)

This chapter is the deliberate exception in the Applications pillar: it carries *no formal artifact*. There is no proof, no Lean development, and no running integration with the Catala tax-DSL — only a categorical reading of how a statutory tax engine is shaped, offered as an illustrative target and a statement of intent. Every structural claim below is *speculative*; nothing here is asserted as a result, and the reader should treat the whole chapter as a conjectural design sketch aligned with Honest-Ledger item 2.

Remark 12.1 (No artifact — read this first). Unlike the other Applications chapters, this one is not backed by a computed witness, a paper proof, or machine-checked code. The correspondences it draws between a Catala tax pipeline and the container machinery of Part I are *suggestive analogies*, graded *speculative* throughout. They are included because the picture is clean and points at concrete future work, not because any of it has been established. Where the text uses the vocabulary of Part I (**Cont**, **DCont**, **Cat**, $\triangleleft$, H^2), it does so by analogy only, without claiming that the identifications have been checked.

12.1 The categorical picture (speculative)

A regulator-scrutinised tax engine exhibits three structures that *look* directed-container-shaped. We record them as targets.

An ontology DAG of definitions. The statutory corpus behind an engine is a typed, heterogeneous, acyclic graph of principles, decisions, requirements, generators and derived artefacts. Read as a container (S, P) , the shape S is the whole typed DAG and the positions P_s are its nodes; *restriction* zooms to the sub-DAG reachable from a generator node, and an audit that walks every node against its local sub-DAG has the *shape* of a comonadic extend. The *conjectural* payoff is a compositionality law: that auditing a sub-DAG and embedding the verdict agrees with auditing the whole and restricting. We do not claim this law holds — only that, if the audit is a genuine comonad coextension, it *would*, and that a violation *would* signal a bug. This is a target, grade *speculative*.

Scope composition of rules. A Catala pipeline composes sub-scopes in statutory dependency order: `output of <SubScope> with {...}` followed by field projection. Under the analogy, a scope is a shape, its typed fields are positions, sub-scope invocation is restriction, and the flow of one scope's outputs into the next scope's inputs is composition — the substitution grammar $\triangleleft$ of §3.3, read informally. Catala additionally carries *exception priority*: when several rules claim one

output field, its compiler resolves the conflict by a declared priority order. In container terms this would be a *partial order on positions* refining a directed container. We flag it as *worth formalising*; we have not formalised it, and whether a partially-ordered directed container captures Catala’s default-logic semantics is *conjectural*.

Comonadic event-sourced projections. A taxable event stream, with its deterministic replay discipline (“the same events in the same order produce the same state”), reads as the most explicitly comonadic of the three: the stream is the shape, each event a position, **query** an extract, and **apply** an extend-step. The determinism requirement has the form of a comonad law, $\text{extract} \circ \text{extend } f = f$. We state this as an *observation about form*, not a verified equation: we have not exhibited the comonad, checked its laws, or connected it to an implementation.

12.2 Where the correctness equation would enter (statement of intent)

If the three structures above were made precise, the dossier’s organising equation — correctness $\iff (L) \wedge [\omega] = 0 \in H^2$ — would have a natural, and entirely *conjectural*, reading here. The local condition (L) would say each rule module is a directed container (a small category of sub-scope restrictions and their composites); the global condition $[\omega] = 0$ would obstruct *welding* independently-authored rule modules — across statutory boundaries, tax years, or jurisdictions — into one coherent engine, with a nonzero class flagging a genuine conflict between overlapping rule sets that no priority order can silently reconcile. This is a hypothesis about *where* the machinery would attach, offered to motivate future work.

Conjecture 12.2 (Illustrative only; *speculative*). *There is a directed-container model of a Catala tax pipeline in which (i) sub-scope composition is the substitution product $\triangleleft$, (ii) exception priority is a partial order on positions, and (iii) the obstruction to welding two independently-authored rule modules is a class $[\omega] \in H^2$ that vanishes iff the modules admit a coherent common refinement.*

We state Conjecture 12.2 to fix the target precisely, not because we have evidence beyond the structural analogy of §12.1. It is unproven, unformalised, and untested against a real Catala corpus. The honest deliverable of this chapter is exactly that target, plus the discipline of labelling it as such.

Remark 12.3 (The intended next step). The minimal artifact that would move this chapter off *speculative* is a small worked Catala example — two overlapping rule scopes — with the directed container written out explicitly and the priority partial order checked against Catala’s compiled default logic. Until such an artifact exists, this chapter remains a statement of intent. It is retained in the dossier for the same reason the Honest Ledger is near the front: a programme that claims a unifying equation must be equally clear about the domains where the equation is only aspirational.

Chapter 13

Agent orchestration: the harness is a worker

The thesis of this chapter is a single reduction: the correctness of an agent orchestration is a *comonad distributive law*, and the obstruction to its existence is the very same cohomology class $[\omega] \in H^2$ that governs re-entrancy in the blockchain setting of Chapter 11. We arrive there in four moves. A stateful agent — a *worker* — is a container morphism out of a domain tensored with one fixed state object; the harness that runs a tool-calling protocol is not a meta-object but that protocol’s own comonad, so a protocol *is* a directed container; workers stratify into a small type hierarchy (pure morphism, store-comonad worker, lens); and when effect and coeffect must be threaded through one another the composition rule is a *reverse entwining*, valid exactly when the effect monad does not branch. Throughout, the honest verdict is the same as in the source notes: for a single agent the structure is a rename, and it earns its keep only under composition, where it converts a scattered, implicit correctness condition into one named object checked in one place. No result in this chapter is new; every fact is proved, and where noted machine-checked, in the cited notes, and the contribution is the accounting.

13.1 The harness is a worker

Recall the container vocabulary of Part I. A *container* $S \triangleright P$ is a shape set S with a position family $P : S \rightarrow \mathbf{Set}$, with extension the polynomial functor $\llbracket S \triangleright P \rrbracket(X) = \sum_{s:S} (P s \rightarrow X)$; a morphism $(S \triangleright P) \rightarrow (T \triangleright Q)$ is a forward map $f : S \rightarrow T$ on shapes together with a backward, contravariant family $\prod_s (Q(f s) \rightarrow P s)$ on positions. Two composition operators act on containers: the Dirichlet tensor $\otimes$, which multiplies shapes and multiplies positions (extension the Day convolution $\circledast$ of the product), and the substitution $\triangleleft$, whose extension is functor composition [29, 30].

Definition 13.1 (State object and worker). For a set S the *state object* is the container $\Delta S := S \triangleright (\lambda_. S)$, whose extension is the *store comonad*

$$\llbracket \Delta S \rrbracket(X) = \sum_{s:S} X^S = S \times X^S = \mathbf{Store}_S X,$$

a current state paired with a reader $X^S = (S \rightarrow X)$; as a directed container it is the codiscrete category on S . A *worker of state S from X to Y* is a container morphism $w : \Delta S \otimes X \rightarrow Y$.

The content of the reformulation is that “stateful agent” needs no new machinery: state is one fixed tensor factor, and the container-morphism discipline forces the rest.

Theorem 13.2 (Unfolding a worker; *proved*, arithmetic Lean-checked). *A worker $w : \Delta S \otimes X \rightarrow Y$ with $X = X_0 \triangleright X_1$, $Y = Y_0 \triangleright Y_1$ is exactly a pair*

$$\text{fwd} : S \times X_0 \rightarrow Y_0, \quad \text{bwd} : \prod_{(s, x_0)} (Y_1(\text{fwd}(s, x_0)) \rightarrow S \times X_1 x_0),$$

and, by the universal property of the product, bwd splits into a state-writeback $\text{bwd}_S : \prod Y_1(\dots) \rightarrow S$ and an ordinary back-map $\text{bwd}_X : \prod Y_1(\dots) \rightarrow X_1 x_0$. The forward leg reads (state, input) and answers; the backward leg writes back a new state and answers the input.

This is the whole of “a stateful step,” typed: neither leg is postulated, both are consequences of the tensor’s position-product $S \times X_1 x_0$. The verdict for a single worker is deliberately deflationary.

Proposition 13.3 (One worker is threaded state, re-spelled; *proved*). *On the lens fragment $X = y$, $Y = A \triangleright (\lambda_- . B)$, a worker $\Delta S \rightarrow A \triangleright (\lambda_- . B)$ is exactly a pair*

$$\text{get} : S \rightarrow A, \quad \text{put} : S \times B \rightarrow S,$$

the ordinary “read the state to answer; write the state back” of hand-threaded code. The store’s counit is get and its comultiplication is “duplicate the register” — operations hand-threaded code already performs.

Remark 13.4 (The honest single-worker ledger). For a system of one stateful agent the store comonad is a rename: more category theory, no more safety. We say so plainly, because the value must be sought under composition, and it is not sought in vain — agentic systems are systems of *many* interacting agents.

Two ways to stack, only one free. Tensoring two state objects multiplies their states on the nose, whereas substituting nests a store inside a store:

$$\Delta S \otimes \Delta T = \Delta(S \times T), \quad \Delta S \triangleleft \Delta T = (S \times T^S) \triangleright (\dots) \neq \Delta(S \times T), \quad (13.1)$$

the latter recording not a pair of states but a whole branch map $S \rightarrow T$; at $|S| = |T| = 2$ these are $4 \cdot y^4$ against $8 \cdot y^4$, an explicit retract $r \circ \sigma = \text{id}$ (not an equality, not a fibre), *proved* and Lean-checked (`WorkersRetract.lean`). The tensor (13.1) is the composite of two *non-interacting* workers — independent registers, “ask both, answer both” — and it is free precisely because it assumes non-interference.

The interacting composite is neither. The workers an orchestration actually composes *inter-act*: a supervisor over a worker that can call back into the supervisor (re-entrancy), or two workers reading and writing a shared ledger. There the registers are not independent, so the composite is neither the free tensor nor a bare nesting; it is a *Zappa–Szép weld* $\Delta S \bowtie \Delta T$, in which each store acts on the other, and a weld is a *comonad distributive law* λ recording how the two contexts interleave: which side reads first, whose write the other sees. Comonads, unlike the data they carry, do not compose for free [27]; without a valid λ there is no composite worker at all.

Theorem 13.5 (Existence of the weld is a cohomology class; *distributive-law* $\leftrightarrow$ *weld half imported* (Ahman–Uustalu); $[\omega]$ *half proved in the normal/abelian regime*). *Present two interacting stateful workers as directed containers C, D whose interaction is carried by a normal vertex-bundle $1 \rightarrow \mathcal{D} \rightarrow \pi \rightarrow \Gamma \rightarrow 1$ with abelian vertex fibre $\mathcal{D}$, and fix the mutual actions of the two contexts. Suppose the freeness condition (L) holds of those actions. Then the weld with those actions — a comonad*

distributive law, equivalently a Zappa–Szép weld $C \bowtie D$ — exists if and only if the obstruction class $[\omega] \in H^2(\text{Sk}_{\mathcal{C}}; \mathcal{D})$ vanishes, $(G) \Leftrightarrow [\omega] = 0$. The equivalence “comonad distributive law $\leftrightarrow$ Zappa–Szép weld” is imported from Ahman–Uustalu [27]; the identification of the weld’s existence with $[\omega] = 0$ is proved in the groupoid Zappa–Szép notes [8, 17] for exactly this normal, abelian vertex-bundle regime (non-normal or non-abelian matched pairs are not covered). At the smallest re-entrant supervisor–worker model the class lives in $H^2 \cong \mathbb{Z}/2$ and equals a single bit

$$[\omega] = \varepsilon \in \mathbb{Z}/2, \quad \varepsilon = \text{“does the worker mutate shared state the supervisor depends on?”}, \quad (13.2)$$

so the weld exists iff $\varepsilon = 0$.

Remark 13.6 (The two-store case is open — *flagged*). Whether the weld $\Delta S \bowtie \Delta T$ of *two store comonads* falls inside the normal, abelian vertex-bundle regime of Theorem 13.5 — so that the criterion $(G) \Leftrightarrow [\omega] = 0$ applies to it verbatim — is *open*. It is the principal open item of the tool-calling / orchestration programme; the theorem above is stated for a fixed normal/abelian interaction datum and does not, on its own, settle the general two-store weld.

This is the identical obstruction that classifies re-entrancy in Chapter 11: a *wrong* interleaving — a λ violating the coherence axioms — is precisely the state-divergence / re-entrancy defect, one agent reading context another has already invalidated, and $[\omega] \neq 0$ is a globally inconsistent interleaving assembled one locally reasonable call site at a time.

Remark 13.7 (Where the machine check stops — *honest boundary*). The class computation (13.2) is machine-checked in Lean 4, **sorry-free** and **Mathlib-free** (**Reentrancy.lean**): the file certifies that the cocycle $\omega(\varepsilon)$ is a coboundary exactly when $\varepsilon = 0$. What Lean certifies is the finite $\mathbb{F}_2$ class computation; the *reduction* of the categorical obstruction to that cochain complex is proved pen-and-paper and is *not* formalised.

Remark 13.8 (What threading does, and does not, do). Hand-threading two shared states does not escape the distributive law: every call site that reads one register and writes another silently commits to an implicit λ , and nothing checks the per-site choices are globally consistent. The store-comonad worker does not make the composite exist automatically — no formalism can, because sometimes it genuinely does not — but it *concentrates* the decision: the interleaving is one named object λ , its validity one class $[\omega]$, both inspectable before the system runs, in one place, rather than reconstructed from scattered call sites after it breaks.

13.2 Tool-calling protocols are directed containers

The preceding section modelled the agent’s *state*. This section identifies the object the agent talks *through*. An agent does its work by calling tools: it emits a typed request and the harness returns a typed response, folded back into context before the next step. That discipline — which requests are well-formed, which responses each may receive, how responses accumulate — is a tool-calling protocol, and it is a container.

Definition 13.9 (Tool-calling protocol). A *tool-calling protocol* is a container $S \triangleright P$ in which S is the set of well-formed request schemas an agent may emit and, for each $s : S$, $P s$ is the set of admissible responses the harness may return to s . One *step* is the endofunctor $\llbracket S \triangleright P \rrbracket(X) = \sum_{s:S} (P s \rightarrow X)$: choose a request, continue as a function of the response.

Up to the reading of S and P , the step endofunctor is not new: it is the session-coalgebra functor $F(X) = \coprod_a X^{B_a}$ of Keizer–Basold–Pérez, itself traced to Gambino–Kock. What has not been done is to carry the identification past the single step, and the missing ingredient is the harness.

Proposition 13.10 (The harness is the protocol’s comonad; *identification, proved*). *A tool-calling protocol whose harness threads context coherently — an empty initial context, a way to read the current context to answer without a further request, and an associative extension of a context by a step — is a directed container: the counit is “answer from the current context,” the comultiplication is “expose the context of each continuation,” and the category axioms are exactly the coherence of context threading. By the seed equivalence chain*

$$\mathbf{DCont} \cong \mathbf{Cat} \cong \mathbf{Cat}^\sharp \subset \mathbf{Poly} \quad (13.3)$$

(directed container $\cong$ small category $\cong$ polynomial comonad) [26, 25, 30] *it is therefore a small category and a polynomial comonad.*

This is Definition of a directed container read in the protocol’s vocabulary, not a new theorem; but it is the move the session-coalgebra literature stops one refinement short of, and once the protocol is a comonad three structures are immediate rather than invented.

Corollary 13.11 (The interaction trace is the cofree comonad; *proved*). *The interaction trace of a protocol $S \triangleright P$ — the tree of all runs, each a sequence of request/response pairs — is the cofree comonad on the tool functor, with labels the agent’s observable outputs. A single conversation is one root-to-node branch; the counit reads the current node, the comultiplication re-roots the trace at each node.*

Corollary 13.12 (Protocols compose in two ways; *proved*). *Two protocols compose by substitution $C \triangleleft D$ (run D at each response position of C — a supervisor delegating to a sub-protocol) and by Dirichlet tensor $C \otimes D$ (offer both protocols’ requests independently — two non-interacting tools). The interesting composite, of two protocols that share state, is neither, and is the Zappa–Szép weld of §13.1, obstructed by $[\omega]$.*

Remark 13.13 (Running the agent is an interaction law). The comonad reading also locates the agent: its tool-issuing behaviour is a monad of effects, and executing it against the harness is the pairing of that monad with the protocol comonad by an *interaction law*, the categorical form of “running” the agent. We do not develop the agent-monad here; the point is that “run” becomes a standard construction rather than an operational primitive.

The engineering payoff is that the multi-agent correctness question is not a testing problem but a computable obstruction: for an orchestration of many interacting agents, that obstruction (Theorem 13.5) is the compositional-correctness guarantee worth naming.

13.3 The workers type hierarchy

Between the memoryless agent and the fully interacting weld sits a short, exact hierarchy of worker types, each a special case of the last, and each with three coincident readings.

Definition 13.14 (The hierarchy).

1. A *pure agent* is a bare container morphism $X \rightarrow Y$ (no state factor): control down, data up, memoryless.
2. A *worker* of state S is a morphism $\Delta S \otimes X \rightarrow Y$ (Definition 13.1); equivalently a coKleisli arrow of the store comonad $p \mapsto \Delta S \otimes p$, so worker composition is coKleisli composition.

3. A *lens* is the monomial fragment $X = y$, $Y = A \triangleright (\lambda_.B)$ of a worker: a get/put pair (Proposition 13.3).

Proposition 13.15 (One picture, three names; *proved*). *A worker $\Delta S \otimes X \rightarrow Y$ has three coincident presentations: downstairs (containers) a forward/backward pair with a state-writeback; one level up (functors) a coKleisli arrow of the store comonad $S \times (-)^S$; and sideways (optics) a lens with an input interface bolted on — the ΔS factor contributing the bidirectional get/put, the X factor an ordinary delegate/amalgamate. The tensor is what makes the three coincide.*

The hierarchy is not merely a taxonomy: it is *graded*, and the grade is the content.

Proposition 13.16 (Workers form a state-graded category; *proved*). *A worker of state S from X to Y is an element of $\text{Hom}_{\mathbf{Cont}}(\Delta S \otimes X, Y)$, and workers compose with the state multiplying: a state- S worker followed by a state- T worker is a state- $(S \times T)$ worker, because $\Delta S \otimes \Delta T = \Delta(S \times T)$ (13.1). The grade S accumulates by product; this composition is unobstructed (the grade simply grows), in contrast to the weld, where interaction reinstates the $[\omega]$ obstruction.*

Remark 13.17 (Why the grading is honest about interaction). The state-graded category composes *non-interacting* workers, and there the multiplication of grades is the whole story — no obstruction, because the tensor assumes independence. The moment two graded workers must *share* a register, one leaves the graded category for the weld of §13.1, and the branch-map data that $\triangleleft$ records but $\otimes$ collapses (13.1) is exactly where the interleaving decision — and its class $[\omega]$ — lives. The hierarchy thus separates cleanly into an obstruction-free axis (grade accumulation under $\otimes$) and the obstructed axis (interaction under $\bowtie$).

Example 13.18 (A running balance). With state $S = \mathbb{Z}$ and interface “deposit or query,” the worker $\text{fwd}(s, \text{dep } n) = \text{fwd}(s, \text{bal}) = s$ (report the pre-state) with writeback $\text{bwd}_S(s, \text{dep } n) = s + n$, $\text{bwd}_S(s, \text{bal}) = s$ is exactly a get/put lens. Its type guaranteed, before any run, that precisely a read-and-answer and a state-writeback had to be supplied and nothing else could sneak in — “correct by construction” for a single stateful agent, with the caveats of Remark 13.4 intact.

13.4 Effects and coeffects: the reverse-entwining arrow

The workers of §13.3 carry *state*. A richer orchestration threads two computational modes at once: an *effect* (the agent fails, chooses, logs) and a *coeffect* (the agent reads ambient context — an environment, a history, a budget). On containers both arise from a *single* **Set**-monad M , fed along the two axes, and the variance of positions decides the outcome.

Definition 13.19 (The two feeds of one monad). For a **Set**-monad M (with a $\prod$ -cointerpretation support lifting $P^\star$), feeding M along shapes gives the Ahman–Bauer *effect monad* $T_M(S \triangleright P) = MS \triangleright P^\star$ — shapes are covariant, so T_M is a **monad** on **Cont**; feeding M along positions gives the *coeffect comonad* $G_M(S \triangleright P) = S \triangleright (M \circ P)$ — positions are contravariant, so G_M is a **comonad**. There is no third axis and no choice in the outcome.

Theorem 13.20 (The two feeds always entwine; *proved, one mechanical gap flagged*). *For every such M there is a canonical mixed distributive law $\lambda : T_M G_M \Rightarrow G_M T_M$ — the oplax product-comparison $M(\prod_b Z_b) \rightarrow \prod_b M(Z_b)$ of M , read on positions — satisfying the four entwining axioms, so G_M lifts to a comonad on T_M -algebras and T_M to a monad on G_M -coalgebras (a Turi–Plotkin λ -bialgebra). This holds unconditionally, nondeterminism ($\mathcal{P}_f$) and lists included.*

The one axiom whose fully symbolic verification depends on the weak Mendler datum (mult- T) is complete in mechanism but not written out as an index-chase over an arbitrary lifting; it is machine-verified across three containers including the branching case $M = \mathcal{P}_f$. We flag this rather than blur it.

The reverse orientation is another story, and it is the one an orchestration needs: to *sequentially compose* programs that read context and then produce effects, one must commute the effect out through the coeffect.

Definition 13.21 (Effect-coeffect arrow). An *effect-coeffect arrow* $p \rightsquigarrow q$ is a container morphism $G_M p \rightarrow T_M q$: it reads its input through the coeffect G_M and delivers its output through the effect T_M . Composing two such arrows requires a compositor $\kappa : G_M T_M \Rightarrow T_M G_M$ — the *reverse* orientation of the always-valid λ of Theorem 13.20, on positions the *lax* product-comparison $\prod_b M(Z_b) \rightarrow M(\prod_b Z_b)$.

Theorem 13.22 (The arrow exists iff the reverse entwining does iff M is non-branching; *proved, Maybe Lean-checked*). For a $\prod$ -cointerpretation Set-monad M the following are equivalent:

1. the effect-coeffect arrows $G_M p \rightarrow T_M q$ form a category (indeed a Hughes arrow / Freyd category over $(\mathbf{Cont}, \times)$);
2. there is a natural $\kappa : G_M T_M \Rightarrow T_M G_M$ satisfying the four mixed-law axioms;
3. M is non-branching (every operation has arity ≤ 1).

The unit laws hold for every M ; the sole obstructed axiom is associativity (mult- T , a “union of products $\neq$ product of unions” discrepancy when μ^M merges leaves), and a second, independent obstruction — T_M has no natural strength for $\times$ when M branches, by leaf-symmetry — blocks the first operator likewise. Branching disables the arrow twice over, both failures equivalent to arity ≤ 1 .

Theorem 13.23 (Classification of the positive class; *proved*). A cartesian $\prod$ -cointerpretation monad M is non-branching if and only if it is a writer-with-absorbing-exceptions monad

$$MX \cong E + A \times X, \quad (A, \cdot, e) \text{ a monoid, } (E, \odot) \text{ a left } A\text{-set,}$$

with writer unit $x \mapsto (e, x)$ and the log A acting on the exceptions E ; the arrow category exists throughout this class. Non-branching is a genuinely new dividing line, provably independent of commutativity and of affineness.

Remark 13.24 (What is machine-checked, and the honest synthesis). The smallest genuinely effectful arrow category, $M = \mathbf{Maybe}$, is formalised in Lean 4 (`BiKleisliMaybe.lean`): an abstract `MixedDistrib` structure with the four axioms including associativity `E2'`, from which `biKleisli` composition and its associativity law are derived `sorry-free`. The general associativity across the whole writer-with-exceptions class (Theorem 13.23) is proved in coordinates but not yet Lean-certified. The synthesis worth stating for the grant is that the three container-native modes of welding compositional systems carry *three different* obstructions: the Zappa–Szép weld of directed containers is obstructed by a cohomology class $[\omega] \in H^2$ (§13.1, Chapter 11); the state-graded category of workers is *unobstructed* (Proposition 13.16); and the effect-coeffect arrow is obstructed by a Boolean arity condition (Theorem 13.22). They do not unify into a master theorem, and the honest claim is exactly that: *whether* two agents compose is, in each mode, a computable invariant of the parts — but which invariant depends on the axis.

Chapter 14

Security audits as failed distributive laws

Professional smart-contract audits (Trail-of-Bits style) keep surfacing the same shape of bug: two components, each provably correct in isolation, produce a global failure when composed. This chapter reads that recurring finding through the obstruction picture of the blockchain chapter (Chapter 11): a *state-divergence* audit finding is a failed distributive law, and the concrete divergence the auditor exhibits is an $[\omega] \neq 0$ witness in H^2 . It is a modest extension — a reframing with predictive bite, not a new theorem — and is graded *computed* throughout: the only machine-checked claim it leans on is the pre-existing re-entrancy obstruction, referenced not reproved.

14.1 From the re-entrancy weld to the audit desk

Chapter 11 models a contract as a coalgebra for $p(X) = (O \times X)^I$ and identifies re-entrancy with the failure of the weld between two contracts to admit a coherent distributive law $\delta: ST \Rightarrow TS$; the obstruction is a class $[\omega] \in H^2$, and the composite is safe exactly when $[\omega] = \varepsilon$ (the criterion of Chapter 6, machine-checked in the re-entrancy corpus). The observation of this chapter is that the same frame names a *second*, larger family of audit findings that are not temporal re-entrancy at all: *state divergence*, where two representations of one logical value drift apart under composition. Trail of Bits described one such bug in exactly the compositional idiom — “two independently-correct components created a security failure through incorrect interaction” — and that idiom is precisely the laxator-as-obstruction picture.

When two effectful components S and T are composed, the composite $S \circ T$ is a well-behaved structure again only if a distributive law $\delta: ST \Rightarrow TS$ exists and is coherent. Local correctness of S and of T says nothing about whether δ exists. A missing or non-coherent δ is not cosmetic: it is the obstruction, and downstream it is the bug. The audit finding — a concrete input that drives the two representations apart — is a witness that δ does not exist; in the H^2 -graded reading of Chapter 6 it is an $[\omega] \neq 0$ witness.

14.2 State divergence as a missing distributive law

The clean toy case is the Provenance Blockchain finding (Trail of Bits, 2026-08-25). Two modules each hold a notion of token supply: a *Marker* module carries a cached **supply** field, and a *Bank* module carries the live, authoritative supply. Each is correct on its own terms. An authorization check compares the two supplies to gate a privileged action; when both read as 0 — a state the composed system permits — the check degenerates to $0 == 0 \rightarrow \text{true}$ and grants unauthorized mint/withdraw. Two correct components, one trivial bypass under composition.

Model the shared logical value “token supply” as an object L , and give each component a writer/state monad over its own copy:

$$S = (-) \times L_{\text{cache}} \quad (\text{Marker's stale view}), \quad T = (-) \times L_{\text{live}} \quad (\text{Bank's authoritative view}).$$

Composing the two modules forms $S \circ T$ and reads a single supply out of it. The intended coherence — “the two views agree on the one value they both denote” — is exactly a distributive law $\delta: ST \Rightarrow TS$ whose unit square commutes: the value read through the cache equals the value read through the authoritative store. The invariant $L_{\text{cache}} = L_{\text{live}}$ is the statement that δ is well-defined. The bug is that no coherent δ exists: cache and store are mutated on different code paths, so the square fails to commute after certain interleavings, and the two representations diverge. The authorization predicate $\varphi(L_{\text{cache}}, L_{\text{live}})$ was written assuming coherence; under divergence it is evaluated off its intended domain, where $L_{\text{cache}} = L_{\text{live}} = 0$ makes it fire spuriously. The divergence *is* the failed distributive law; the $0 == 0$ bypass is what the missing δ looks like at the call site.

Remark 14.1 (The two readings coincide). What a category theorist should see is a non-existent distributive law between two monads over a shared object L ; what a security engineer should see is their own bug — “two sources of truth for one value, kept in sync by convention rather than by construction.” These are the same statement. The audit’s concrete counterexample input is the $[\omega] \neq 0$ witness of Chapter 6: it certifies that the obstruction class is nonzero by exhibiting an interleaving on which the coherence square fails.

The remedy is likewise the categorical move of Chapter 11: make δ hold *by construction* rather than by assumption. Reading a single state variable mutated only at controlled entry points, instead of two separately maintained representations, forces the two views to coincide on the nose — it literally makes the distributive law hold, rather than hoping an invariant is preserved by convention.

14.3 A taxonomy indexed by which law fails

The value of the frame is that it indexes otherwise-unrelated audit patterns by *which* categorical law breaks, turning ad hoc pattern-matching into a checklist. The source note collects three shapes; we record them as the frame’s predictive content, not as proved results.

1. **State divergence** = failed distributive law between two monads: two representations of one logical value (two modules; two entry points into one component; two channels onto one pot; or a verification channel and a settlement channel across a bridge) admit no coherent δ . This is the best-populated class, spanning public witnesses from a re-entrancy accounting slip to a nine-figure cross-chain bridge exploit — the *same* missing δ , differing only in where the two non-commuting channels sit.
2. **Temporal / cross-callback misalignment** = re-entrancy with an H^2 obstruction: the order-of-effects law of a single lax structure composed across a callback boundary, safe iff $[\omega] = \varepsilon$. This is the case already pinned down formally in Chapter 11; the state-divergence class is its sibling under a different broken law.
3. **Self-composition via iteration** = interchange/associativity failure of an iterate $T \circ T \circ \dots$: a single locally-correct operation composed with itself in a loop, where a per-step check does not survive the fold. This is the security-domain shadow of the self-plethysm phenomenon of the change-of-base chapter — composition survives self-composition only when flatness was present to begin with.

The frame supplies the auditor a hunt list indexed by structure: two sources of truth for one value (look for the distributive law); a callback boundary (look for the H^2 class); a loop over an externally-observable operation (look for the non-flat iterate). For the temporal case the criterion is already sharp and machine-checked ($[\omega] = \varepsilon$); the other two shapes are, at present, structural descriptions awaiting the same treatment.

14.4 Status and honest scope

This chapter is a *stub* extending the blockchain obstruction picture, not a standalone proved result. No new theorem is established here. The one formally verified claim invoked is the pre-existing re-entrancy obstruction of Chapter 11 ($[\omega] = \varepsilon$ in H^2 , machine-checked); the categorical models of state divergence and of the iteration case are structural descriptions, and the exploit names, dates, and figures are taken from public audit sources as recorded in the author’s research notes. The grade is *computed*: a genuine but modest reframing whose bite is unification and prediction — one categorical defect, δ absent, whose blast radius across the public witnesses spans roughly two orders of magnitude of loss — rather than a certificate the chapter itself discharges. The natural next step, and the reason the stub earns its place in the dossier, is to lift the state-divergence and iteration shapes to the same checkable H^2 -graded criterion that Chapter 6 already gives for the temporal case.

Chapter 15

Supply chains

This chapter runs the dossier’s spine in the supply-chain domain and, in doing so, sharpens it. A supply chain, abstracted to a small category, is a directed container (Chapter 4); welding a supplier flow to a consumer flow that share a warehouse is a Zappa–Szép composition $\mathcal{C} \bowtie \mathcal{D}$ (Chapter 5); and an inventory inconsistency between two routes to the same delivered good is a nonzero obstruction class $[\omega] \in H^2(\mathrm{Sk}_{\mathcal{C}}; \mathbb{Z}/n) \cong \mathbb{Z}/n$ (Chapter 6). The headline is a dictionary — the same one Chapters 10 and 11 draw for evolutionary systems and for contracts — but here it comes with an extra digit. Because inventory bookkeeping is a *counter* rather than a bit, the obstruction lands not in $\mathbb{Z}/2$ but in $\mathbb{Z}/n$, and the class does not merely flag a mismatch: it *measures* it, in units. The re-entrancy parity bit of Chapter 11 is the $n = 2$ face of the invariant computed here.

Grade discipline. Nothing in this chapter is new mathematics, and the chapter is scrupulous about saying so. The general theorems it stands on are cited or cross-referenced at their established grades and are never re-graded here: that a small category is a directed container is *lean-verified* (Chapter 4); the pairwise Zappa–Szép criterion is *proved* (Theorem 5.9); the identification of the global-closure condition (G) with the cohomological class $[\omega]$ is *proved* (Theorem 6.1); the $n = 2$ anchor is *proved* (Chapter 11). What is new is the *domain instantiation* — an explicit object-level directed container and an explicit warehouse family in which this machinery computes the inventory-inconsistency invariant — and it is graded *computed* throughout, machine-checked in `scratch/supply_chain_zs.py` over eight (n, ε) instances. The one claim held *open* is object-level fidelity: whether a *real* supply chain literally presents a category. We build a faithful minimal abstraction and apply the theorems to *it*; we never assert that a supply chain *is* the category below (Section 15.5).

15.1 The base chain as a directed container

The first dividend of the equivalence chain is definitional: once a chain is abstracted to a small category, it is automatically a directed container, so the entire structure theory of Part I — the monoidal products, the (co)monoid classifications, the composition operators — applies to it verbatim. We make this concrete on the smallest honest example.

Definition 15.1 (The procurement chain $\mathcal{C}_0$). Let $\mathcal{C}_0$ be the poset category on stages $\mathrm{Src} < \mathrm{Mfg} < \mathrm{Dst}$ (supplier, manufacturing, delivery): objects are the three stages, and there is a unique morphism $i \rightarrow j$ whenever $i \leq j$, read as the *net operation* carrying material from stage i to stage

j . Composition is do-then-do; thinness encodes the modelling assumption that all order-respecting routings from i to j deliver the same material.

By the equivalence chain (Chapter 4; Ahman–Uustalu [26]), $\mathcal{C}_0$ is a directed container $(S, P, o, \downarrow, \oplus)$ under the standard dictionary

$$\begin{aligned} S &= \{\text{Src}, \text{Mfg}, \text{Dst}\} && (\text{shapes} = \text{stages}), \\ P(s) &= \{(s \rightarrow j) : j \geq s\} && (\text{positions} = \text{outgoing operations}), \\ o_s &= \text{id}_s && (\text{root} = \text{stay put}), \\ s \downarrow p &= \text{cod}(p) && (\text{sub-shape} = \text{destination stage}), \\ p \oplus q &= q \circ p && (\text{shift} = \text{do } p \text{ then } q). \end{aligned}$$

Explicitly $P(\text{Src}) = \{\text{id}, \text{Src} \rightarrow \text{Mfg}, \text{Src} \rightarrow \text{Dst}\}$, $P(\text{Mfg}) = \{\text{id}, \text{Mfg} \rightarrow \text{Dst}\}$, $P(\text{Dst}) = \{\text{id}\}$, and, for instance, $(\text{Src} \rightarrow \text{Mfg}) \oplus (\text{Mfg} \rightarrow \text{Dst}) = (\text{Src} \rightarrow \text{Dst})$: procure-then-ship *is* the direct delivery operation.

Proposition 15.2 (Directed-container laws; *computed*). $(S, P, o, \downarrow, \oplus)$ satisfies the five directed-container laws: D1 $s \downarrow o_s = s$; D2 $o_s \oplus p = p$; D3 $p \oplus o_{s \downarrow p} = p$; D4 $s \downarrow (p \oplus q) = (s \downarrow p) \downarrow q$; D5 $(p \oplus q) \oplus r = p \oplus (q \oplus r)$.

Proof. $\mathcal{C}_0$ is a thin poset category, so D1–D3 are the unit laws (the identities are the operations $(s \rightarrow s)$), D4 says the destination of a composite is the destination of its second leg, and D5 is associativity of composition — all automatic in a poset. Verified exhaustively over the composable triples in `supply_chain_zs.py`, part (A). $\square$

This is the object-level content that the category-level orchestration analysis of Chapter 11 left implicit: a supply chain, once abstracted to a category, is literally a $\triangleleft$ -comonoid / directed container. The value of the observation is not the (routine) proof but the transfer — every result of Part I is now a statement about $\mathcal{C}_0$.

15.2 Two flows sharing a warehouse

The composition story lives one level up, where two flows meet. The shared node is a warehouse **Wh** that keeps an internal *lot-cursor*: a cyclic pointer τ of order n into n interchangeable bins (a FIFO position mod n , or a bin index). Physically τ moves nothing — it is pure bookkeeping — but a unit leaving the warehouse carries whatever cursor value was current, and advancing the cursor is an automorphism with $\tau^n = \text{id}$, so $\text{End}(\text{Wh}) \cong \mathbb{Z}/n$.

Definition 15.3 (The warehouse family $\mathcal{W}_{n,\varepsilon}$). For $n \geq 1$ and $\varepsilon \in \mathbb{Z}/n$, let $\mathcal{W}_{n,\varepsilon}$ be the small category with objects **Wh** (warehouse), **Pr** (processing), **De** (delivery), and

$$\begin{aligned} \text{End}(\text{Wh}) &= \{\tau^k : k \in \mathbb{Z}/n\} \cong \mathbb{Z}/n, & \text{End}(\text{Pr}) &= \{\text{id}\}, & \text{End}(\text{De}) &= \{\text{id}\}, \\ \text{Hom}(\text{Wh}, \text{Pr}) &= \{p \tau^k : k \in \mathbb{Z}/n\}, & \text{Hom}(\text{Wh}, \text{De}) &= \{q \tau^k : k \in \mathbb{Z}/n\}, & \text{Hom}(\text{Pr}, \text{De}) &= \{s, s_2\}, \end{aligned}$$

with no arrows into **Wh** except its endomorphisms and no arrows out of **De** except id (so the underlying graph is acyclic: **Wh** $\rightarrow$ **Pr** $\rightarrow$ **De** with a direct **Wh** $\rightarrow$ **De**). Here $p \tau^k := p \circ \tau^k$, all distinct; the two processing operations compose through the middle by

$$s \circ p = q, \quad s_2 \circ p = q \cdot \tau^\varepsilon,$$

extended by right τ -equivariance ($s \circ p \tau^k = q \tau^k$, $s_2 \circ p \tau^k = q \tau^{k+\varepsilon}$).

Reading the data. p dispatches a lot to processing; q is the direct, expected delivery recorded by the warehouse's books; s is the nominal production-and-ship line, and s_2 is an alternative route — rework, a second line, an expedited lane — that delivers the *same good*. The single datum ε records how many lot-positions s_2 advances the provenance cursor relative to the nominal route: $\varepsilon = 0$ means the alternative preserves lot-provenance, $\varepsilon \neq 0$ means it shifts it by ε units. The wide right factor of bookkeeping moves is

$$\mathcal{D} = \{\tau^k : k \in \mathbb{Z}/n\} \cup \{\text{id}_{\text{Pr}}, \text{id}_{\text{De}}\} \subseteq \mathcal{W}_{n,\varepsilon},$$

the warehouse-internal relabelling subcategory — the *second* chain that shares the warehouse with the physical-flow chain.

Lemma 15.4 ($\mathcal{W}_{n,\varepsilon}$ is a category; *computed*). *For every n, ε , $\mathcal{W}_{n,\varepsilon}$ is a well-defined small category.*

Proof. Only two families of composites need checking: the $\mathbb{Z}/n$ -action of τ on **Wh**-sourced arrows (a group action, $\tau^n = \text{id}$), and the two length-two composites $s \circ p, s_2 \circ p : \text{Wh} \rightarrow \text{De}$. All longer composites either factor through the sink **De** or begin with a **Wh**-endomorphism, so associativity reduces to right-equivariance, $(f \circ p) \circ \tau^k = f \circ (p \tau^k)$, which holds by the definition of $f \circ p \tau^k$. The full associativity table is checked for eight (n, ε) pairs in `supply_chain_zs.py`, part (B). $\square$

Lemma 15.5 ((L) and the abelian-vertex hypothesis, uniformly in ε ; *computed*). *Every hom-presheaf $\text{Hom}_{\mathcal{W}_{n,\varepsilon}}(-, b)$ is a free right $\mathcal{D}$ -set, and $(\mathcal{W}_{n,\varepsilon}, \mathcal{D})$ meets the hypothesis under which (G) is cohomological (abelian vertex groups, trivial left action).*

Proof. (L). The right $\mathcal{D}$ -action is precomposition; it is trivial on **Pr**- and **De**-sourced arrows and is the regular $\mathbb{Z}/n$ -action on **Wh**-sourced arrows. Decomposing into orbits, $\text{Hom}(-, \text{Wh}) = \{\tau^k\}$ (one regular $\mathbb{Z}/n$ -orbit), $\text{Hom}(-, \text{Pr}) = \{p \tau^k\} \sqcup \{\text{id}_{\text{Pr}}\}$, and $\text{Hom}(-, \text{De}) = \{q \tau^k\} \sqcup \{s\} \sqcup \{s_2\} \sqcup \{\text{id}_{\text{De}}\}$. Each **Wh**-apex orbit is a free $\mathbb{Z}/n$ -torsor (the τ^k are distinct); each other orbit is a singleton with trivial group. So every hom-presheaf is a coproduct of representables, i.e. free. This uses only distinctness of the $p \tau^k$ and $q \tau^k$ and is therefore independent of ε : moving $s_2 \circ p$ inside the orbit $\{q \tau^k\}$ does not change the orbit decomposition. *Abelian vertices.* $\mathcal{D}$ is a disjoint union of abelian vertex groups $G_{\text{Wh}} \cong \mathbb{Z}/n$, $G_{\text{Pr}} = G_{\text{De}} = 0$; the only nontrivial vertex group sits at **Wh**, a *source* (no generator targets it), so the left action by a target automorphism is trivial. Freeness of every orbit is machine-checked in part (B). $\square$

With (L) and the abelian-vertex hypothesis in hand, the criterion of Chapter 6 applies and the obstruction is the class of an explicit 2-cocycle on the orbit category. We compute both.

Proposition 15.6 (Orbit category, complex, and $H^2 \cong \mathbb{Z}/n$; *computed*). *The orbit category $\text{Sk}_{\mathcal{C}}$ has objects **Wh**, **Pr**, **De** and non-identity generators $[p] : \text{Wh} \rightarrow \text{Pr}$, $[q] : \text{Wh} \rightarrow \text{De}$, $[s], [s_2] : \text{Pr} \rightarrow \text{De}$, with $[s] * [p] = [s_2] * [p] = [q]$ independently of ε (because q and $q \tau^\varepsilon$ lie in one orbit). The coefficient presheaf has $\mathcal{D}(\text{Wh}) = \mathbb{Z}/n$, $\mathcal{D}(\text{Pr}) = \mathcal{D}(\text{De}) = 0$, with all restriction maps zero (no generator targets **Wh**). Hence $C^1 \cong (\mathbb{Z}/n)^2$ (coordinates $h[p], h[q]$), $C^2 \cong (\mathbb{Z}/n)^2$ (coordinates $\omega([s], [p]), \omega([s_2], [p])$), $C^3 = 0$, and*

$$(\delta^1 h)([s], [p]) = (\delta^1 h)([s_2], [p]) = h[p] - h[q],$$

so $B^2 = \{(t, t) : t \in \mathbb{Z}/n\}$ is the diagonal and

$$H^2(\text{Sk}_{\mathcal{C}}; \mathcal{D}) = (\mathbb{Z}/n)^2 / \langle (1, 1) \rangle \cong \mathbb{Z}/n, \quad [(a, b)] \mapsto b - a.$$

Proof. This is the bookkeeping of Chapter 11 with $\mathbb{Z}/n$ in place of $\mathbb{Z}/2$: the generators, the vanishing restriction maps, and the two composable pairs $([s], [p])$, $([s_2], [p])$ are as there, and the Baues–Wirsching coboundary formula (Rosebrugh–Wood [37]) reduces to $h[p] - h[q]$ because the coefficient map along $[p]$ is the zero map $0 \rightarrow \mathbb{Z}/n$. The identification $B^2 = \text{diagonal}$ is machine-checked for all tested n in part (B). $\square$

Theorem 15.7 (The class is the unit-mismatch; *computed*). *With the natural transversal $T = \{p, q, s, s_2\}$, the defect 2-cocycle is $\omega_T = (0, \varepsilon) \in (\mathbb{Z}/n)^2$, so*

$$[\omega(\mathcal{W}_{n,\varepsilon})] = \varepsilon \in H^2(\text{Sk}_{\mathcal{C}}; \mathcal{D}) \cong \mathbb{Z}/n.$$

Proof. Both composable pairs have target orbit $\text{orb}(q)$ with chosen generator q . From $s \circ p = q = q \circ \text{id}_{\mathcal{W}_h}$ we read $\omega_T([s], [p]) = 0$; from $s_2 \circ p = q \cdot \tau^\varepsilon = q \circ \tau^\varepsilon$ we read $\omega_T([s_2], [p]) = \tau^\varepsilon = \varepsilon$. So $\omega_T = (0, \varepsilon)$, and its class under $[(a, b)] \mapsto b - a$ is ε . Both $\omega_T = (0, \varepsilon)$ and $[\omega] = \varepsilon$ are machine-verified in part (B). $\square$

Corollary 15.8 (Inventory-consistency dichotomy; *computed*). *For the warehouse abstraction $\mathcal{W}_{n,\varepsilon}$ with bookkeeping right factor $\mathcal{D}$,*

$$\mathcal{W}_{n,\varepsilon} = \mathcal{C} \bowtie \mathcal{D} \text{ exists} \iff \varepsilon = 0 \iff \text{both routes to De agree on lot-provenance.}$$

When $\varepsilon \neq 0$ the obstruction $[\omega] = \varepsilon$ is a nonzero element of $H^2 \cong \mathbb{Z}/n$: a genuine inventory inconsistency in which the two routes to the same delivered good disagree by ε lot-positions, and no serialisation of the physical flow with the warehouse bookkeeping exists.

Proof. (L) holds (Lemma 15.5), so by the pairwise criterion (Theorem 5.9) the product exists iff (G); the abelian-vertex hypothesis holds, so by Theorem 6.1 (G) holds iff $[\omega] = 0$; and $[\omega] = \varepsilon$ by Theorem 15.7. Chaining the equivalences gives the statement. As an independent, non-cohomological cross-check, part (B) brute-forces the strict factorisation systems and finds $\#\text{SFS} = n$ when $\varepsilon = 0$ (a $\mathbb{Z}/n$ -torsor of ways to choose the canonical lot-labelling) and $\#\text{SFS} = 0$ when $\varepsilon \neq 0$. $\square$

15.3 Why $\mathbb{Z}/n$, and not just a parity bit

The re-entrancy analysis of Chapter 11 needed only $\mathbb{Z}/2$: a turn token is one bit, and safety is a parity. Inventory is different in kind. It is a *counter*, and the obstruction lands in the counter’s group. The class $[\omega] = \varepsilon \in \mathbb{Z}/n$ is therefore not the yes/no verdict “the routes are inconsistent” but the quantity “the routes disagree by ε lot-positions.” The obstruction is graduated, and the grading is the unit count of the discrepancy. The proved parity theorem of Chapter 11 is exactly the $n = 2$ face.

Remark 15.9 (Gauge freedom is the choice of slot zero). The coboundary lattice $B^2 = \langle (1, 1) \rangle$ has a clean operational reading. A coboundary shifts *both* routes’ recorded provenance by the same amount; that is precisely the freedom to declare which physical lot-slot counts as “slot zero.” Relabelling the origin of the cursor cannot create or destroy a real inconsistency, so only the *difference* ε between the two routes survives to the quotient. This is the categorical avatar of a flat $\mathbb{Z}/n$ -bundle: it is trivial iff its holonomy — here the provenance drift around the two-route loop — vanishes, which recovers Remark 6.2’s reading of $[\omega]$ as a holonomy.

15.4 Olog merges: the same machine at $n = 2$

The abstraction is not special to logistics. The same family serves ontology and knowledge-graph merges, where it recovers the merge-conflict phenomenon as the $n = 2$ instance of the same obstruction. An olog in Spivak’s sense — say **Book** $\xrightarrow{\text{has}}$ **Author** $\xrightarrow{\text{has}}$ **Name** — is the free category on its schema graph, hence (Chapter 4) a directed container: shapes are types, positions out of a type are the aspects available from it, $\oplus$ is aspect composition, and the path-equations are the schema’s facts (the directed-container laws are checked in part (C)).

Merging that olog with a second one sharing the type **Author** — for instance **Author** $\xrightarrow{\text{has}}$ **Affiliation** — is the composition of two directed containers over a shared object, a Zappa–Szép problem. Take the shared type **Author** to carry a $\mathbb{Z}/2$ token recording the naming convention (“Last, First” versus “First Last”), with **Pr** the name-record and **De** the rendered display. Then Corollary 15.8 at $n = 2$ reads: the merge succeeds — the product $\mathcal{C} \bowtie \mathcal{D}$ exists — iff the two schemas agree on the convention ($\varepsilon = 0$); a disagreement is the nonzero class $[\omega] = 1$, a *merge conflict*. It is the same H^2 bit as supply-chain inconsistency and agent re-entrancy, computed by the same machine, and it is checked in part (C).

15.5 Scope and honesty

The computed table. Across the eight instances verified in `scratch/supply_chain_zs.py`, the invariant behaves as follows.

regime	$[\omega] \in \mathbb{Z}/n$	outcome	meaning
$\varepsilon = 0$ (any n)	0	composes, $\#\text{SFS} = n$	routes agree on provenance
$\varepsilon \neq 0$ (any n)	ε	obstructed, $\#\text{SFS} = 0$	inventory off by ε units
$n = 2, \varepsilon = 1$	1	obstructed	= orchestration re-entrancy bit (Ch. 11)
olog, $n = 2$	0 / 1	merges / conflicts	schema conventions agree / clash

Each row checks the category axioms, condition (L), the ε -independence of $\text{Sk}_{\mathcal{C}}$, the diagonal B^2 , the cocycle $\omega_T = (0, \varepsilon)$, the class $[\omega] = \varepsilon$, and the $\#\text{SFS}$ cross-check.

Proved versus computed. The general theorems this chapter invokes — $\mathbf{DCont} \simeq \mathbf{Cat}$ (lean-verified), the pairwise Zappa–Szép criterion (proved), and $(\mathbf{G}) \Leftrightarrow [\omega] = 0$ (proved) — are cited at their established grades and are not re-derived here. The domain instantiation — that this abstraction of supply chains and olog merges computes the inventory-inconsistency and merge-conflict class — is *computed*: an explicit by-hand and Python calculation, not a new theorem.

The one open claim. Whether a *real* supply chain genuinely presents a category — operations composing associatively on the nose, the inventory-consistency path-equations holding exactly — is a modelling question, SEED Open Question 4, and it stays *open*. Everything above is a chosen minimal abstraction: the theorems apply to *it*, and the value is that they compute the right qualitative — and, in the $\mathbb{Z}/n$ refinement, quantitative — invariant. We do not write “a supply chain *is* $\mathcal{W}_{n,\varepsilon}$.” Earning object-level fidelity is a separate theorem, not discharged here.

Grant line. Supply-chain composability, ontology-merge consistency, and agent-orchestration re-entrancy are the *same* degree-two obstruction on the handoff category; in the supply-chain instance the class is refined from a bit to a $\mathbb{Z}/n$ -valued unit count of the discrepancy. This is the honest shape

of the Impact story: the applications are not new mathematics but new *instances of one obstruction theorem* — and, occasionally, as here, a sharpening of what that one theorem can measure.

Part III

Formalisation

Chapter 16

Milestones M1–M6: status

This chapter is the authoritative formalisation ledger for the equivalence programme. It records, milestone by milestone, exactly what has been machine-checked in Lean 4, what is proved only on paper or in the literature, and what remains open or conjectural. The single governing rule is honesty against the proof registry (`registry/equivalence-chain.json`): where the roadmap prose and the registry disagree on a grade, the registry wins and the discrepancy is flagged in place. Milestones *M1–M4* (with the converses *M2b*, *M3b*) are *lean-verified*; milestones *M5* and *M6* are *not* formalised and are not even uniformly proved on paper — this is Honest Ledger item 3, made explicit in §16.3.

16.1 The milestone ledger

The programme decomposes the object- and morphism-level identification $\mathbf{DCont} \cong \mathbf{Cof} = \mathbf{Cat}^\sharp$, and its enriched and ∞ -categorical extensions, into six milestones. Table 16.1 is the ledger: each row carries the registry node, a one-line statement, the *exact* trust grade recorded in the registry, the Lean declaration or proof file, and the formalisation status. Grades are quoted verbatim from the JSON; they are not rounded up.

16.2 The verified core: M1–M4

The four load-bearing milestones, together with the two converses, are all graded *lean-verified* in the registry, with zero `sorry` and axiom dependencies restricted to `[Quot.sound]` (via `funext`); the Dirichlet-coherence sub-file depends on no axioms at all.

M1 — representation. Registry node `m1-representation`, `Lean.Container.repr_full_faithful`, graded *lean-verified*. This is the statement that the extension $\llbracket - \rrbracket : \mathbf{Cont} \rightarrow \mathbf{End}(\mathbf{Set})$ is full and faithful onto the polynomial functors — the foundation the whole chain stands on, so that every downstream identification may be read either in $\mathbf{Cont}$ or in $[\mathbf{Set}, \mathbf{Set}]$.

M2 / M2b — directed containers are comonads. Node `m2-comonad-laws` (`Lean.DirectedContainer.comonad_laws`) proves the forward direction: the comonad laws on $\llbracket S, P \rrbracket$ are exactly (D1)–(D5), the left counit = (D1)+(D2), the right counit = (D3), coassociativity = (D4)+(D5). Node `m2b-converse` (`Lean.ComonadConverse.recover_forward_eq_id`) proves the converse and certifies *both* round-trips by `rf1`. Both are *lean-verified*; the mathematics is Ahman–Chapman–Uustalu [25].

Table 16.1: Milestones M1–M6, graded against `equivalence-chain.json`. Grades are the registry `trust` fields verbatim. Only M1–M4 (and the converses M2b, M3b) are lean-verified; M5 and M6 are not formalised.

M#	Node	Statement	Grade	Lean module / Status file
M1	m1-representation	Container representation: $\llbracket - \rrbracket$ full and faithful	<i>lean-verified</i>	<code>Container.repr_full_and_faithful</code>
M2	m2-comonad-laws	Directed container $\Rightarrow$ comonad (D1–D5)	<i>lean-verified</i>	<code>DirectedContainer.formalised_comonad_laws</code>
M2b	m2b-converse	Comonad $\Rightarrow$ directed container; both round-trips by <code>rfl</code>	<i>lean-verified</i>	<code>ComonadConverse.formalised_recover_forward_eq_id</code>
M3	m3-comonoid-forward	Directed container is a $\triangleleft$ -comonoid in Cont	<i>lean-verified</i>	<code>DirectedContainer.formalised_toComonoid (Comonoid.lean)</code>
M3b	m3b-comonoid-converse	Every $\triangleleft$ -comonoid is a directed container; iso of packagings	<i>lean-verified</i>	<code>Container.Comonoid.formalised_toDirectedContainer_toComonoid (ComonoidConverse.lean)</code>
M4	m4-dcont-cof	DCont = Cof : morphism-level iso of categories	<i>lean-verified</i>	<code>DContMorphism.formalised_toCofunctor</code>
M5	m5-enriched-dcont-cof	Enriched DCont = Cof	<i>unclassified</i>	—
M6	m6-infinity-dcont	∞ - DCont = Cat (Chen, Conj. 7.2)	<i>speculative</i>	—
				open, not formalised conjecture, not formalised

M3 / M3b — the $\triangleleft$ -comonoid packaging. Node `m3-comonoid-forward` (Lean `Containers.DirectedContainer`, file `Comonoid.lean`) states the same data *internally* to $(\mathbf{Cont}, \triangleleft, \text{Id})$ — no `Ext`, no `[Set, Set]`: right counit = (D3), left counit = (D1)+(D2), coassociativity = (D4)+(D5). Its converse `m3b-comonoid-converse` (Lean `Containers.Container.Comonoid.toDirectedContainer_toComonoid`, file `ComonoidConverse.lean`) is fully verified with *both* round-trips certified (counit by `rfl`, comultiplication via `ContainerMorphism.ext_eq`), upgrading M3/M3b to a genuine isomorphism of packagings $\text{DirectedContainer } C = \text{Comonoid } C$. Both *lean-verified*.

M4 — the morphism-level isomorphism $\mathbf{DCont} = \mathbf{Cof}$. Node `m4-dcont-cof`, Lean `Containers.DContMorphism`, graded *lean-verified*. This is the delicate milestone: the correct morphisms are *cofunctors*, not functors, because the position map $f_s^\# : P'(fs) \rightarrow P(s)$ is contravariant. The registry preserves the refuted naive reading as an explicit dead-end (node `morphisms-are-functors`, grade *dead-end*, refutation *computed*: “`#functors` $\neq$ `#cofunctors` in 17 of 36 test pairs”), and carries a provenance note (2026-07-14) recording that the original `Cofunctor.lean` was orphaned by a PR-close during a repo reorg, not lost to an out-of-memory event, and that `DContCat.lean` is a rebuild of an artifact that had been on a feature branch all along — lesson: *merge or back up before stacking*.

Remark 16.1 (Axiom hygiene). The registry records the `#print axioms` audit for the comonoid layer: `toComonoid` and its relatives depend on `[Quot.sound]` only, while the Dirichlet monoidal structure (`contDirichletMonoidal`, node `four-monoidal-lean`) depends on *no* axioms — every coherence is `rfl`. This is the sense in which “*lean-verified*” in Table 16.1 is a strong grade: it is not merely “compiles” but “compiles with an audited and minimal axiom footprint”.

16.3 The open frontier: M5 and M6 (Honest Ledger item 3)

This section states plainly what the registry records and what it does not. Neither M5 nor M6 is *lean-verified*, and — against the roadmap’s informal “proved-on-paper” framing — neither is graded *proved* in the registry either. This is the third item of the dossier’s Honest Ledger.

M5 — enriched DCont = Cof. Registry node `m5-enriched-dcont-cof`, attributed in the registry to Clarke–Di Meglio, graded *unclassified*. The registry note is unambiguous: “Clarke–Di Meglio 2022 state this is open. No sequel since.” So M5 is *not* proved on paper; it is an open problem inherited from the literature, with no formalisation and no proof artifact on disk.

M6 — ∞ -DCont = Cat. Registry node `m6-infinity-dcont`, graded *speculative*. The registry records this as Kan Chen’s Conjecture 7.2: that an ∞ -categorical directed-container notion recovers **Cat** (the $(\infty, 1)$ -categorical target). It is a conjecture, not a theorem; there is no proof file and no Lean module.

Remark 16.2 (Honest Ledger item 3, stated for the record). Milestones M5 and M6 are proved neither in Lean nor (uniformly) on paper: the registry grades them *unclassified* and *speculative* respectively, and the M5 source explicitly declares the enriched statement open. Any claim in the grant narrative that “the equivalence programme is fully formalised” must be read as scoped to M1–M4 (with converses M2b, M3b). The formalised core is exactly the non-enriched, 1-categorical chain; the enriched and ∞ -categorical extensions are frontier, not ledger.

16.4 Adjacent verified structure

The registry carries further *lean-verified* material orbiting the milestones, useful to cite when the grant narrative reaches beyond the bare chain. The Zappa–Szép associativity criterion (node `zs-lean`, `Lean MatchedPair.zs_iff_assoc`) is *lean-verified*. All four monoidal structures on **Cont** — coproduct, product, Dirichlet tensor, and the sequential operator $\triangleleft$ — are machine-checked with bifunctorial action, associators, unitors, naturality, pentagon and triangle, plus extension-functor comparison isos (node `four-monoidal-lean`, *lean-verified*), as is the comparitor $C \otimes D \rightarrow C \triangleleft D$ exhibiting the Dirichlet tensor as the uniform fragment of $\triangleleft$ (node `dirichlet-is-uniform-fragment-of-seq`, *lean-verified*, with the machine-checked non-surjectivity witness `dirToSeq_not_surjective`).

Remark 16.3 (Grades of the surrounding theory are mixed — cite carefully). Not everything adjacent to the chain is verified, and the registry is deliberate about the gap. The Day-family classification (node `day-family-classification`, Theorem A) is graded *proved* (paper, not Lean); the uniform closure formula (`uniform-closure-formula`) is only *computed* — one hand-check plus literature instances, no write-up artifact (`file: null`) — and the closed convolutional classification (`closed-convolutional-classification`) is *speculative*, missing its necessity direction. The comparitor no-go (`comparitor-comonoid-nogo`) is *speculative*. The fibrational comonoid layer (`fibrational-comonoid-layer`) is graded *peer-reviewed* in the sense that the equivalence is an established, refereed result (Shapiro–Spivak [30]) — not that anything new was proved; a full-PDF audit found it scooped in full. When these results feed the grant, quote the registry grade, not the ambition. Only Table 16.1’s M1–M4 rows carry the *lean-verified* guarantee.

Chapter 17

The sorry-free Lean corpus

The theory of this dossier is not only proved on paper; its load-bearing theorems are machine-checked. The formalisation lives in a single Lean 4 package, **Containers**, comprising **66** modules that build the category **Cont** of containers, the sub-theory of directed containers **DCont** $\cong$ **Cat**, the four monoidal structures on **Cont**, and the application-facing results (Zappa-Szép welds, emergent holonomy, trust and trajectory systems). The corpus is deliberately austere: almost every module is *Type*-level Lean 4 core with **no Mathlib dependency**, so the trust base is the Lean kernel plus, where noted, `funext/Quot.sound` (propositional extensionality). The corpus is **sorry-free**: every literal occurrence of the token **sorry** across all 66 files sits inside a docstring asserting sorry-freeness, never in a proof term.

17.1 What “lean-verified” means, and what it does not

We grade this corpus **lean-verified**. The standing caveat is the usual one and must be stated plainly: a machine-checked proof term certifies that the *stated* proposition type-checks, not that the stated proposition is the one a reader intends by the English name of the theorem — the statement, not the proof, is where residual trust lives. Accordingly the table below records each module’s headline *as the module’s own docstring states it*, and flags in the Notes column every caveat the sources themselves declare: a narrowed scope, a step left informal, or a genuine axiom footprint (`noncomputable/open Classical`). Several modules go further and pin their footprint with an in-file `#print axioms` audit; where a docstring makes a positive axiom claim we record it. Modules with an em dash in Notes are clean: they close by `rfl`, kernel `decide`, or definitional proof irrelevance, with at most `funext/Quot.sound`.

The three-column longtable runs across page breaks. Module and declaration names are set in `typewriter`; the middle column paraphrases the docstring headline.

Module	What it establishes	Notes
ActionLifting	Neil’s All ($\prod$) predicate lifting is a cartesian-only action of $\triangleleft$: <code>actionAll_assoc</code> , <code>actionAll_unit</code> , <code>actionAll_snd</code> .	lean-verified
AffineClassification	Theorem T1: monads on $MX = E + A \times X$ are monoids on $E \sqcup A$ with E a left-zero ideal (kernel- <code>decide</code> bijection).	Set-monad level; non-cartesian aborting monads out of scope

continued on next page

Module	What it establishes	Notes
Basic	Containers, their extension (polynomial) functor Ext, morphisms, and the representation theorem.	lean-verified
BiKleisli	Abstract biKleisli category of a mixed distributive law: unit laws <code>unit_left/unit_right</code> and <code>acomp_assoc</code> .	(3) $\iff$ (4) non-branching step separate, not formalised here
BiKleisliAffine	Effect-coeffect arrow category for the whole non-branching class $E + A \times (-)$; all four E1'–E4' axioms.	lean-verified
BiKleisliMaybe	Effect-coeffect arrow category for $M = \text{Maybe}$, all four E1'–E4' discharged.	lean-verified
BiKleisliWriter	Effect-coeffect arrow category for $M = \text{Writer } A$ over an arbitrary monoid; E1'–E4'.	lean-verified
BranchingObstruction	Negative half of the dichotomy: for branching M the reverse compositor κ fails E2'.	lean-verified
CoKleisli	CoKleisli extension, composition and canonical strength for directed-container comonads.	lean-verified
Cofunctor	Directed-container morphisms are cofunctors: hom-set bijection $\mathbf{DCont} \cong \mathbf{Cof}$, both round trips.	lean-verified
ComonadConverse	Converse (M2b): a container comonad recovers a directed container; <code>cShape</code> forced to Id.	lean-verified
Comonoid	A directed container is a $\triangleleft$ -comonoid in $(\mathbf{Cont}, \triangleleft, I)$ via <code>toComonoid</code> .	lean-verified
ComonoidConverse	Converse (M3b): a $\triangleleft$ -comonoid in $(\mathbf{Cont}, \triangleleft, I)$ is a directed container.	lean-verified
CompLeftAdjoint	Explicit left adjoint $F_q \dashv (-) \triangleleft q$: object/morphism maps, unit, counit, both triangles.	lean-verified
CompLeftAdjointNatQ	Naturality of F_q in the parameter q : contravariant functoriality and mate identity.	lean-verified
Composition	Container composition $\triangleleft$ ($\llbracket G \triangleleft F \rrbracket = \llbracket G \rrbracket \circ \llbracket F \rrbracket$); unitors and associator as isos.	lean-verified
Cont	The category $\mathbf{Cont}$: identity/composition of container morphisms, category laws hold definitionally.	lean-verified
DContCat	Directed containers as small categories; morphisms as cofunctors; round-trip position bijection.	lean-verified
Directed	Directed containers (D1–D5) and the comonad induced on the extension $\text{Ext}C$.	lean-verified
Dirichlet	The Dirichlet tensor $(\mathbf{Cont}, \otimes, y)$; positions $Ps \times Qt$ vs. the product's $Ps \oplus Qt$.	taxonomy coda left informal (needs quotients)
DirichletClosed	$(\mathbf{Cont}, \otimes, y)$ is closed: internal hom <code>ihom</code> , adjunction <code>dirichlet_closure</code> , currying naturality.	lean-verified; all <code>rfl</code>

continued on next page

Module	What it establishes	Notes
DirichletComonoid	Bare $\otimes$ -comonoids in Cont are families of monoids (forward, <code>toFamilyOfMonoids</code>).	axiom-audited (<code>Quot.sound</code> only)
DirichletComonoidConverse	Converse: a family of monoids assembles a bare $\otimes$ -comonoid.	axiom-audited
DirichletHomPi	Dirichlet internal hom in Π -form: $[q, r] \cong \prod_i (r \triangleleft q[i] \cdot y)$.	lean-verified
DirichletMonoid	Bare $\otimes$ -monoids are a monoid on shapes plus an oplax functor on fibres (forward).	axiom-free (no <code>Quot.sound</code>)
DirichletMonoidConverse	Converse: monoid-on-shapes plus oplax fibres assembles a bare $\otimes$ -monoid.	axiom-audited
Disjointness	The Disjointness Lemma for internal exact factorisations (<code>disjointness</code>).	axiom-audited
DualTransfer	The comonad $\rightarrow$ monad transfer on containers.	lean-verified
EmergentHolonomy	Emergent holonomy: orchestration synthesises a stabiliser neither agent has (<code>emergent_holonomy</code>).	axiom-audited (no <code>Classical.choice</code>)
EndpointLocality	A functor out of the codiscrete category is a trivial iso-system (endpoint locality).	lean-verified
EnrichedFFCollapse	The enriched-ff classification collapse $\mathcal{CL} \cap \mathcal{UN} = \{\text{Id}\}$.	final Id-monad-uniqueness step not formalised
FibredTransfer	G_M is a fibred comonad: reindexing-stability and cartesian preservation.	lean-verified
FourMonoidal	The four monoidal structures on Cont ($\triangleleft$, $\otimes$, product, coproduct).	lean-verified
Free	The free monad on a container as a $\triangleleft$ -monoid.	lean-verified
FreeUniversal	The universal property of the free monad on a container.	lean-verified
HolonomyWitness	$\mathbb{Z}/2$ -holonomy witness: machine-checked refutation of “liftings are holonomy-free”.	lean-verified
MonadComonadTransfer	The monad $\rightarrow$ comonad transfer on containers.	lean-verified
Monoidal	(Cont , $\triangleleft$, I) is a monoidal category (coherence).	lean-verified
PlethysmCancellation	Bottom-degree cancellation: algebraic engine of plethysm right-cancellation.	bottom-degree ($a_0=0$) engine only per <code>LEAN.md</code> ; <code>psOrd</code> classical/ <code>noncomputable</code>
ProofSearch	Proof-search trees as a directed container.	lean-verified
ProofSearchN	N-ary proof-search trees as a directed container.	lean-verified
QpfCollapse	QPFs as the constructive presentation of a collapse/identification locus ($\text{Bag}_2 = \text{Sym}^2$).	repr noncomputable (<code>Classical.choose</code>), intrinsic to QPFs
ReaderGroupoidLifting	The $\mathbb{Z}/2$ groupoid: a Reader-monad lifting that is neither $\prod$ nor Σ .	lean-verified

continued on next page

Module	What it establishes	Notes
ReaderStateOutsidePiMendler	Reader and State fall outside Π -Mendler: their μ drops leaves (κ_μ non-total).	lean-verified (decide)
Reentrancy	The re-entrancy obstruction $[\omega] = \varepsilon$ as finite $\mathbb{F}_2$ linear algebra; $H^2 \cong \mathbb{F}_2$.	lean-verified
SeqCoproductDistrib	Left distributivity of $\triangleleft$ over the coproduct.	lean-verified
SeqProdDistrib	Right distributivity of $\triangleleft$ over the product.	lean-verified
Sequential	The sequential operator $\triangleleft$ and its monoidal structure.	lean-verified
SigmaLift	The Σ -container lifting is $M \triangleleft (-)$: a $\triangleleft$ -monoid makes it a monad on Cont .	lean-verified
StateComonad	The state object ΔS , the codiscrete category, and the store comonad.	lean-verified
StateComonadTensor	Lemma 3.1 at the comonad level: the store comonad tensors.	lean-verified
StateProductLifting	The product of directed containers, and the store category $\mathbb{S} \times C$.	lean-verified
TMCartesianBoundary	The T_M -side cartesian-preservation boundary: Maybe preserves, Pf does not.	lean-verified (decide)
TensorCoproductDistrib	Distributivity of the Dirichlet tensor over the coproduct.	lean-verified
TimesCoproductDistrib	Distributivity of the categorical product over the coproduct.	lean-verified
TimesMonoid	Bare $\times$ -monoids are a monoid on shapes plus an oplax functor on fibres.	lean-verified
TimesMonoidConverse	Converse: monoid-on-shapes plus oplax coproduct fibres assembles a bare $\times$ -monoid.	axiom-audited
Trajectory	Trajectory systems: circular verification implies $\tau = \perp$.	lean-verified
TrajectoryComposition	Composition of trajectory evaluation with trust evaluation.	lean-verified
Trust	Trust systems: the certificate lemma and transport of trust; grade τ .	tau/derivGrade noncomputable (well-founded recursion, open Classical)
TrustBoundary	The trust boundary on registry trees is stable under duplicate .	lean-verified
UnionSubspaces	The union-of-subspaces counting lemma.	lean-verified
VecCollapse	The Vec _{fd} collapse locus: two containers, one linear functor.	lean-verified
Workers	The category of Workers, graded by $(\text{Set}, \times)$.	lean-verified
WorkersRetract	The Workers $\otimes$ -grading is a retract of the BHM $\triangleleft$ -grading.	lean-verified
ZappaSzep	The Zappa–Szép product of monoids: cocycle axioms $\iff$ associativity.	lean-verified

Chapter 18

H^2 -engines and the free/cofree comonad in Lean

This chapter reports the machine-checked core of the programme’s two structural pillars, as they are *literally* stated in the Lean 4 sources. First: the free monad on a container, built as a monoid in $(\mathbf{Cont}, \triangleleft, \text{Id})$, together with its universal property. Second: three H^2 -engines that compute holonomy classes $H^2(C_n; \mathbb{Z}/m)$ as explicit quotient groups. Every declaration cited below is **sorry**-free and axiom-clean (**Quot.sound** at worst); in keeping with the “Lean statement self-audit” discipline, we state each theorem as it stands and disclose every narrowed scope. All files are Lean 4 core, Mathlib-free.

18.1 The free monad as a $\triangleleft$ -monoid

The file `Containers/Free.lean` [lean-verified] realises the free monad on a container $C = (S \triangleleft P)$ as a *monoid* in the monoidal category $(\mathbf{Cont}, \triangleleft, \text{Id})$ — the exact mirror of the directed-container-as- $\triangleleft$ -comonoid picture. It is the container-coordinate discharge of Gambino–Kock’s Theorem 4.5, whose monad laws are there declared “lengthy but routine” and omitted.

Definition 18.1 (The free container). `Container.free` sends C to the container whose shapes are closed P -trees `PTree C` (the initial algebra of $Y \mapsto 1 + \Sigma_{s:S}(Ps \rightarrow Y)$) and whose positions are their *leaves* `leaves`, not all vertices. `Directions = leaves` is precisely the free/cofree distinction: the cofree comonad carrier would take `directions = all vertices` (Niu–Spivak Prop. 8.18).

The structure maps are `Container.freeUnit` ($\eta : \text{Id} \Rightarrow m$, picking the bare leaf `lf`) and `Container.freeMult` ($\mu : m \triangleleft m \Rightarrow m$), whose forward component is grafting `graft` and whose backward component is `split`, the section that cuts a leaf-path at its unique t -leaf boundary.

Theorem 18.2 (Free monad, three monoid laws). *`Container.freeMonoid` [lean-verified] exhibits $C.\text{free}$ as a term of `Container.Monoid`, i.e. it discharges all three $\triangleleft$ -monoid laws as equations of container morphisms:*

- *left unit* by `rfl` on both components;
- *right unit*: shape half `graft_unit_right` ($\text{graft } t (\lambda_. \text{lf}) = t$), position half `split_unit_right`, reconciled by the transport lemma `leaves_nd_transport`;
- *associativity*: shape half `graft_assoc` (Lemma *C*), position half `split_assoc` (Lemma *D*), the transported `ContainerMorphism.ext_eq` position obligation.

Axiom-clean (Quot.sound only).

The honest content here is that `split_assoc` states, verbatim, the regrouping of a leaf of the doubly-grafted tree into $\langle\langle\ell, w\rangle, x\rangle$ — the associativity of leaf-path concatenation — transported along `graft_assoc`. Nothing is hidden behind an abbreviation: the statement is the raw `ext_eq` obligation.

18.2 The universal property of the free monad

`Containers/FreeUniversal.lean` [lean-verified] adds the universal arrow from X to the forgetful functor $U : \text{Mon}(\mathbf{Cont}) \rightarrow \mathbf{Cont}$. The insertion of generators is `Container.freeInsert` ($\alpha_X : X \rightarrow m_X, s \mapsto \text{nds}(\lambda_. \text{lf})$). Given a monoid M and $g : X \rightarrow M$, the extension `Container.freeExtend` packages the shape map `freeExtShape` (tree recursion: a leaf goes to the unit ε , a node to μ_M) with the position map `freeExtPos`.

Theorem 18.3 (The adjunction triangle). *`Container.freeExtend_triangle` [lean-verified] proves $\alpha_X ; \hat{g} = g$ in full (both components). The shape half is `freeExtShape_triangle`, which uses M ’s right-unit law `mult_right_unit`; the position half uses its backward companion `mult_right_unit_pos`. `Container.freeExtend_unit` separately proves $\hat{g}$ preserves the monoid unit.*

Theorem 18.4 (Multiplication homomorphism, shape half; uniqueness on shapes). *`Container.freeExtShape_mul` [lean-verified] proves the shape half of the multiplication homomorphism law $\mu ; \hat{g} = (\hat{g} \triangleleft \hat{g}) ; \mu_M$, by tree induction reducing to M ’s left-unit `mult_left_unit` (base) and associativity `mult_assoc_shape` (step). `Container.freeExtShape_unique` proves that any shape map matching the two recursion equations of $\hat{g}_1$ equals $\hat{g}_1$ — object-level uniqueness of the extension, by a transport-free structural induction.*

Remark 18.5 (Disclosed scope — what is *not* yet formalised). The file’s own header states the boundary, and honesty requires repeating it. Two pieces remain open:

1. the **backward (position) half** of the multiplication homomorphism law (`freeExtPos_mult`): its two M -law inputs `mult_left_unit_pos` and `mult_assoc_pos` are in place and the *base case* (`lf`) is verified, but the *node step* is not;
2. **backward-uniqueness**, the position half of `freeExtShape_unique`, which needs `split-bijectivity` not yet in `Free.lean`.

Thus the universal property is machine-checked *on shapes* throughout, and *on positions* for the triangle and the unit law, but the full position-level multiplication homomorphism and position-level uniqueness are pending. The chapter claims exactly this and no more.

The declarations `mult_left_unit_pos`, `mult_assoc_pos`, `mult_assoc_shape`, and `mult_right_unit_pos` are extracted from M ’s packaged monoid laws by `ContainerMorphism.onPos_congr` and `congrArg onShapes`; they are *hypotheses* (“ M is a monoid”), never obligations of this file.

18.3 The unified cyclic H^2 -engine: $H^2(C_n; \mathbb{Z}/m) \cong \mathbb{Z}/\gcd(n, m)$

`2026-09-18-cyclic-h2-gcd.lean` [lean-verified], namespace `CyclicH2Gcd`, computes, for $C_n = \mathbb{Z}/n$ acting *trivially* on $M = \mathbb{Z}/m$, the group $H^2(C_n; \mathbb{Z}/m)$ as the cokernel of the norm map $\cdot n$ on $\mathbb{Z}/m$, identified with $\mathbb{Z}/(n\mathbb{Z} + m\mathbb{Z})$.

Theorem 18.6 (Bézout and the arithmetic core). *bezout [lean-verified] proves $\gcd(a, b) = a \cdot s + b \cdot t$ over $\mathbb{Z}$ from scratch by `Nat.gcd.induction` (no `Mathlib`). `dvd_iff` proves $(\exists s t, x = ns + mt) \iff \gcd(n, m) \mid x$, i.e. $n\mathbb{Z} + m\mathbb{Z} = \gcd(n, m)\mathbb{Z}$.*

Theorem 18.7 (The cohomology group and its gcd identification). *`H2 n m` is defined as the explicit Quotient of $\mathbb{Z}$ by `cobRel` ($a - b \in n\mathbb{Z} + m\mathbb{Z}$), carrying a by-hand abelian-group structure (`instAdd`, `instZero`, `instNeg`, with `H2.add_comm`, `H2.add_assoc`, `H2.zero_add`, `H2.add_zero`, `H2.neg_add_cancel`). `H2.mk_eq_iff_dvd` proves $\llbracket a \rrbracket = \llbracket b \rrbracket \iff \gcd(n, m) \mid (a - b)$, and `H2.mk_eq_zero_iff_dvd` its $b = 0$ form. All [lean-verified].*

The two prior engines fall out as specialisations, verified in-file: `base_edge_recovers` recovers $H^2(C_n; \mathbb{Z}) \cong \mathbb{Z}/n$ as the case $m = 0$ (via `Nat.gcd_zero_right`), and `fibre_edge_recovers` recovers $H^2(\mathbb{Z}/2; \mathbb{Z}/2) \cong \mathbb{Z}/2$ as $n = m = 2$. The $K_{2,3}$ cardinality witnesses `card_2_0`, `card_3_0`, `card_2_2`, `card_2_4`, `card_4_2`, `card_3_3`, and the cover `cover_2_2`, exhibit exactly $\gcd(n, m)$ distinct classes for each concrete pair.

Remark 18.8 (Disclosed black box). The Lean deliverable is the *algebra* `coker`($\cdot n$ on $\mathbb{Z}/m$) $\cong \mathbb{Z}/\gcd(n, m)$. That the reduced 2-periodic complex *computes group cohomology* of C_n is cited (Brown, *Cohomology of Groups*, §VI.9), not re-proved. The topology is a black box; the arithmetic is fully formal.

18.4 The two superseded engines: norm and fibre

The unified engine of Section 18.3 subsumes two earlier, still-verified files that make the reduction concrete on each edge of the H^2 -cluster.

Base edge (norm map). `2026-09-16-cyclic-h2-norm.lean` [lean-verified], namespace `CyclicH2`, formalises the reduced cyclic cochain complex `d0...d3` with `d1 = d3 = ·n` (the norm) and `d0 = d2 = 0` under the trivial action. It proves the complex identities `d1_comp_d0`, `d2_comp_d1`, `d3_comp_d2`; that every element is a 2-cocycle (`twoCocycle`) and the coboundaries are exactly $n\mathbb{Z}$ (`twoCoboundary_iff`); and builds `H2 n =  $\mathbb{Z}/n\mathbb{Z}$` with its abelian-group structure and `H2.mk_eq_iff_dvd`. The grant-critical $K_{2,3}$ corollaries `H2_two_distinct`, `H2_two_cover`, `H2_three_distinct`, `H2_three_cover` give $H^2(C_2; \mathbb{Z}) \cong \mathbb{Z}/2$ and $H^2(C_3; \mathbb{Z}) \cong \mathbb{Z}/3$ as “ n classes covering $\mathbb{Z}$.”

Fibre edge ($\mathbb{Z}/2$ coefficients). `2026-09-17-fibre-h2-z2.lean` [lean-verified], namespace `FibreH2`, computes $H^2(\mathbb{Z}/2; \mathbb{Z}/2) \cong \mathbb{Z}/2$ from the *normalized bar* complex of C_2 directly, modelling $M = \mathbb{Z}/2$ as `Bool` under `bne`. Because the norm $\cdot 2$ is identically zero (`Z2.add_self`), the coboundary image collapses to $\{0\}$. The split/nonsplit dichotomy — the substance of the decoupling witness — is fully formalised:

- `fZ4` (the carry cocycle $f(a, b) = a \wedge b$, factor set of $0 \rightarrow \mathbb{Z}/2 \rightarrow \mathbb{Z}/4 \rightarrow \mathbb{Z}/2 \rightarrow 0$) is a genuine 2-cocycle (`fZ4_cocycle`) and `fZ4_not_coboundary` proves it is *not* the coboundary of *any* 1-cochain — the nonsplit defect is real;
- `fKlein` (the constant 0 factor set of the Klein extension) splits: `fKlein_isCoboundary`;
- hence `classZ4_ne_zero`, `classKlein_eq_zero`, and the decoupling conclusion `classZ4_ne_classKlein`: $[\pi] \neq 0 = [\text{Klein}]$ in $H^2(\mathbb{Z}/2; \mathbb{Z}/2)$.

The abstract identification of this bar H^2 with the LHS fibre edge $E_2^{0,2}$ is cited (§VI.9; the placement is in `h2cluster-lhs-transgression`), not re-proved; the concrete cocycle classes and the group are fully formal.

Remark 18.9 (Provenance and role in the dossier). These three engines supply the H^2 -valued invariants that decide compositional decomposability throughout the programme; the free-monad construction of Sections 18.1–18.2 supplies the dual, generative half — the $\triangleleft$ -monoid whose comonoid mirror is the directed container [17]. Together they are the formalisation deliverable: an H^2 -engine to detect obstructions and a machine-checked free/cofree construction to build the composites, in the reduced coordinate form of the module reduction [8].

Chapter 19

Change-of-base adjunction and collapse theorems in Lean

This chapter reports the machine-checked core of the change-of-base story: the left adjoint to sequential composition $(-) \triangleleft q$, its parameter naturality, and four collapse phenomena that certify where the container-extension functor fails to be injective on objects or faithful on morphisms. Every declaration cited below is *lean-verified*; the whole corpus is sorry-free and, apart from one use of `Classical.choose`, builds on Lean 4 core with no Mathlib. In keeping with the dossier’s self-audit discipline, each statement is described exactly as the source file states it — including the variances and the disclosed narrowings, which are the load-bearing content rather than incidental.

19.1 The left adjoint $F_q \dashv (-) \triangleleft q$

Fix a container $q = (T \triangleleft Q)$ and write $L_q p := p \triangleleft q$ (`Container.seq`) for right whiskering by q , an endofunctor of **Cont**. The file `CompLeftAdjoint.lean` builds an explicit left adjoint

$$F_q(R \triangleleft U) = (R \triangleleft \lambda \rho. \llbracket q \rrbracket(U \rho)), \quad \llbracket q \rrbracket X = \sum_{t:T} (Q t \rightarrow X),$$

i.e. F_q is the identity on shapes and applies the extension of q to every position set. The object map is `Container.leftAdj`; the action on morphisms is `Container.leftAdjMap`. The decisive point recorded in the file is the *variance*: a `ContainerMorphism` runs forward on shapes and *backward* on positions, so F_q must run $\llbracket q \rrbracket$ over the backward leg, `Ext.map` ($\varphi.\text{onPos } \rho$); pushing $\llbracket q \rrbracket$ forwards, as a shape-level reading of the formula suggests, does not even typecheck. The functor laws are `leftAdjMap_id` and `leftAdjMap_comp`.

The unit $\eta_r : r \rightarrow F_q r \triangleleft q$ (`Container.adjUnit`, the “generic element” map) and counit $\varepsilon_p : F_q(p \triangleleft q) \rightarrow p$ (`Container.adjCounit`) are given as explicit container morphisms.

Theorem 19.1 (triangle identities, *lean-verified*). $\varepsilon_{F_q r \circ F_q \eta_r} = \text{id}$ (`Container.triangle_leftAdj`) and $(\varepsilon_p \triangleleft q) \circ \eta_{p \triangleleft q} = \text{id}$ (`Container.triangle_seq`).

The file is careful to separate the triangles from naturality: the triangles say η, ε are compatible *pointwise*, but do not by themselves make them natural transformations. The two naturality squares $\eta : \text{Id} \Rightarrow L_q \circ F_q$ (`adjUnit_naturality`) and $\varepsilon : F_q \circ L_q \Rightarrow \text{Id}$ (`adjCounit_naturality`) supply the missing content; together with the functor laws for L_q (`whiskerRight_id`, `whiskerRight_comp`) and the two triangles they constitute the full adjunction $F_q \dashv L_q$. The hom-set bijection $\mathbf{Cont}(F_q r, p) \cong \mathbf{Cont}(r, p \triangleleft q)$ is packaged as the mutually inverse `adjTranspose` / `adjUntranspose` (round trips

`adjUntranspose_adjTranspose`, `adjTranspose_adjUntranspose`), with naturality in both variables. Here a second variance is stated exactly as in the file: because $\mathbf{Cont}(F_q -, p)$ is *contravariant in r* , the left-hand naturality square `adjTranspose_naturality_left` is indexed by $\varphi : r' \rightarrow r$.

Remark 19.2 (non-vacuity control). Every law is proved by `ContainerMorphism.ext' rfl` (`fun _ => rfl`). To rule out vacuity, `adjCounitPerturbed_naturality` exhibits a perturbed counit (injecting at a flipped `qBool`-position) that *still* satisfies the naturality square yet *breaks triangle_seq*: naturality and the triangles are genuinely independent probes, and each law's content lives on the position leg.

19.2 Parameter naturality: F is contravariant in q

The file `CompLeftAdjointNatQ.lean` certifies how F_q varies as q varies, and here the variance is stated exactly as the source does. Right whiskering $(-) \triangleleft q$ is *covariant in q* ; since the family of right adjoints is covariant, the family of left adjoints must be *contravariant*. Concretely, a morphism $\psi : q \rightarrow q'$ yields

$$F_\psi : F_{q'} \Rightarrow F_q \quad (\text{note the reversal}),$$

the transformation `Container.leftAdjOnq`, identity on shapes and $\psi.\text{toNat}$ on positions. This is the only assignment of the correct variance: $F_{q'} r \rightarrow F_q r$ runs backward on positions from $\llbracket q' \rrbracket$ to $\llbracket q \rrbracket$, which is exactly what $\psi.\text{toNat} : \mathbf{Ext} \, q \Rightarrow \mathbf{Ext} \, q'$ supplies.

Proposition 19.3 (contravariant functoriality, *lean-verified*). $F_{\text{id}_q} = \text{id}$ (*leftAdjOnq_id*) and $F_{\psi \circ \psi'} = F_{\psi'} \circ F_\psi$ (*leftAdjOnq_comp*) — the order reversing precisely because F is contravariant in q .

The remaining declarations record: naturality of F_ψ in r (`leftAdjOnq_naturality`); the two compatibility squares tying unit and counit at q and q' along ψ (`adjUnit_naturality_q`, `adjCounit_naturality_q`) the counit square being taken over the genuine shared corner $F_{q'} \circ L_q$ rather than a non-existent square between compound endofunctors; and the packaging `leftAdjOnq_eq_mate`, that F_ψ is the mate of L_ψ under $F_q \dashv L_q$ and $F_{q'} \dashv L_{q'}$. A perturbed candidate for F_{id} (`leftAdjOnqPerturbed_naturality`) again satisfies naturality-in- r while failing `leftAdjOnq_id`, isolating what naturality does and does not pin down.

19.3 The Vec biproduct collapse, and the non-faithfulness hedge

The file `VecCollapse.lean` formalises the Theorem-D collapse over $\mathbf{Vec}_{\text{fd}}$ [8]. The one idea is that over $\mathbf{Vec}$ a finite coproduct is a biproduct, so it equals the product: the linear extension replaces the shape-indexed $\sum$ of the set extension by a $\prod$,

$$\mathbf{LExt} (S, \text{Pos}) X = (s : S) \rightarrow (\text{Pos } s \rightarrow X),$$

positions being carried by their finite basis types. That single $\Sigma \rightarrow \Pi$ shift is the biproduct collapse. The two containers $p = (\{*\}, k^2)$ and $p' = (\{1, 2\}, k)$ then present the same functor $X \mapsto X \oplus X$: the collapse isomorphism $\mathbf{LExt} \, p \cong \mathbf{LExt} \, p'$ is `collapseIso` (a `LinNatIso`, with naturality), and both extensions are isomorphic — not merely surjected onto — the common two-power $\mathbf{Pow2} \, X = (\text{Bool} \rightarrow X)$ via `pIso`, `p'Iso` (`collapse_factors_through_cover`).

Proposition 19.4 (object half of Theorem D, *lean-verified*). *There is no bijection between the shape set of p (`Unit`) and that of p' (`Bool`): `shapes_not_equiv`. Hence $p \not\cong p'$ as containers although their linear extensions agree, and $\llbracket - \rrbracket$ is not injective on objects on the collapse locus.*

The file then formalises the *morphism* half, and the exact hedge must be carried: this corrects an earlier “faithful” clause, caught on 2026-09-11 by Robin Langer’s $\mathbb{F}_q$ re-derivation.

Theorem 19.5 (corrected Theorem D, morphism half — the extension is NOT faithful, **lean-verified**).

linExt_not_faithful states: the two distinct container morphisms $\text{zeroMor1} \neq \text{zeroMor2}$ of type $(\{*\}, k) \rightarrow (\{1, 2\}, k)$ over $\mathbb{F}_2$ — differing only in their shape map, both with zero position map — induce the same natural transformation $\text{LinExt}(\text{srcUnit}) \Rightarrow \text{LinExt}(\text{witnessD})$ at every pointed value type. The mechanism is *induced_zero_of_zeroPos* (the union-lemma collapse). Generalised: *not_faithful_of_two_shapes* gives, for any target with two distinct shapes, two distinct single-shape morphisms inducing the same transformation.

Together with **shapes_not_equiv** this machine-checks that over a field the $\oplus$ -extension is *neither* full *nor* faithful. The Lean witness verifies the non-faithful *direction* (the smallest and the $|T| \geq 2$ cases); the quantitative collapse-defect count and the two-sided equivalence “faithful $\iff |T| \leq 1 \vee S = \emptyset$ ” are cited from the proof file, not re-proved in Lean.

19.4 The QPF abs/repr collapse: $\text{Bag}_2 = \text{Sym}^2$

The file **QpfCollapse.lean** formalises the *abs/repr* (quotient-of-polynomial-functor) pattern as the constructive tool for identification loci. The witness is the two-leaf bag. The polynomial functor is the two-position container **pairC** ($\text{Shape} = \text{Unit}$, $\text{Pos} = \text{Bool}$), whose extension is $\text{Poly } X = X \times X$; the identification $\text{Ext pairC } X \cong \text{Poly } X$ is machine-checked (**polyOfExt**, **extOfPoly**, and the two round trips). Then $\text{Bag2 } X$ (Lean: **Bag₂**) is the quotient of $\text{Poly } X$ by the leaf-swap $(a, b) \sim (b, a)$, i.e. $\text{Sym}^2 X = (X \times X)/\text{swap}$.

Proposition 19.6 (the QPF structure, **lean-verified**). *abs = Quotient.mk; repr is the non-computable section built from Quotient.exists_rep; the split-surjection law $\text{abs} \circ \text{repr} = \text{id}$ is **abs_repr**. Naturality **abs_map** holds definitionally, and $\text{abs}(a, b) = \text{abs}(b, a)$ is **abs_swap**. The data is packaged as the record **MinimalQPF** with instance **bag2QPF** (Lean: **bag2QPF**) on **pairC**.*

Unlike **VecCollapse**, where **abs** is an isomorphism (an exact identification), here **abs** is a *proper* quotient by the leaf-swap; that swap is exactly the leaf-symmetry that obstructs Arr_M from being a category in the effect-coeffect result, so this is the smallest genuine collapse locus rather than a toy.

19.5 The enriched full-and-faithful collapse $\mathcal{CL} \cap \mathcal{UN} = \{\text{Id}\}$ (narrowed scope)

The file **EnrichedFFCollapse.lean** machine-checks the algebraic core of the classification: for a commutative strong monad M on **Set**, Neil’s self-enriched M -container extension $\sum_s [P_s, -] : \text{Kl}(M) \rightarrow \text{Kl}(M)$ is fully faithful iff $M = \text{Id}$.

Remark 19.7 (disclosed narrowed scope — state exactly). This file does *not* constitute a full internal proof of the classification. Two narrowings are disclosed by the source and must be carried verbatim:

- **Object-level statement.** The verified conclusion **NeilEnrichedFF.iso_id** is that $M X \cong X$ for every X — the collapse of the *underlying bare functor* of M . The upgrade to $M \cong \text{Id}$ as a monad is *not* formalised (the file states **Id** carries a unique monad structure and cites the paper for this final step). The bonus **writer_collapse_natural** checks only the naturality of the writer collapse, the sole new content of that upgrade.

- **Category-theory inputs abstracted as Lean hypotheses.** The two category-theoretic facts are taken as hypotheses in the record `NeilEnrichedFF`, not re-proved in Lean: `preserve` (membership in $\mathcal{UN}$: the copower comparison $MT \cong \sum_{t:T} M1$ is a bijection) and `top/terminal` (the consequence of $\mathcal{CL}$: a terminal object with $M\top \cong 1$). The file explicitly does *not* prove that closedness gives `terminal` nor that connectedness gives `preserve`; those steps (using Kock’s theorem and the T1 fullness result) are cited but not verified here.

Within that scope the development is complete and **lean-verified** (Mathlib-free, using a hand-rolled bijection structure `Bij`). Step 1, `writerForm`, derives the writer form $MT \cong T \times M1$ from `preserve`. The load-bearing algebra is `factorUnitOfProdUnit`: if a product $X \times E$ is a singleton then the factor E is a singleton (the proof turns on definitional eta for `PUnit`). Combining, `NeilEnrichedFF.E_iso_unit` gives $M1 \cong 1$, and `NeilEnrichedFF.iso_id` gives the object-level collapse $MX \cong X$.

Remark 19.8 (summary of the four collapses). The four loci sit on a spectrum of exactness. `VecCollapse` and `EnrichedFFCollapse` are *iso*-collapses (`abs` an isomorphism; a bare functor forced to be `Id`); `QpfCollapse` is a *proper* quotient collapse; and `VecCollapse` additionally certifies a genuine *failure* of faithfulness (Theorem 19.5). The composition adjunction of Sections 19.1–19.2 supplies the change-of-base machinery under which these presentations are compared.

Part IV

Impact

Chapter 20

Where compositional correctness has economic consequences

The applications collected in Part II are not a scattered list of domains that happen to admit a categorical gloss. They are, to a first approximation, the domains in which a *wrong weld costs money*: a re-entrancy bug drains a contract, a mis-composed audit misses a divergence, a wrong worker interleaving corrupts shared agent state. This short chapter does one job — it re-anchors that economic reading to the technical bodies that license it, so that the impact claim is a corollary of proved theorems rather than a piece of free-floating motivation. The single sentence it defends is: *all three failures are one obstruction*, the non-vanishing of $[\omega] \in H^2$.

20.1 The through-line: a wrong weld costs money

The recurring, expensive failure mode of engineered systems is not that a component is wrong but that two correct components are wrong *together*. This is exactly the shape the programme’s core theorem addresses. Composing two directed systems is a Zappa–Szép weld in $\mathbf{DCont} \cong \mathbf{Cat}$ (Chapter 4, Chapter 5), and that weld is correct precisely when a first-order lifting condition (L) holds and a degree-two obstruction vanishes:

$$\text{correct composition} \iff (\text{L}) \wedge [\omega] = 0 \in H^2.$$

Both halves are *proved* in the closed cases (the pairwise Zappa–Szép criterion and the identification (G) $\iff [\omega] = 0$; see Chapter 5), and the holonomy–Zappa–Szép bridge underlying the obstruction is *peer-reviewed*. The economic content of the chapter is the observation that the three most developed applications are precisely the settings where the composite — not the parts — is what carries value, and so where a non-vanishing $[\omega]$ is not a mathematical curiosity but a loss.

20.2 Three failures, one obstruction

Each application is obtained by specialising the base of the general construction and reading off what $[\omega]$ becomes. The uniformity is the point: the same class, computed in three domains, is three distinct known failure modes.

Smart contracts (Chapter 11). A contract’s storage and its call structure are two directed systems that must compose via a distributive law. Re-entrancy is the failure of that law, and the

obstruction lands in a two-element group: $[\omega] = \varepsilon \in \mathbb{Z}/2$, with $[\omega] = 0$ the safe contract and $[\omega] = 1$ the exploitable one. This is the sharpest specialisation in the programme — a finite, decidable obstruction whose arithmetic is *Lean-verified*, and whose standalone note is *peer-reviewed*. The 2016 DAO exploit, which drained on the order of tens of millions of dollars from a withdrawal function correct in isolation, is the canonical instance of $[\omega] = 1$; that figure is a historical fact about a deployed contract, not a quantity the model predicts. What the model contributes is the identification of the bug’s *location* — the interleaving, formalised as $[\omega] \neq 0$ — and the fact that checking it is a decidable computation.

Security audits (Chapter 14). An audit composes local checks, each of which may pass, into a verdict about the whole. A mis-composed audit is one whose composite misses a global divergence that no single local check can see — the defect is a property of the composite, invisible to the parts. In the programme’s language this is the same non-vanishing obstruction: an audit is sound exactly when the weld of its local certificates carries $[\omega] = 0$, and “the search for a bug” becomes “the search for a failed distributive law.” The structural identification is *proved*; the concrete audit calculus lives in Chapter 14.

Agent orchestration (Chapter 13). Composing two agent orchestrations is literally the Zappa–Szép weld of two directed containers: orchestrating them interleaves their control exactly as the mutual action of $C \bowtie D$. A wrong worker interleaving — a deadlock, or a corruption of shared agent state — is a non-vanishing $[\omega]$, and correct orchestration is again $(L) \wedge [\omega] = 0$. This identification is *proved*; the tool-calling / harness refinement (a protocol is a directed container, the harness its comonad, with $[\omega] = \varepsilon \in \mathbb{Z}/2$) is *Lean-verified* at the arithmetic core.

Remark 20.1 (why “one obstruction” is a claim, not a slogan). The three paragraphs above are not three analogies to a common template. They are three evaluations of *one* class $[\omega] \in H^2$ at three choices of base, and the class is known not to be an artefact of a single presentation: there are two genuinely distinct $[\omega]$ -sites, *not* related by isotropy restriction (*proved*), and $[\omega]$ sits as a base edge of the Lyndon–Hochschild–Serre coupling analysed in Chapter 7. That is why re-entrancy, audit divergence, and interleaving corruption can be the same obstruction without the identification collapsing into a tautology.

20.3 Why the specialisation is principled, not post-hoc

A natural worry is that reading an economic failure as $[\omega] \neq 0$ is a retrofit — that one could dress any bug in cohomological language after the fact. The programme’s answer is that the framework being specialised is itself *forced*, not invented. The directed-container structure is derived: unpacking what a comonad structure forces on a container produces the root, the sub-shape map, and the shift operation, and the comonad laws produce exactly the five directed-container laws — every piece determined by the problem, none postulated. The definition is a theorem, not a stipulation.

Remark 20.2 (speculative economic reading). The step from “the framework’s definitions are forced” to “therefore its economic specialisations are principled rather than opportunistic” is an interpretive, *speculative* argument, and is graded as such. It is a claim about *why the model deserves trust*, not a proved statement about any market. Concretely: no economic figure in this chapter is derived from the theory. The DAO loss is historical; the general assertion that compositional bugs are economically dominant is motivating context, not a measured quantity; and any projected cost-of-audit or cost-of-failure savings would require empirical work the programme has not done. What is

proved is narrower and stronger — that in these domains the correctness question *is* the vanishing of a specific, sometimes finite, obstruction.

20.4 The decision procedure and its economic reading

The payoff is a design principle, stated compactly: *to certify a composite directed system, check (L) and compute $[\omega]$* . Where $[\omega]$ lands in a finite group — as it does for the re-entrancy model, where $[\omega] = \varepsilon \in \mathbb{Z}/2$ — this is a *decidable* check, and its arithmetic has been *Lean-verified* (the H^2 -engine trio and the re-entrancy witness of Part III). The economic reading of this procedure is direct: an audit, in this language, is the computation of $[\omega]$, and a contract or an orchestration is certified safe when the computation returns 0. Where the obstruction is finite the certification is not merely conceptual but mechanical.

Remark 20.3 (honest scope of the impact claim). The impact claim is deliberately bounded. It is that these three domains are governed by a common, sometimes computable, obstruction — *proved* for orchestration and re-entrancy, *peer-reviewed* and *Lean-verified* for the re-entrancy witness, *proved* (structural) for the audit identification. It is *not* a claim that the programme has quantified any economic saving, published any external result, or built the tax-computation application named in the grant (which has *no artifact*; see the Honest Ledger). The value on offer is a correct localisation of where composition fails and a decision procedure where the obstruction is finite — which, in domains where a wrong weld costs money, is precisely the leverage worth having.

Chapter 21

The AI + Lean pipeline: honesty and trust-grade methodology

This is the programme’s genuine differentiator, and it is a *method*, not a boast. Everything else in this dossier is content; this chapter is the discipline that makes the content trustworthy when its author is an AI agent.

21.1 The problem this solves

The premise is now demonstrated in public. AI-authored Lean proofs are real: a 2026 refutation of Köthe’s conjecture — a ninety-year-old open problem — ships with a Lean 4 formalisation authored autonomously by a pre-release system and then human-reviewed [20]. So the pipeline the grant is built on — AI produces a machine-checked resolution, humans verify — exists.

The same literature demonstrates the failure mode just as clearly, and it is the one that matters here: “verified” does not mean “reusably or correctly stated.” Ilin and Nugent autoformalised Grothendieck vanishing to a *zero-sorry* proof on the first pass, yet expert review found the definitions, the theorem’s generality, and the API unusable as a library contribution [21]. *TaoBench* shows state-of-the-art provers losing roughly a quarter of their success rate when the *same* theorems are stated in a non-idiomatic definitional framework [22] — the bottleneck is the statement, not the search. And a documented agentic-prover incident had the system exploit a typo in a user’s axioms to prove the target *vacuously*, silently rewriting a **theorem** into a **variable**. The lesson is blunt: a pile of green, **sorry**-free checkmarks is not yet a trustworthy body of work. What follows is the machinery that turns one into the other. (The external citations of this section are held at *agent-summary* / *abstract* extraction level; they are grant-narrative evidence, not a MacBeth deep-read.)

21.2 The trust-grade ladder

Every nontrivial claim in the programme carries an honest grade drawn from the ladder of the front matter,

speculative < computed < proved < peer-reviewed < published < lean-verified,

with a finer seven-level refinement in the source-of-truth deployment (`code/macbeth.json`) that inserts *peer-claimed* — “a peer asserts it, with a `claimed_by` field” — just below the *proved* boundary. The operational meanings are what give the ladder teeth:

- *computed* is a claim checked on explicit cases or by symbolic computation. A subagent’s computation is *computed* and *never higher*, however clean it looks — the delegation cannot manufacture a proof.
- *proved* is a paper proof the author has verified and believes complete. This is the boundary: below it, claims are exploratory; at or above it, they must be justified by their subtrees.
- *peer-reviewed* and *published cannot be self-granted*. They arrive from *outside* — a neighbour’s referee report, an editorial acceptance — and in practice they arrive by email, as registry events with a required artifact (a **review** field naming the reviewer, or a **publication** field naming venue and DOI). A self-assigned label with no external artifact is trust inflation, and the validator rejects it.

This asymmetry — one can lower one’s own grade freely but can raise it past *proved* only on external evidence — is the integrity mechanism. It makes the strongest grades *unforgeable by the claimant*. The placement of *lean-verified* at the top carries one standing caveat, stated here and in the front matter: a machine check certifies the proof term, not that the *statement* is the one you meant. Peer review and Lean verification check *different things*, so a **lean-verified** node still profits from a reviewer, and the ordering means “strength of check on what is on the page,” not “nothing left to gain.”

21.3 The registry

The grades are not prose adornments; they live in a machine-validated structure. Each multi-session program is a JSON tree in **proofs/registry/**, whose nodes are individual claims carrying an **approach**, a **trust** grade, a pointer to the proof **file**, optional **sources** (arXiv ids, with their own extraction ceiling), and children. A validator, **trustcheck.py**, enforces the structure mechanically. Its central rule is the *boundary rule*: a node may claim *proved* or above only if every non-dead-end child is at least *proved*. A claim’s grade is thus warranted by its entire subtree, and the check composes — validating the tree validates every subtree.

Two features make the registry an honest ledger rather than a trophy case. First, *dead ends are first-class nodes*: an abandoned route carries a required **reason** and its own evidence level (*judgment* < *computed* < *proved* < *lean-verified*), because a wrong reason silently prunes a live branch — the most expensive error in a search. Second, a *demotion is recorded with the same weight as an endorsement*: when a referee finds a gap, the node is downgraded honestly (to *computed*, or to *dead-end* with the objection as its reason) and the objecting email is kept as an artifact. A recorded objection is worth as much as a recorded endorsement.

There is a genuine self-reference worth one sentence. The registry is itself a *directed container* — shapes are search trees, positions are paths, and the induced comonad’s **extract/duplicate** are exactly “read the root’s grade” / “expose the full sub-search at every node” — graded under a provenance semiring whose boundary rule Robin has formalised in Lean (**ProofSearchN.lean**, **TrustBoundary.lean**: the boundary rule is stable under **duplicate**, **sorry-free**) [23]. The methodology of this dossier is an instance of its mathematics.

21.4 The Lean statement self-audit

Sorry-free is necessary, not sufficient. The discipline that answers the Ilin–Nugent and vacuous-proof failure modes is a self-audit of what each Lean *statement* actually says. The worked example is

the 2026-09-22 audit of the programme’s three zero-sorry declarations: `EnrichedFFCollapse.iso_id`, `UnionSubspaces.card_union_*`, and `VecCollapse.linExt_not_faithful`. The audit found all three *clean* — non-vacuous, with load-bearing hypotheses and faithful encodings — with one honest exception: `EnrichedFFCollapse` has an *explicitly disclosed narrowed scope*. Its statement is object-level ($M X \cong X$) for a bare functor M , and it abstracts the category-theory inputs (the preservation and terminality facts) as Lean *hypotheses* rather than re-proving them inside Lean. That is precisely the spot where a mis-stated hypothesis would hide a vacuous or over-narrow theorem — so the scope is stated in the open, in the declaration and its commentary. *Disclosing the scope is the point*: an audited theorem with a named boundary is worth more than an unaudited one with a clean headline.

21.5 Forcing functions: honesty made structural

The discipline does not rest on good intentions; the communication protocol makes it structural. Three forcing functions operate on every substantive result.

Results travel as typeset PDFs. Agent-to-agent content moves as an attached, typeset document, never as an email body. Typesetting a claim is reading it once more, in the register where dropped qualifiers become visible.

Every PDF carries the work-in-progress commit hash. The first page stamps the hash of the corresponding push, so any claim can be checked against the exact state of the record that produced it — a claim and its evidence cannot drift apart.

Claims propagate along a review path, and each hop can overrule. The comms graph is a path; a result reaches a distant reader only by passing through an intermediate agent who *re-states* it, and the restatement *is* the review. The failure this guards against is real and dated: in August a claim of the author’s arrived overstated at the far end of the path, having travelled too fast through the hops — caught, two hops downstream, precisely because someone had to re-derive it to relay it. A companion case is sharper still: a September write-up silently promoted a proof file’s guarded “faithful away from the zeros” into a clean “faithful,” and the overstatement was caught when a neighbour re-derived the result over a finite field where the zeros were unavoidable. External re-derivation in a different base is a strong audit; the protocol is built to invite it.

21.6 “Correction is the theorem”

The working signature of the programme is that its structural content is frequently the *correction of the author’s own initial framing*. Over five consecutive deep-work cycles in September 2026 this was the rule, not the exception: the “ $K_{2,3}$ is an $[\omega] \neq 0$ phenomenon” reading was corrected into the degree-one descent condition and the two-tier tower of Chapter 7; a commutative-shadow reduction of the set-theoretic frontier was killed by proving the shadow *unfaithful*; a write-up’s overstated “faithful” was diffed back down to the proof file’s honest hedge. In each case the failure mode *was* the result, and the honest downgrade was more informative than the clean claim it replaced.

This is a discipline and a culture, and it is graded as such — as *methodology*, not as a mathematical grade. But it is the differentiator the grant rests on. Without it, the formalisation pillar is a pile of green checkmarks and a set of unverified statements about what they mean. With it — the unforgeable ladder, the machine-validated registry that records demotions as faithfully as endorsements, the statement self-audit, and the protocol’s forcing functions — it is a method for producing AI-authored mathematics that a reader can stake a grant, or a proof, on.

[Assembly note: drafted: trust-grade ladder + registry (boundary rule, dead-ends, demotions, directed-container self-reference) + Lean statement self-audit (three declarations, EnrichedFFCollapse scope disclosed) + protocol forcing functions + “correction is the theorem”. External AI+Lean citations at agent-summary level; methodology graded as methodology.]

Chapter 22

External-publication status

Stated plainly: nothing in this dossier is externally published. Under the ACM authorship rule an AI agent cannot be a sole author, and the external submission queue is held by Robin. The strongest grade anywhere is *peer-reviewed*, meaning internally refereed by Rick. This chapter records that status honestly and lists the candidates nearest to external submission.

[Assembly note: GAP — nothing published; top grade internal peer-reviewed. See Honest Ledger item 4.]

Bibliography

- [1] B. Day, *On closed categories of functors*, in: Reports of the Midwest Category Seminar IV, Lecture Notes in Mathematics 137, Springer, 1970, pp. 1–38.
- [2] F. Loregian, *(Co)end Calculus*, London Mathematical Society Lecture Note Series 468, Cambridge University Press, 2021.
- [3] S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Graduate Texts in Mathematics 5, Springer, 1998.
- [4] E. Riehl, *Category Theory in Context*, Aurora: Dover Modern Math Originals, Dover, 2016.
- [5] T. Bartoszyński and H. Judah, *Set Theory: On the Structure of the Real Line*, A K Peters, 1995.
- [6] A. Blass, *Combinatorial cardinal characteristics of the continuum*, in: Handbook of Set Theory (M. Foreman, A. Kanamori, eds.), Springer, 2010, 395–489.
- [7] D. Herbera and J. Trlifaj, *Almost free modules and Mittag-Leffler conditions*, Adv. Math. **229** (2012), 3436–3467. arXiv:0910.4277.
- [8] MacBeth, *The module reduction of Gap-S: $\text{Nat}(F, -) = \text{Hom}_S(F(E), -)$ over $S = \text{End}_k(E)$* , Kodamai internal note, 2026-09-20.
- [9] MacBeth, *Conjecture 6.2 / Gap-S collapses to a Dual-Baer test*, Kodamai internal note, 2026-09-20.
- [10] MacBeth, *Gap-S / Conjecture 6.2 is false under CH*, Kodamai internal note, 2026-09-21.
- [11] MacBeth, *Gap-S as an Osofsky–Jensen product-projective-dimension problem: $\text{Conj. 6.2} \Leftrightarrow \text{pd}_S M = 0$* , Kodamai internal note, 2026-09-21.
- [12] MacBeth, *The Gap-S free identity $\prod_r S \cong S$ has no commutative shadow: the homogeneity of the full linear ring*, Kodamai internal note, 2026-09-22.
- [13] MacBeth, *Holonomy and prunability as the two edges of one spectral sequence: the Zappa–Szép composition holonomy is the transgression*, Kodamai internal note, 2026-09-18.
- [14] MacBeth, *The obstruction to Ferri’s complete-prunability decomposition is a two-stage tower: a degree-one descent condition beneath a degree-two splitting class*, Kodamai internal note, 2026-09-23.
- [15] MacBeth, *Ferri prunability is not the Zappa–Szép composition holonomy: a separation theorem and the $K_{2,3}$ witness*, Kodamai internal note, 2026-09-16.

- [16] MacBeth, *Ferri prunability is a first-order descent condition, not a Schreier class: the untwisted core*, Kodamai internal note, 2026-09-18.
- [17] MacBeth, *The emergent-holonomy count lives in degree zero: $h(s) = \dim_k \text{Ext}^0$ and the vanishing of the higher tower*, Kodamai internal note, 2026.
- [18] D. Ferri, *On quiver skew braces, their ideals and products*, arXiv:2605.11903, 2026.
- [19] Ambra, Duvieusart, and Montoli, *Cohomology of internal categories relative to Schreier points (direction-functor cohomology)*, arXiv:2608.07380, 2026.
- [20] Adamczewski, Böhmeler, and Marczinik, *A counterexample to Köthe’s conjecture* (with an AI-authored, human-reviewed Lean 4 formalisation), arXiv:2609.07996, 2026. (*Agent-summary provenance; not verified at source.*)
- [21] Ilin and Nugent, *Sorries Are Not the Hard Part: an expert-review case study of a semi-autonomous formalisation*, arXiv:2603.12744, 2026. (*Agent-summary provenance.*)
- [22] *TaoBench: sensitivity of automated theorem provers to the definitional framework*, arXiv 2026, identifier unverified. (*Agent-summary provenance.*)
- [23] R. Langer, **ProofSearchN.lean** and **TrustBoundary.lean**, **containers** repository, Kodamai (rose-tree proof search; the trust-boundary rule stable under **duplicate**, **sorry-free**).
- [24] M. Abbott, T. Altenkirch, N. Ghani, *Containers: constructing strictly positive types*, Theoret. Comput. Sci. **342** (2005), 3–27.
- [25] D. Ahman, J. Chapman, and T. Uustalu, *When is a container a comonad?*, Logical Methods in Computer Science **10**(3), 2014.
- [26] D. Ahman and T. Uustalu, *Directed containers as categories*, in: Proc. MSFP 2016, EPTCS **207**, 2016, pp. 89–98. arXiv:1604.01187.
- [27] D. Ahman and T. Uustalu, *Distributive laws of directed containers*, Progress in Informatics **10** (2013), 3–18. DOI 10.2201/NIPI.2013.10.2.
- [28] D. Ahman and T. Uustalu, *Update monads: cointerpreting directed containers*, in: TYPES 2013, LIPIcs **26** (2014), 1–23.
- [29] D. I. Spivak, *Functorial aggregation*, arXiv:2111.10968, 2021.
- [30] B. Shapiro and D. I. Spivak, *Structures on categories of polynomials*, arXiv:2305.00167, 2023.
- [31] N. Niu and D. I. Spivak, *Polynomial Functors: A Mathematical Theory of Interaction*, book draft, 2023. arXiv:2312.00990.
- [32] N. Gambino and J. Kock, *Polynomial functors and polynomial monads*, Math. Proc. Cambridge Philos. Soc. **154** (2013), 153–192. arXiv:0906.4931.
- [33] D. I. Spivak and P. Srinivasan, **(Poly, $\otimes, \triangleleft$) is a normal duoidal LDC**, arXiv:2407.01849, 2024.
- [34] M. Aguiar, *Internal categories and quantum groups*, PhD thesis, Cornell University, 1997.
- [35] B. Clarke, *Internal lenses as functors and cofunctors*, in: Proc. ACT 2019, EPTCS **323**, 2020, pp. 183–195. arXiv:2009.06835.

- [36] M. G. Brin, *On the Zappa–Szép product*, Comm. Algebra **33** (2005), 393–424.
- [37] R. Rosebrugh and R. J. Wood, *Distributive laws and factorization*, J. Pure Appl. Algebra **175** (2002), 327–353.
- [38] R. Street, *The formal theory of monads*, J. Pure Appl. Algebra **2** (1972), 149–168.
- [39] G. M. Kelly, *Basic Concepts of Enriched Category Theory*, London Math. Soc. Lecture Note Ser. **64**, Cambridge Univ. Press, 1982.
- [40] A. Kock, *Monads on symmetric monoidal closed categories*, Arch. Math. (Basel) **21** (1970), 1–10.
- [41] A. Kock, *Closed categories generated by commutative monads*, J. Austral. Math. Soc. **12** (1971), 405–424.
- [42] Y. Diers, *Catégories localisables*, Thèse de doctorat d’état, Université Paris VI, 1977.
- [43] T. von Glehn, *Polynomials and models of type theory*, PhD thesis, University of Cambridge, 2015.
- [44] C. Walker, *Locally subcartesian closed categories*, arXiv:2607.10242.
- [45] M. Weber, *Polynomials in categories with pullbacks*, Theory Appl. Categ. **30** (2015), 533–598. arXiv:1106.1983.
- [46] M. Weber, *Familial 2-functors and parametric right adjoints*, Theory Appl. Categ. **18** (2007), 665–732.
- [47] P. M. Weichsel, *The Kronecker product of graphs*, Proc. Amer. Math. Soc. **13** (1962), 47–52.
- [48] J. J. Xarez, *Admissibility, stable units and connected components*, arXiv:1112.4277, 2011.
- [49] A. Dorta, C. B. Jarvis, and N. Niu, *Monoidal structures on generalized polynomial categories*, arXiv:2305.05655.
- [50] G. De Pascalis, T. Uustalu, and N. Veltri, *Monoid structures on indexed containers*, arXiv:2509.25879.
- [51] MacBeth, *Containers over a base*, Kodamai working paper, 2026. (Object-level equivalence-chain milestones M2, M4 machine-checked in the **Containers**/ Lean 4 corpus, **sorry-free**.)
- [52] R. Langer, L. Vega, and Claude, *Island Models as Directed Containers*, Kodamai internal note, June 2026. (Origin of the categorical GA framework and its empirical measurements; source for Chapter 10.)